



UNIVERSITAS INDONESIA

**PENGEMBANGAN *JOINT DOCUMENT LEVEL SENTIMENT-TOPIC MODEL*
BERBASIS *EXTENDED LONG SHORT-TERM MEMORY* UNTUK ANALISIS
SENTIMEN**

SKRIPSI

HAIKAL FIKRI RABANI

2206823713

**FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
PROGRAM STUDI SARJANA STATISTIKA
DEPOK
JUNI 2026**



UNIVERSITAS INDONESIA

**PENGEMBANGAN *JOINT DOCUMENT LEVEL SENTIMENT-TOPIC MODEL*
BERBASIS *EXTENDED LONG SHORT-TERM MEMORY* UNTUK ANALISIS
SENTIMEN**

SKRIPSI

**Diajukan sebagai salah satu syarat memperoleh gelar
Sarjana Statistika**

HAIKAL FIKRI RABANI

2206823713

**FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
PROGRAM STUDI SARJANA STATISTIKA
DEPOK
JUNI 2026**

HALAMAN PERNYATAAN ORISINALITAS

**Skripsi ini adalah hasil karya saya sendiri,
dan semua sumber baik yang dikutip maupun dirujuk
telah saya nyatakan dengan benar.**

Nama : Haikal Fikri Rabani

NPM : 2206823713

Tanda Tangan



Tanggal : 26 Juni 2026

HALAMAN PENGESAHAN

Skripsi ini diajukan oleh :

Nama : Haikal Fikri Rabani
NPM : 2206823713
Program Studi : Sarjana Statistika
Judul Skripsi : Pengembangan *Joint Document Level Sentiment-Topic Model*
Berbasis *Extended Long Short-Term Memory* untuk Analisis
Sentimen

Telah berhasil dipertahankan di hadapan Dewan Penguji dan diterima sebagai bagian persyaratan yang diperlukan untuk memperoleh gelar Sarjana Statistika pada Program Studi Sarjana Statistika, Fakultas Matematika dan Ilmu Pengetahuan Alam, Universitas Indonesia.

DEWAN PENGUJI

Pembimbing I : Prof. Dr. rer. nat. Hendri Murfi, S.Si., M.Kom. ()

Pembimbing II : Dra. Nora Hariadi, M.Si. ()

Penguji I : Sarini Abdullah, S.Si., M.Stats., Ph.D ()

Penguji II : Devvi Sarwinda, M.Kom. ()

Ditetapkan di: Depok
Tanggal : 26 Juni 2026

HALAMAN PERNYATAAN PENGGUNAAN KECERDASAN ARTIFISIAL

Saya menyatakan bahwa dalam penyusunan Tugas Akhir ini, saya menggunakan teknologi Kecerdasan Buatan dengan *platform* ChatGPT untuk tujuan penulisan dan saya bertanggung jawab penuh atas keakuratan, orisinalitas, dan integritas seluruh isi Tugas Akhir ini.

Nama : Haikal Fikri Rabani

NPM : 2206823713

Tanda Tangan : 

Tanggal : 26 Juni 2026

KATA PENGANTAR

Puji syukur kepada Tuhan Yang Maha Esa atas segala berkat dan rahmat-Nya, sehingga penulis dapat menyelesaikan skripsi ini dengan sebaik-baiknya. Selama masa penulisan skripsi ini, penulis telah mendapat banyak dukungan, doa, bantuan, inspirasi, dan motivasi dari berbagai pihak. Untuk itu, penulis ingin berterima kasih kepada:

1. Bapak Prof. Dr. rer. nat. Hendri Murfi, S.Si., M.Kom. selaku Dosen Pembimbing I yang telah membimbing penulis dengan penuh kesabaran serta memberikan arahan, masukan, dan ilmu yang sangat berharga selama proses penyusunan skripsi ini. Waktu, perhatian, dan dedikasi yang beliau berikan sangat membantu penulis dalam memahami materi serta menyelesaikan penelitian ini dengan baik.
2. Ibu Dra. Nora Hariadi, M.Si. selaku Dosen Pembimbing II yang telah memberikan arahan dan masukan terkait tata cara penulisan skripsi yang sesuai dengan pedoman yang berlaku. Bimbingan beliau sangat membantu penulis dalam menyusun karya ilmiah yang sistematis dan sesuai kaidah akademik.
3. Ibu Sarini Abdullah, S.Si., M.Stats., Ph.D selaku Dosen Penguji I dan Ketua Departemen Matematika Fakultas Matematika dan Ilmu Pengetahuan Alam Universitas Indonesia yang telah meluangkan waktu untuk mengevaluasi skripsi ini serta memberikan kritik dan saran yang konstruktif. Masukan yang diberikan sangat bermanfaat dalam penyempurnaan penelitian ini.
4. Ibu Devvi Sarwinda, M.Kom. selaku Dosen Penguji II yang telah memberikan evaluasi, kritik, dan saran yang membangun sehingga skripsi ini dapat menjadi lebih baik dan layak untuk dipublikasikan.
5. Ibu Gianinna Ardanewari, S.Si., M.Si. selaku Ketua Program Studi Sarjana Statistika Universitas Indonesia yang telah memberikan dukungan, arahan, serta menciptakan lingkungan akademik yang kondusif sehingga penulis dapat menempuh dan menyelesaikan studi dengan baik.
6. Ibu Kurnia Susvitasari, S.Si., M.Sc., Ph.D. selaku Pembimbing Akademis penulis yang telah memberikan bimbingan, motivasi, serta arahan selama masa perkuliahan. Dukungan beliau sangat berarti dalam pembelajaran akademik penulis.

7. Ibu Prof. Dr. Titin Siswantining, D.E.A. selaku Ketua Pojok Statistik yang telah memberikan ilmu, pengalaman, serta kesempatan berharga dalam pengembangan diri penulis, sehingga membentuk penulis menjadi pribadi yang lebih baik secara akademik maupun personal.
8. Seluruh dosen Departemen Matematika yang telah memberikan ilmu pengetahuan, wawasan, dan pengalaman selama masa perkuliahan, yang menjadi bekal penting bagi penulis dalam menyelesaikan skripsi ini.
9. Keluarga tercinta, yaitu Bapak Eka Adi Susila, Ibu Else Dwi Listiani, dan Adik Miftahul Rizqi Rabani, yang senantiasa memberikan doa, dukungan moral, serta semangat yang tiada henti. Kehadiran dan dukungan mereka menjadi sumber kekuatan utama bagi penulis dalam menyelesaikan studi ini.
10. Ester Oktafia yang telah memberikan dukungan, masukan, serta semangat kepada penulis selama proses penyusunan skripsi ini, sehingga penulis dapat tetap termotivasi dalam menghadapi berbagai tantangan.
11. Teman-teman seperjuangan dari grup "Ujang Kedu" yang telah kebersamaian penulis sejak awal perkuliahan hingga tahap akhir studi. Kebersamaan, dukungan, dan pengalaman yang dibagikan menjadi bagian penting dalam perjalanan akademik penulis.
12. Teman-teman dari grup "Citsit Kompstat" yang telah berperan aktif dalam kegiatan belajar bersama, khususnya dalam mempersiapkan ujian, sehingga membantu penulis dalam memahami materi perkuliahan dengan lebih baik.
13. Seluruh *mentee* dari kelompok "Piecewise" yang telah memberikan semangat dan motivasi kepada penulis, serta menjadi pengingat untuk terus berkembang dan menyelesaikan tanggung jawab akademik dengan baik.
14. Tri Hidayatullah dan Edwin Daniel selaku adik asuh di Departemen Matematika yang telah memberikan dukungan serta kebersamaan selama masa perkuliahan.
15. Seluruh pihak yang telah terlibat dan memberikan kontribusi dalam penelitian ini yang tidak dapat disebutkan satu per satu, baik secara langsung maupun tidak langsung.

Penulis menyadari bahwa penyusunan skripsi ini tidak terlepas dari berbagai keterbatasan dan masih memiliki kekurangan, baik dari segi penulisan maupun substansi

keilmuan. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun sebagai bahan evaluasi untuk perbaikan di masa yang akan datang. Besar harapan penulis agar skripsi ini dapat memberikan manfaat, baik sebagai referensi akademik maupun sebagai kontribusi dalam pengembangan ilmu pengetahuan, khususnya di bidang yang terkait dengan penelitian ini.

Depok, 26 Juni 2026

Haikal Fikri Rabani

HALAMAN PERNYATAAN PERSETUJUAN PUBLIKASI TUGAS AKHIR UNTUK KEPENTINGAN AKADEMIS

Sebagai sivitas akademik Universitas Indonesia, saya yang bertanda tangan di bawah ini:

Nama : Haikal Fikri Rabani
NPM : 2206823713
Program Studi : Sarjana Statistika
Departemen : Matematika
Fakultas : Matematika dan Ilmu Pengetahuan Alam
Jenis Karya : Skripsi

demikian pengembangan ilmu pengetahuan, menyetujui untuk memberikan kepada Universitas Indonesia **Hak Bebas Royalti Noneksklusif** (*Non-exclusive Royalty Free Right*) atas karya ilmiah saya yang berjudul:

Pengembangan *Joint Document Level Sentiment-Topic Model* Berbasis *Extended Long Short-Term Memory* untuk Analisis Sentimen

berserta perangkat yang ada (jika diperlukan). Dengan Hak Bebas Royalti Noneksklusif ini, Universitas Indonesia berhak menyimpan, mengalihmedia/formatkan, mengelola dalam bentuk pangkalan data (*data base*), merawat, dan memublikasikan tugas akhir saya selama tetap mencantumkan nama saya sebagai penulis/pencipta dan sebagai pemilik Hak Cipta.

Demikian pernyataan ini saya buat dengan sebenarnya.

Dibuat di : Depok
Pada tanggal : 26 Juni 2026

Yang menyatakan



(Haikal Fikri Rabani)

ABSTRAK

Nama : Haikal Fikri Rabani

Program Studi : Sarjana Statistika

Judul : Pengembangan *Joint Document Level Sentiment-Topic Model* Berbasis *Extended Long Short-Term Memory* untuk Analisis Sentimen

Peningkatan volume data teks tidak terstruktur dari berbagai platform digital, seperti ulasan produk dan media sosial, mendorong kebutuhan akan metode analisis otomatis untuk mengekstrak opini publik secara efisien. Analisis sentimen menjadi pendekatan utama dalam *Natural Language Processing* (NLP) untuk mengidentifikasi polaritas opini dalam teks. Namun, pendekatan konvensional pada tingkat dokumen mengasumsikan satu polaritas untuk setiap dokumen, meskipun dalam praktiknya satu dokumen bisa memuat beberapa topik dengan sentimen yang berbeda-beda. Pendekatan yang lebih rinci, seperti *topic-level sentiment analysis*, mampu menangani kondisi tersebut, tetapi memerlukan proses anotasi dan pemodelan yang lebih kompleks. Oleh karena itu, berkembang *Joint Document Level Sentiment-Topic Model* (JDST) sebagai alternatif antara pendekatan sederhana dan analisis yang lebih rinci, yang mempelajari topik dan sentimen secara simultan dalam satu kerangka terpadu. Penelitian ini mengusulkan model JDST berbasis *Extended Long Short-Term Memory* (xLSTM) dengan pendekatan *Multi-Task Learning* (MTL) dan skema *hard parameter sharing*. Model memanfaatkan xLSTM sebagai model representasi teks yang terlebih dahulu dilatih bersama *encoder* dan *decoder* dalam kerangka *autoencoder* melalui rekonstruksi *embedding* untuk menghasilkan representasi token yang informatif. Pada tahap *joint training*, bobot xLSTM dibekukan dan representasi token yang dihasilkan diproses oleh *encoder* untuk membentuk representasi bersama yang digunakan secara simultan oleh klasifikasi sentimen melalui MLP dan pendeteksian topik melalui *Deep Embedded K-Means* (DEKM). Evaluasi dilakukan pada *dataset* IMDB dan Yelp dengan variasi jumlah topik ($K = 30, 50, 80$) dan empat konfigurasi xLSTM (1:0, 0:1, 1:1 - ms dengan urutan mLSTM→sLSTM, dan 1:1 - sm dengan urutan sLSTM→mLSTM), dengan JDST-LSTM-1L (*satu layer*) dan JDST-LSTM-2L (*dua layer*) sebagai model pembanding. Hasil eksperimen menunjukkan bahwa seluruh konfigurasi JDST-xLSTM mengungguli kedua konfigurasi JDST-LSTM pada tugas analisis sentimen di kedua *dataset*. JDST-xLSTM[1:1]-ms memperoleh performa terbaik pada mayoritas skenario, dengan *accuracy* tertinggi sebesar $0,8875 \pm 0,0013$ pada IMDB ($K = 80$) dan $0,9201 \pm 0,0010$ pada Yelp ($K = 50$). Pada tugas pendeteksian topik, JDST-xLSTM[1:1]-ms juga mengungguli seluruh model pada metrik TC-W2V di *dataset* IMDB untuk semua nilai K , sedangkan pada Yelp, JDST-xLSTM[1:1]-sm menunjukkan keunggulan pada $K = 30$ dan $K = 50$. Secara keseluruhan, konfigurasi xLSTM menghasilkan performa yang lebih tinggi sekaligus lebih stabil dibandingkan *baseline* LSTM, dengan JDST-xLSTM[1:1]-ms sebagai konfigurasi terbaik secara keseluruhan.

Kata kunci: *Deep Embedded K-Means, Extended Long Short-Term Memory, Joint Document Level Sentiment-Topic Model, Multi-Task Learning.*

EXTENDED ABSTRACT

Undergraduate Thesis

Development of a Joint Document Level Sentiment-Topic Model Based on Extended Long Short-Term Memory for Sentiment Analysis

Student Name : Haikal Fikri Rabani

Student ID Number (NPM) : 2206823713

Study Program : Undergraduate Study Program of Statistics

Every day, millions of people share opinions through social media, product reviews, and online news platforms. Extracting useful information from this large amount of text has become an important challenge in Natural Language Processing (NLP). One of the most widely used approaches is sentiment analysis, which aims to identify whether a text expresses a positive or negative opinion. At the document level, sentiment analysis is usually treated as a binary classification task. However, this approach assumes that a document reflects only one overall sentiment. In practice, a single review often discusses several aspects of a product or service and may express different opinions about each of them. As a result, reducing an entire review to a single sentiment label can overlook important information about what the writer actually likes or dislikes. While topic-level approaches can capture this finer-grained information, they typically require more complex annotation and modeling, motivating the development of Joint Document Level Sentiment-Topic Modeling as a middle ground between the two.

To address this limitation, this study proposes a Joint Document Level Sentiment-Topic (JDST) model based on Extended Long Short-Term Memory (xLSTM) within a Multi-Task Learning (MTL) framework. Unlike the approach proposed by Gui *et al.* (2022), where sentiment classification and topic modeling are learned through separate encoders connected via mutual learning, the proposed model adopts a hard parameter sharing strategy. xLSTM serves as the text representation model, which is first pre-trained together with an encoder and decoder in an autoencoder framework through embedding reconstruction to produce informative token representations. During the joint training stage, the xLSTM weights are frozen and the token representations are processed by the encoder to form a shared representation, which is then used simultaneously by a sentiment classification head (MLP) and a topic detection module based on Deep Embedded K-Means (DEKM), an unsupervised clustering method that operates directly on the shared representation. The overall workflow of the proposed framework is shown in Figure 1.

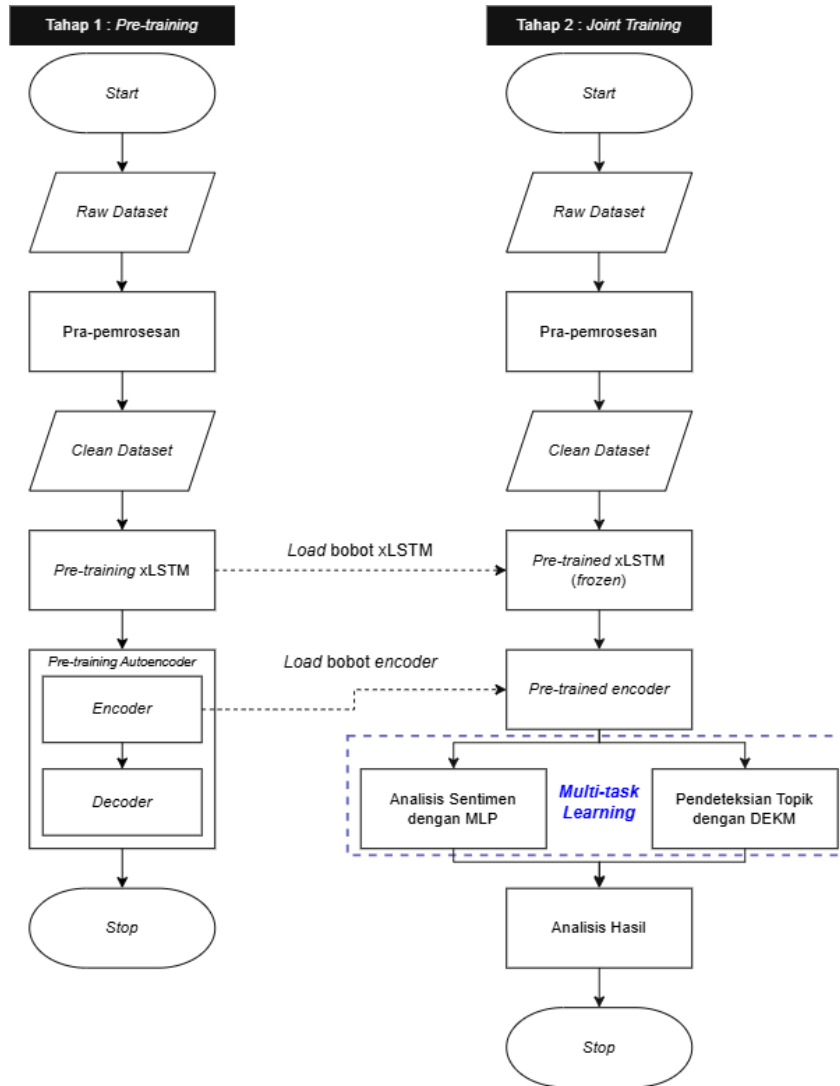


Figure 1: Schematic Diagram of Research Framework

The experiments were conducted on the IMDB Movie Reviews and Yelp Polarity datasets, each containing 50,000 reviews with binary sentiment labels. Four xLSTM configurations were evaluated, namely xLSTM[1:0], xLSTM[0:1], xLSTM[1:1] - ms with mLSTM→sLSTM order, and xLSTM[1:1] - sm with sLSTM→mLSTM order, while JDST-LSTM-1L (one layer) and JDST-LSTM-2L (two layer) was used as the baseline model. Topic quality was examined using three topic settings ($K = 30, 50, 80$), and each experiment was repeated three times with different random seeds. Sentiment classification performance was measured using accuracy, precision, recall, and F1-score, whereas topic quality was evaluated using Topic Coherence based on word embeddings (TC-W2V) and Topic Diversity (TD).

The pre-training results show that all xLSTM and LSTM variants achieve lower reconstruction loss than both LSTM variants. Among them, xLSTM[1:1]-ms achieves

the lowest validation MSE of 0.0027, followed closely by xLSTM[1:1]-sm with 0.0028, whereas LSTM-2L records the highest validation MSE of 0.0122. These results indicate that the xLSTM architectures learn more effective latent representations during the pre-training stage, leading to better reconstruction performance than the conventional LSTM models.

For sentiment classification, JDST-xLSTM[1:1]-ms delivers the best performance on the IMDB dataset across all values of K , achieving an accuracy of 0.8827 ± 0.0016 at $K = 30$, 0.8855 ± 0.0026 at $K = 50$, and reaching the highest value of 0.8875 ± 0.0013 at $K = 80$. On the Yelp dataset, JDST-xLSTM[1:1]-ms also achieves the best performance at $K = 30$ and $K = 50$, with the overall highest accuracy of 0.9201 ± 0.0010 at $K = 50$, while at $K = 80$ the best result is obtained by JDST-xLSTM[1:0] with an accuracy of 0.9127 ± 0.0009 . Both LSTM baselines consistently underperform compared to all xLSTM configurations on both datasets. JDST-LSTM-2L obtains a maximum accuracy of 0.8680 ± 0.0031 on IMDB and 0.9114 ± 0.0016 on Yelp, while exhibiting higher variance across runs. These results indicate that the exponential gating and matrix memory mechanisms in xLSTM produce more informative token representations for sentiment classification than conventional LSTM architectures.

The topic detection results exhibit a more varied pattern. On the IMDB dataset, JDST-xLSTM[1:1]-ms achieves the highest TC-W2V across all values of K , with scores of 0.3946 ± 0.0177 at $K = 30$, 0.3867 ± 0.0035 at $K = 50$, and 0.3723 ± 0.0097 at $K = 80$. This configuration also obtains the highest Topic Diversity at $K = 30$ and $K = 50$, with values of 0.3798 ± 0.0240 and 0.3646 ± 0.0100 , indicating that the generated topics are both semantically coherent and distinct from one another. At $K = 80$, JDST-LSTM-2L achieves the highest Topic Diversity (0.3645 ± 0.2166), although with substantially higher variance. On the Yelp dataset, JDST-xLSTM[1:1]-sm achieves the highest TC-W2V at $K = 30$ (0.3518 ± 0.0129) and $K = 50$ (0.2750 ± 0.0357), as well as the highest Topic Diversity at $K = 30$ (0.3933 ± 0.0661). At $K = 80$, the best TC-W2V is obtained by JDST-xLSTM[0:1] (0.2852 ± 0.0153). Both LSTM baselines produce consistently lower TC-W2V scores and exhibit larger standard deviations compared to the xLSTM configurations, particularly on the Yelp dataset where JDST-LSTM-2L obtains TC-W2V values as low as 0.1535 ± 0.0208 at $K = 80$.

Overall, the results demonstrate that replacing LSTM with xLSTM in the proposed end-to-end JDST framework substantially improves both sentiment classification and topic detection performance. Among the xLSTM configurations, JDST-xLSTM[1:1]-ms achieves the strongest results on sentiment classification across both datasets, while topic detection performance varies across configurations depending on the dataset and the

number of topics. The two-block configurations ([1:1]-ms and [1:1]-sm), which combine mLSTM and sLSTM blocks, consistently outperform the single-block configurations on sentiment classification, suggesting that the combination of scalar and matrix memory mechanisms produces richer representations. Furthermore, xLSTM configurations generally exhibit lower variance across runs compared to the LSTM baselines, indicating more stable and reliable performance.

In conclusion, the proposed xLSTM-based JDST model is effective for jointly learning document-level sentiment and topics in an end-to-end manner. JDST-xLSTM[1:1]-ms achieves the strongest sentiment classification performance across most settings, while JDST-xLSTM[1:1]-sm shows particular strength in topic coherence on the Yelp dataset. All xLSTM configurations consistently outperform the LSTM baselines on both tasks. Future studies may explore systematic methods for determining the optimal number of topics K , investigate multilingual datasets, or extend the framework to finer-grained sentiment analysis tasks.

Table 1: Comparison of training loss and validation loss during pre-training

<i>Epoch</i>	LSTM-1L		LSTM-2L		xLSTM[1:0]		xLSTM[0:1]		xLSTM[1:1]-sm		xLSTM[1:1]-ms	
	latih	val	latih	val	latih	val	latih	val	latih	val	latih	val
1	0,1509	0,0554	0,1616	0,0680	0,1431	0,0429	0,1415	0,0453	0,1477	0,0435	0,1506	0,0446
2	0,0402	0,0277	0,0501	0,0356	0,0319	0,0194	0,0341	0,0227	0,0325	0,0206	0,0325	0,0211
3	0,0259	0,0193	0,0343	0,0272	0,0195	0,0120	0,0217	0,0150	0,0204	0,0138	0,0207	0,0141
4	0,0202	0,0153	0,0284	0,0226	0,0144	0,0084	0,0164	0,0110	0,0156	0,0106	0,0159	0,0108
5	0,0169	0,0125	0,0248	0,0195	0,0117	0,0064	0,0134	0,0086	0,0128	0,0081	0,0132	0,0088
6	0,0147	0,0105	0,0223	0,0174	0,0100	0,0052	0,0114	0,0070	0,0110	0,0067	0,0114	0,0073
7	0,0132	0,0092	0,0206	0,0158	0,0090	0,0045	0,0101	0,0059	0,0097	0,0058	0,0101	0,0062
8	0,0121	0,0082	0,0193	0,0146	0,0083	0,0040	0,0091	0,0051	0,0089	0,0051	0,0092	0,0056
9	0,0113	0,0075	0,0183	0,0138	0,0078	0,0037	0,0085	0,0046	0,0083	0,0047	0,0086	0,0051
10	0,0108	0,0070	0,0176	0,0132	0,0075	0,0034	0,0080	0,0043	0,0073	0,0044	0,0082	0,0043
11	0,0104	0,0067	0,0171	0,0128	0,0072	0,0032	0,0077	0,0041	0,0069	0,0035	0,0079	0,0033
12	0,0102	0,0065	0,0167	0,0125	0,0071	0,0031	0,0075	0,0039	0,0068	0,0032	0,0073	0,0030
13	0,0100	0,0064	0,0165	0,0123	0,0069	0,0030	0,0074	0,0038	0,0066	0,0030	0,0068	0,0029
14	0,0099	0,0063	0,0164	0,0123	0,0069	0,0030	0,0073	0,0038	0,0066	0,0028	0,0065	0,0027
15	0,0099	0,0063	0,0164	0,0122	0,0069	0,0030	0,0073	0,0038	0,0065	0,0028	0,0065	0,0027

Table 2: Sentiment analysis evaluation results on the IMDB dataset.

<i>Model</i>	<i>K</i>	<i>Accuracy</i>	<i>F1-Macro</i>	<i>Precision</i>	<i>Recall</i>
JDST-LSTM-1L	30	0,8637 $\pm$ 0,0104	0,8637 $\pm$ 0,0104	0,8642 $\pm$ 0,0104	0,8637 $\pm$ 0,0104
	50	0,8599 $\pm$ 0,0057	0,8597 $\pm$ 0,0056	0,8612 $\pm$ 0,0064	0,8599 $\pm$ 0,0057
	80	0,8696 $\pm$ 0,0029	0,8696 $\pm$ 0,0029	0,8701 $\pm$ 0,0032	0,8696 $\pm$ 0,0029
JDST-LSTM-2L	30	0,8680 $\pm$ 0,0031	0,8678 $\pm$ 0,0033	0,8710 $\pm$ 0,0009	0,8680 $\pm$ 0,0031
	50	0,8616 $\pm$ 0,0027	0,8610 $\pm$ 0,0029	0,8671 $\pm$ 0,0015	0,8616 $\pm$ 0,0027

<i>Model</i>	<i>K</i>	<i>Accuracy</i>	<i>F1-Macro</i>	<i>Precision</i>	<i>Recall</i>
	80	0,8561 $\pm$ 0,0063	0,8557 $\pm$ 0,0061	0,8603 $\pm$ 0,0082	0,8561 $\pm$ 0,0063
JDST- xLSTM[0:1]	30	0,8760 $\pm$ 0,0007	0,8759 $\pm$ 0,0008	0,8773 $\pm$ 0,0004	0,8760 $\pm$ 0,0007
	50	0,8733 $\pm$ 0,0028	0,8732 $\pm$ 0,0029	0,8740 $\pm$ 0,0023	0,8733 $\pm$ 0,0028
	80	0,8754 $\pm$ 0,0011	0,8754 $\pm$ 0,0011	0,8755 $\pm$ 0,0012	0,8754 $\pm$ 0,0011
JDST- xLSTM[1:0]	30	0,8704 $\pm$ 0,0025	0,8703 $\pm$ 0,0025	0,8710 $\pm$ 0,0029	0,8704 $\pm$ 0,0025
	50	0,8772 $\pm$ 0,0007	0,8772 $\pm$ 0,0007	0,8776 $\pm$ 0,0008	0,8772 $\pm$ 0,0007
	80	0,8783 $\pm$ 0,0027	0,8783 $\pm$ 0,0027	0,8786 $\pm$ 0,0026	0,8783 $\pm$ 0,0027
JDST-xLSTM [1:1] – sm	30	0,8761 $\pm$ 0,0060	0,8760 $\pm$ 0,0062	0,8773 $\pm$ 0,0046	0,8761 $\pm$ 0,0060
	50	0,8795 $\pm$ 0,0049	0,8794 $\pm$ 0,0049	0,8798 $\pm$ 0,0047	0,8795 $\pm$ 0,0049
	80	0,8830 $\pm$ 0,0018	0,8830 $\pm$ 0,0018	0,8833 $\pm$ 0,0016	0,8830 $\pm$ 0,0018
JDST-xLSTM [1:1] – ms	30	0,8827 $\pm$ 0,0016	0,8827 $\pm$ 0,0016	0,8833 $\pm$ 0,0015	0,8827 $\pm$ 0,0016
	50	0,8855 $\pm$ 0,0026	0,8855 $\pm$ 0,0027	0,8862 $\pm$ 0,0023	0,8855 $\pm$ 0,0026
	80	0,8875 $\pm$ 0,0013	0,8875 $\pm$ 0,0013	0,8879 $\pm$ 0,0009	0,8875 $\pm$ 0,0013

Table 3: Sentiment analysis evaluation results on the Yelp dataset.

<i>Model</i>	<i>K</i>	<i>Accuracy</i>	<i>F1-Macro</i>	<i>Precision</i>	<i>Recall</i>
JDST- LSTM-1L	30	0,8967 $\pm$ 0,0019	0,8967 $\pm$ 0,0019	0,8977 $\pm$ 0,0020	0,8967 $\pm$ 0,0019
	50	0,9051 $\pm$ 0,0095	0,9050 $\pm$ 0,0096	0,9057 $\pm$ 0,0089	0,9051 $\pm$ 0,0095
	80	0,9031 $\pm$ 0,0076	0,9031 $\pm$ 0,0076	0,9035 $\pm$ 0,0074	0,9031 $\pm$ 0,0076
JDST- LSTM-2L	30	0,9019 $\pm$ 0,0080	0,9019 $\pm$ 0,0081	0,9024 $\pm$ 0,0076	0,9019 $\pm$ 0,0080
	50	0,8957 $\pm$ 0,0107	0,8957 $\pm$ 0,0107	0,8960 $\pm$ 0,0109	0,8957 $\pm$ 0,0107
	80	0,9114 $\pm$ 0,0016	0,9114 $\pm$ 0,0016	0,9118 $\pm$ 0,0013	0,9114 $\pm$ 0,0016
JDST- xLSTM[0:1]	30	0,9105 $\pm$ 0,0029	0,9105 $\pm$ 0,0029	0,9108 $\pm$ 0,0027	0,9105 $\pm$ 0,0029
	50	0,9065 $\pm$ 0,0079	0,9064 $\pm$ 0,0079	0,9069 $\pm$ 0,0081	0,9065 $\pm$ 0,0079
	80	0,9096 $\pm$ 0,0023	0,9096 $\pm$ 0,0023	0,9101 $\pm$ 0,0022	0,9096 $\pm$ 0,0023
JDST- xLSTM[1:0]	30	0,9106 $\pm$ 0,0006	0,9105 $\pm$ 0,0006	0,9110 $\pm$ 0,0008	0,9106 $\pm$ 0,0006
	50	0,9102 $\pm$ 0,0009	0,9102 $\pm$ 0,0009	0,9106 $\pm$ 0,0007	0,9102 $\pm$ 0,0009
	80	0,9127 $\pm$ 0,0009	0,9126 $\pm$ 0,0009	0,9130 $\pm$ 0,0010	0,9127 $\pm$ 0,0009
JDST-xLSTM [1:1]-sm	30	0,9191 $\pm$ 0,0013	0,9191 $\pm$ 0,0013	0,9192 $\pm$ 0,0014	0,9191 $\pm$ 0,0013
	50	0,9117 $\pm$ 0,0099	0,9117 $\pm$ 0,0099	0,9122 $\pm$ 0,0096	0,9117 $\pm$ 0,0099
	80	0,9065 $\pm$ 0,0124	0,9064 $\pm$ 0,0124	0,9071 $\pm$ 0,0122	0,9065 $\pm$ 0,0124
JDST-xLSTM [1:1]-ms	30	0,9191 $\pm$ 0,0004	0,9191 $\pm$ 0,0004	0,9192 $\pm$ 0,0004	0,9191 $\pm$ 0,0004
	50	0,9201 $\pm$ 0,0010	0,9201 $\pm$ 0,0010	0,9202 $\pm$ 0,0010	0,9201 $\pm$ 0,0010
	80	0,9193 $\pm$ 0,0017	0,9193 $\pm$ 0,0017	0,9195 $\pm$ 0,0016	0,9193 $\pm$ 0,0017

Table 4: Topic detection evaluation results on the IMDB dataset

<i>Model</i>	<i>K</i>	<i>TC-W2V</i>	<i>Topic Diversity</i>
JDST-LSTM-1L	30	0,3128 $\pm$ 0,1256	0,3228 $\pm$ 0,2029
	50	0,2413 $\pm$ 0,1307	0,2841 $\pm$ 0,1542
	80	0,3121 $\pm$ 0,0556	0,2174 $\pm$ 0,1149
JDST-LSTM-2L	30	0,3885 $\pm$ 0,0131	0,2467 $\pm$ 0,0386
	50	0,3285 $\pm$ 0,0590	0,3242 $\pm$ 0,1328
	80	0,2742 $\pm$ 0,1490	0,3645 $\pm$ 0,2166
JDST-xLSTM[0:1]	30	0,3827 $\pm$ 0,0207	0,2689 $\pm$ 0,0707
	50	0,3541 $\pm$ 0,0395	0,3100 $\pm$ 0,1143
	80	0,3543 $\pm$ 0,0235	0,2521 $\pm$ 0,0737
JDST-xLSTM[1:0]	30	0,3488 $\pm$ 0,0462	0,2792 $\pm$ 0,1257
	50	0,3410 $\pm$ 0,0477	0,2566 $\pm$ 0,1338
	80	0,3289 $\pm$ 0,0163	0,1780 $\pm$ 0,0106
JDST-xLSTM[1:1]-sm	30	0,3356 $\pm$ 0,0669	0,2599 $\pm$ 0,0822
	50	0,3853 $\pm$ 0,0025	0,1894 $\pm$ 0,0149
	80	0,3669 $\pm$ 0,0088	0,1772 $\pm$ 0,0309
JDST-xLSTM[1:1]-ms	30	0,3946 $\pm$ 0,0177	0,3798 $\pm$ 0,0240
	50	0,3867 $\pm$ 0,0035	0,3646 $\pm$ 0,0100
	80	0,3723 $\pm$ 0,0097	0,2890 $\pm$ 0,0314

Table 5: Topic detection evaluation results on the Yelp dataset.

<i>Model</i>	<i>K</i>	<i>TC-W2V</i>	<i>Topic Diversity</i>
JDST-LSTM-1L	30	0,1656 $\pm$ 0,0346	0,3321 $\pm$ 0,0573
	50	0,2334 $\pm$ 0,0757	0,3024 $\pm$ 0,1585
	80	0,1489 $\pm$ 0,0822	0,2865 $\pm$ 0,1161
JDST-LSTM-2L	30	0,2104 $\pm$ 0,1078	0,3625 $\pm$ 0,2552
	50	0,2033 $\pm$ 0,1182	0,3290 $\pm$ 0,2248
	80	0,1535 $\pm$ 0,0208	0,2709 $\pm$ 0,0285
JDST-xLSTM[0:1]	30	0,2748 $\pm$ 0,0260	0,3538 $\pm$ 0,0699
	50	0,2389 $\pm$ 0,1032	0,3449 $\pm$ 0,1074
	80	0,2852 $\pm$ 0,0153	0,2899 $\pm$ 0,0464
JDST-xLSTM[1:0]	30	0,2971 $\pm$ 0,0105	0,2501 $\pm$ 0,0440
	50	0,2637 $\pm$ 0,0206	0,2885 $\pm$ 0,0291
	80	0,2664 $\pm$ 0,0162	0,2471 $\pm$ 0,0332
JDST-xLSTM[1:1]-sm	30	0,3518 $\pm$ 0,0129	0,3933 $\pm$ 0,0661
	50	0,2750 $\pm$ 0,0357	0,3190 $\pm$ 0,1556

<i>Model</i>	<i>K</i>	TC-W2V	<i>Topic Diversity</i>
	80	0,2207 $\pm$ 0,0900	0,2600 $\pm$ 0,1682
JDST-xLSTM [1:1]-ms	30	0,3153 $\pm$ 0,0096	0,2533 $\pm$ 0,0260
	50	0,2682 $\pm$ 0,0315	0,2827 $\pm$ 0,0600
	80	0,2579 $\pm$ 0,0318	0,2917 $\pm$ 0,0925

Keywords: *Deep Embedded K-Means, Extended Long Short-Term Memory, Joint Document Level Sentiment-Topic Modeling, Multi-Task Learning.*

DAFTAR ISI

HALAMAN PERNYATAAN ORISINALITAS	iii
HALAMAN PENGESAHAN	v
SURAT PERNYATAAN	vii
KATA PENGANTAR	ix
HALAMAN PERNYATAAN PERSETUJUAN PUBLIKASI	xiii
ABSTRAK	xv
EXTENDED ABSTRACT	xvii
DAFTAR ISI	xxv
DAFTAR TABEL	xxix
DAFTAR GAMBAR	xxxii
DAFTAR LAMPIRAN	xxxiii
BAB 1 PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	4
1.3 Tujuan Penelitian	5
1.4 Manfaat Penelitian	5
1.5 Ruang Lingkup Penelitian	6
BAB 2 LANDASAN TEORI	7
2.1 Analisis Sentimen	7
2.2 Pendeteksian Topik	7
2.3 <i>Artificial Neural Network</i>	8

2.3.1	<i>Arsitektur Artificial Neural Network</i>	9
2.3.1.1	<i>Perceptron</i>	9
2.3.1.2	<i>Multi-Layer Perceptron</i>	10
2.3.1.3	Fungsi Aktivasi	11
2.3.1.4	Teknik Normalisasi	15
2.3.1.5	Fungsi <i>Loss</i>	17
2.3.1.6	<i>Backpropagation</i>	18
2.3.1.7	Optimasi <i>Adaptive Moment Estimation with Decoupled Weight Decay</i>	21
2.4	<i>Recurrent Neural Network</i>	22
2.5	<i>Long Short-Term Memory</i>	24
2.6	Transformer	25
2.6.1	Arsitektur Transformer	25
2.6.2	<i>Word Embedding</i>	26
2.6.3	<i>Encoder</i>	26
2.6.4	Mekanisme <i>Attention</i>	27
2.6.4.1	<i>Scaled Dot-Product Attention</i>	27
2.6.4.2	<i>Multi-Head Attention</i>	27
2.6.5	<i>Skip Connection</i>	28
2.6.6	<i>Decoder</i>	29
2.7	<i>Causal Convolution</i>	29
2.8	<i>K-Means Clustering</i>	30
2.9	<i>Autoencoder</i>	32
2.10	<i>Multi-Task Learning</i>	33
2.11	Metrik Evaluasi	34
2.11.1	Metrik Evaluasi Analisis Sentimen	34
2.11.2	Metrik Evaluasi Pendeteksian Topik	36

BAB 3	<i>METODE JOINT DOCUMENT LEVEL SENTIMENT-TOPIC MODEL BERBASIS EXTENDED LONG SHORT-TERM MEMORY UNTUK ANALISIS SENTIMEN</i>	39
3.1	<i>Extended Long Short-Term Memory</i>	39
3.1.1	<i>Scalar Long Short-Term Memory</i>	40
3.1.1.1	Arsitektur Sel sLSTM	41
3.1.1.2	Blok sLSTM	43
3.1.2	<i>Matrix Long Short-Term Memory</i>	44

3.1.2.1	Arsitektur Sel mLSTM	44
3.1.2.2	Blok mLSTM	47
3.2	<i>Deep Embedded K-Means</i>	48
3.3	Model <i>Joint Document Level Sentiment-Topic Model</i> Berbasis <i>Extended Long Short-Term Memory</i>	50
BAB 4	APLIKASI	55
4.1	Spesifikasi Mesin dan Perangkat Lunak	55
4.2	Tahapan Umum Simulasi	55
4.3	Data	57
4.3.1	Pengumpulan Data	57
4.3.2	Prapengolahan Data	58
4.3.3	Representasi Teks	61
4.3.4	Pembagian Data	63
4.4	Implementasi <i>Joint Document Level Sentiment-Topic Model</i>	63
4.5	Analisis Kinerja Model <i>Joint Document Level Sentiment-Topic Model</i>	64
4.5.1	Analisis Kinerja Hasil Pra-pelatihan	65
4.5.2	Analisis Kinerja Hasil <i>Joint Training</i>	66
4.5.2.1	Analisis Kinerja Hasil Analisis Sentimen	67
4.5.2.2	Analisis Kinerja Hasil Pendeteksian Topik	70
4.6	Analisis Kontribusi Token pada Representasi Bersama untuk Analisis Sentimen dan Pendeteksian Topik	74
BAB 5	PENUTUP	77
5.1	Kesimpulan	77
5.2	Saran	78
	DAFTAR PUSTAKA	79

DAFTAR TABEL

Tabel 1	Comparison of training loss and validation loss during pre-training	xx
Tabel 2	Sentiment analysis evaluation results on the IMDB dataset.	xx
Tabel 3	Sentiment analysis evaluation results on the Yelp dataset.	xxi
Tabel 4	Topic detection evaluation results on the IMDB dataset	xxii
Tabel 5	Topic detection evaluation results on the Yelp dataset.	xxii
Tabel 2.1	Ilustrasi <i>Confusion Matrix</i>	35
Tabel 3.1	Notasi dan interpretasi simbol pada DEKM (Guo <i>et al.</i> , 2021).	48
Tabel 4.1	Spesifikasi mesin dan perangkat lunak	55
Tabel 4.2	Contoh data ulasan IMDB (Maas <i>et al.</i> , 2011).	58
Tabel 4.3	Contoh data ulasan Yelp (Zhang <i>et al.</i> , 2015).	58
Tabel 4.4	Contoh hasil prapengolahan pada <i>dataset</i> IMDB dan Yelp	60
Tabel 4.5	Contoh proses tokenisasi pada <i>dataset</i> IMDB dan Yelp	61
Tabel 4.6	Contoh proses <i>embedding</i> pada <i>dataset</i> IMDB dan Yelp	62
Tabel 4.7	<i>Hyperparameter</i> pra-pelatihan	63
Tabel 4.8	<i>Hyperparameter joint training</i> model JDST	64
Tabel 4.9	Perbandingan <i>loss</i> pelatihan dan <i>loss</i> validasi pra-pelatihan	65
Tabel 4.10	Hasil evaluasi analisis sentimen model pada <i>dataset</i> IMDB	67
Tabel 4.11	Hasil evaluasi analisis sentimen model pada <i>dataset</i> Yelp	68
Tabel 4.12	Hasil evaluasi pendeteksian topik model pada <i>dataset</i> IMDB	70
Tabel 4.13	Hasil evaluasi pendeteksian topik model pada <i>dataset</i> Yelp	72
Tabel 4.14	Contoh kontribusi token pada dokumen ulasan bersentimen negatif	75
Tabel 4.15	Contoh kontribusi token pada dokumen ulasan bersentimen positif	76

DAFTAR GAMBAR

Gambar 1	Schematic Diagram of Research Framework	xviii
Gambar 2.1	Struktur neuron biologis (Aggarwal <i>et al.</i> , 2018).	8
Gambar 2.2	Struktur MLP (Aggarwal <i>et al.</i> , 2018).	10
Gambar 2.3	Grafik fungsi sigmoid.	12
Gambar 2.4	Grafik fungsi tanh.	12
Gambar 2.5	Grafik fungsi eksponensial.	13
Gambar 2.6	Grafik fungsi GeLU.	14
Gambar 2.7	Grafik fungsi Swish.	14
Gambar 2.8	Grafik fungsi <i>softmax</i> untuk kasus dua kelas.	15
Gambar 2.9	Ilustrasi graf komputasi <i>MLP</i> dua jalur (Aggarwal <i>et al.</i> , 2018), telah diolah kembali.	19
Gambar 2.10	Arsitektur RNN (Mienye <i>et al.</i> , 2024).	23
Gambar 2.11	Arsitektur dasar LSTM (Mienye <i>et al.</i> , 2024).	24
Gambar 2.12	Arsitektur dasar Transformer (Vaswani <i>et al.</i> , 2023), telah diolah kembali.	25
Gambar 2.13	Ilustrasi proses pembentukan <i>query</i> , <i>key</i> , dan <i>value</i> dari <i>input</i> <i>embedding</i> (Pradipta Hardinatha <i>et al.</i> , 2025), telah diolah kembali.	28
Gambar 2.14	Ilustrasi <i>skip connection</i> (Xu <i>et al.</i> , 2025).	29
Gambar 2.15	Ilustrasi <i>autoencoder</i> (Aggarwal <i>et al.</i> , 2018).	32
Gambar 2.16	Ilustrasi pendekatan MTL: (a) <i>hard parameter sharing</i> dan (b) <i>soft parameter sharing</i> (Ruder, 2017), telah diolah kembali.	33
Gambar 3.1	Alur proses model JDST berbasis xLSTM	39
Gambar 3.2	Arsitektur xLSTM (Beck <i>et al.</i> , 2024).	40
Gambar 3.3	Arsitektur sLSTM (Alharthi & Mahmood, 2024), telah diolah kembali.	41
Gambar 3.4	Struktur blok sLSTM (Beck <i>et al.</i> , 2024), telah diolah kembali.	43
Gambar 3.5	Arsitektur mLSTM (Alharthi & Mahmood, 2024), telah diolah kembali.	44

Gambar 3.6	Arsitektur blok mLSTM (Beck <i>et al.</i> , 2024), telah diolah kembali.	47
Gambar 3.7	Arsitektur lengkap model JDST berbasis xLSTM	50
Gambar 3.8	Alur pra-pelatihan xLSTM dengan <i>autoencoder</i>	52
Gambar 3.9	Alur lengkap jalur analisis sentimen	53
Gambar 3.10	Alur lengkap jalur pendeteksian topik	53
Gambar 4.1	Diagram alur penelitian.	56
Gambar 4.2	Perbandingan <i>loss</i> data latih pra-pelatihan seluruh <i>encoder</i>	66
Gambar 4.3	Perbandingan <i>loss</i> data validasi pra-pelatihan seluruh <i>encoder</i>	66
Gambar 4.4	Perbandingan <i>Accuracy</i> dan F1- <i>Macro</i> analisis sentimen model pada <i>dataset</i> IMDB	68
Gambar 4.5	Perbandingan <i>Accuracy</i> dan F1- <i>Macro</i> analisis sentimen model pada <i>dataset</i> Yelp	70
Gambar 4.6	Perbandingan TC-W2V dan TD pendeteksian topik model pada <i>dataset</i> IMDB	72
Gambar 4.7	Perbandingan TC-W2V dan TD pendeteksian topik model pada <i>dataset</i> Yelp	74

DAFTAR LAMPIRAN

1

(Guo *et al.*, 2021)50

Code Pra-pelatihan xLSTM 85

Code Joint Training JDST 95

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Pada era digital, volume data teks tidak terstruktur dari platform daring seperti media sosial, ulasan produk, dan portal berita mengalami peningkatan yang masif. Setiap hari, jutaan pengguna menghasilkan konten yang mencerminkan opini, emosi, dan persepsi mereka terhadap berbagai topik, menciptakan lonjakan informasi yang kaya akan wawasan (Andreotta *et al.*, 2019). Kondisi ini membuka peluang besar untuk memahami opini publik dan tren sosial, namun sekaligus menghadirkan tantangan baru karena proses analisis manual tidak efisien. Sebagaimana dinyatakan oleh Nikhil (2015), sebagian besar data di internet bersifat tidak terstruktur dan memerlukan komputasi seperti *text mining* dan *Natural Language Processing* (NLP) agar dapat dimanfaatkan secara optimal.

Salah satu cabang utama NLP yang terus berkembang adalah analisis sentimen, yaitu proses untuk mengidentifikasi dan mengelompokkan opini serta emosi yang terkandung dalam teks (Pang & Lee, 2008). Dalam bidang *machine learning*, analisis sentimen termasuk ke dalam masalah *supervised learning*, khususnya klasifikasi, sehingga proses pelatihan model memerlukan data teks yang telah memiliki label sentimen (Murfi *et al.*, 2018). Tingkatan analisis sentimen yang paling umum digunakan adalah *document level sentiment analysis*, yaitu metode yang menganggap setiap dokumen memiliki satu sentimen utama secara keseluruhan, seperti positif, negatif, atau netral (Pang & Lee, 2008). Metode ini banyak digunakan karena pemodelannya relatif sederhana dan komputasinya lebih efisien. Selain itu, setiap dokumen hanya memerlukan satu label sentimen sehingga proses pelabelan data menjadi lebih sederhana dibandingkan pendekatan yang melakukan anotasi pada tingkat yang lebih rinci (Pang & Lee, 2008). Karakteristik tersebut membuat *document level sentiment analysis* banyak diterapkan dalam analisis opini publik berskala besar (Zhang *et al.*, 2018).

Meskipun demikian, *document level sentiment analysis* memiliki keterbatasan mendasar karena metode ini mengasumsikan bahwa sebuah dokumen bersifat homogen secara emosional. Dalam praktiknya, sebuah dokumen sering kali memuat beberapa aspek atau topik dengan polaritas yang berbeda-beda (Pang & Lee, 2008). Akibatnya, informasi sentimen yang lebih rinci dan bermakna dapat hilang. Kondisi ini mendorong perkembangan *Topic-Level Sentiment Analysis* (TLSA), yaitu metode yang bertujuan mengidentifikasi sentimen terhadap topik-topik tertentu dalam suatu dokumen (Titov & McDonald, 2008). Metode ini mampu menghasilkan analisis yang lebih terperinci dan

berguna dalam konteks evaluasi produk, layanan, maupun kebijakan publik.

Perkembangan TLSA kemudian menghasilkan berbagai model yang berupaya mempelajari topik dan sentimen secara simultan. Salah satu model awal yang menjadi landasan pengembangan ini adalah *Joint Sentiment-Topic* (JST) yang dikembangkan oleh Lin (2009), yang memperluas *Latent Dirichlet Allocation* (LDA) (Blei, 2012) dengan menambahkan dimensi sentimen sehingga topik dan sentimen dapat dipelajari secara simultan dalam satu kerangka probabilistik. Pengembangan selanjutnya dilakukan melalui *Topic Sentiment Latent Dirichlet Allocation* (TSLDA) untuk memperbaiki pemodelan hubungan topik dan sentimen sehingga variasinya pada berbagai topik dapat direpresentasikan dengan lebih baik (Nguyen & Shirai, 2015). Namun, model-model tersebut masih menggunakan representasi berbasis *bag-of-words* yang belum memanfaatkan informasi semantik antar kata. Untuk mengatasi hal tersebut, *Topic Sentiment model with Word Embedding* (TSWE) mengintegrasikan *word embedding* sehingga kesamaan makna antar kata dapat dimanfaatkan dalam proses pemodelan (Fu *et al.*, 2016), dan dikembangkan lebih lanjut melalui *Embedding-enhanced JST* (ELJST) yang menggabungkan *word embedding* dengan model berbasis *skip-gram* untuk menghasilkan representasi yang lebih koheren (Sengupta *et al.*, 2021).

Meskipun TLSA mampu menghasilkan analisis yang lebih terperinci, penerapannya memerlukan proses anotasi dan pemodelan yang lebih kompleks karena model perlu mengenali topik yang dibahas sekaligus sentimen yang terkait dengan setiap topik tersebut (Pang & Lee, 2008). Sebagai alternatif, pendekatan *document level sentiment analysis* juga dikembangkan lebih jauh dengan mengintegrasikan informasi topik ke dalam proses pemodelannya. Dalam kerangka ini, terdapat dua varian utama, yaitu *Separate Document Level Sentiment-Topic Model* (SDST) dan *Joint Document Level Sentiment-Topic Model* (JDST).

Pada varian *separate*, pendeteksian topik dan klasifikasi sentimen dilakukan secara terpisah oleh model yang berbeda. Varian ini memiliki kelebihan berupa proses yang efisien dan fleksibel, namun karena kedua tugas dilatih secara independen, hubungan antara topik dan sentimen tidak dipelajari secara langsung sehingga tidak ada jaminan bahwa sentimen yang diprediksi benar-benar berkaitan dengan topik yang terdeteksi (Sengupta *et al.*, 2021). Sementara itu, JDST mempelajari topik dan sentimen secara simultan dalam satu kerangka pelatihan, sehingga kedua tugas dapat saling memengaruhi satu sama lain selama proses pelatihan. Dalam kerangka ini, kedua tugas berbagi satu representasi laten yang sama, sehingga fitur yang dipelajari untuk mengenali sentimen akan berkontribusi dalam membentuk topik, dan sebaliknya, informasi topik yang ada di dalam representasi laten juga dapat membantu model membedakan polaritas sentimen.

Salah satu implementasi JDST berbasis *deep learning* diusulkan oleh Gui *et al.* (2022) melalui kerangka *Multi-Task Learning* (MTL) dengan representasi bersama yang dipelajari menggunakan satu model *encoder*, yaitu *Gated Recurrent Unit* (GRU) (Chung *et al.*, 2014), varian yang lebih sederhana dari *Long Short-Term Memory* (LSTM) (Mienye *et al.*, 2024). Model ini menerapkan *two-stage pipeline*, di mana pada tahap pertama, representasi laten yang dihasilkan *encoder* digunakan secara bersamaan untuk klasifikasi sentimen melalui *fully connected layer* dan untuk rekonstruksi *input* melalui *decoder*. Pada tahap kedua, representasi laten yang sama kemudian dikelompokkan menggunakan *K-Means* untuk membentuk topik-topik laten secara eksplisit, namun proses ini dilakukan secara terpisah setelah tahap pertama selesai karena *K-Means* bersifat non-diferensiabel dan tidak dapat dioptimasi bersama dalam satu proses pelatihan. Meskipun pendekatan ini berhasil meningkatkan akurasi klasifikasi sentimen sekaligus koherensi topik yang dihasilkan, pendeteksian topik masih memerlukan tahap tambahan yang terpisah dari proses pelatihan utama, sehingga analisis sentimen dan pendeteksian topik belum dipelajari secara *end-to-end* dalam satu proses pelatihan yang sama.

Perbedaan utama antara pendekatan *two-stage* dan *end-to-end* terletak pada proses pembentukan representasi bersama. Meskipun keduanya menghasilkan *output* yang sama, yaitu sentimen dan topik dari satu representasi, cara representasi tersebut dipelajari berbeda. Pada pendekatan *two-stage*, representasi dibentuk terlebih dahulu untuk mendukung tugas analisis sentimen. Representasi yang telah dihasilkan kemudian langsung dimanfaatkan pada tahap pendeteksian topik tanpa melalui penyesuaian lebih lanjut (Gui *et al.*, 2022). Akibatnya, apabila representasi tersebut belum mampu membedakan topik dengan baik, model tidak memiliki mekanisme untuk memperbaikinya.

Sebaliknya, pada pendekatan *end-to-end*, representasi bersama dipelajari secara simultan berdasarkan kebutuhan tugas selama proses pelatihan (Ruder, 2017). Informasi yang berasal dari tugas analisis sentimen maupun pendeteksian topik sama-sama digunakan untuk memperbarui parameter model melalui proses optimasi. Dengan mekanisme tersebut, representasi yang dihasilkan tidak hanya efektif untuk mengenali sentimen, tetapi juga mampu menangkap karakteristik yang diperlukan dalam membedakan topik. Oleh karena itu, pendekatan *end-to-end* menghasilkan representasi yang lebih seimbang karena dioptimalkan untuk mendukung kedua tugas secara bersamaan.

Perkembangan model *deep learning* untuk data sekuensial, khususnya arsitektur berbasis LSTM, melahirkan berbagai pengembangan baru untuk meningkatkan kemampuan pemodelan konteks jangka panjang. Salah satu di antaranya adalah

Extended Long Short-Term Memory (xLSTM), yang memperkenalkan mekanisme *exponential gating* dan *matrix memory* untuk meningkatkan kapasitas representasi serta kemampuan menangkap konteks jangka panjang (Beck *et al.*, 2024; Alharthi & Mahmood, 2024). Karakteristik tersebut menjadikan xLSTM berpotensi digunakan sebagai model representasi teks yang lebih efektif dalam kerangka *multi-task learning*.

Berdasarkan perkembangan tersebut, penelitian ini mengusulkan model JDST berbasis xLSTM yang mempelajari sentimen dan topik secara *end-to-end*. Berbeda dengan Gui *et al.* (2022) yang memerlukan tahap tambahan secara terpisah untuk pendeteksian topik, model yang diusulkan terlebih dahulu melakukan tahap pra-pelatihan pada xLSTM menggunakan rekonstruksi *embedding*. Tahap ini bertujuan untuk menghasilkan representasi token yang lebih informatif. Selanjutnya, representasi token tersebut diproses oleh sebuah *encoder* berbasis *Multi-Layer Perceptron* dua *layer* untuk menghasilkan representasi bersama berdimensi lebih rendah. Representasi bersama tersebut kemudian digunakan secara simultan oleh dua tugas, yaitu klasifikasi sentimen melalui MLP tiga *layer* dan pendeteksian topik melalui *Deep Embedded K-Means* (DEKM), yaitu pengembangan *K-Means* yang mengintegrasikan pembelajaran representasi laten dengan proses *clustering* berbasis *neural network* (Guo *et al.*, 2021). Seluruh komponen model dilatih secara simultan dalam satu proses pelatihan sehingga representasi bersama yang dihasilkan dapat dimanfaatkan oleh kedua tugas untuk mempelajari informasi sentimen maupun topik.

Untuk menilai efektivitas model yang diusulkan, dilakukan perbandingan antara JDST berbasis xLSTM dengan JDST berbasis LSTM sebagai *baseline* pada arsitektur yang serupa. Evaluasi ini bertujuan mengukur sejauh mana jenis model representasi teks yang digunakan memengaruhi kinerja kedua tugas secara bersamaan. Pada tugas analisis sentimen, kinerja model dievaluasi menggunakan metrik berbasis *confusion matrix*, meliputi *accuracy*, *precision*, *recall*, dan *F1-score* (Sokolova & Lapalme, 2009). Pada tugas pendeteksian topik, kualitas topik yang dihasilkan dinilai menggunakan *topic coherence* berbasis *word embedding* (TC-W2V) (O’Callaghan *et al.*, 2015) serta *topic diversity* (Dieng *et al.*, 2019). Dengan demikian, evaluasi yang dilakukan diharapkan dapat memberikan gambaran mengenai kontribusi xLSTM sebagai model representasi teks terhadap kualitas *output* kedua tugas secara keseluruhan.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, rumusan masalah pada penelitian ini dijabarkan sebagai berikut:

1. Bagaimana merancang model *Joint Document Level Sentiment-Topic Model* (JDST) berbasis xLSTM yang mengintegrasikan analisis sentimen dan pendeteksian topik secara *end-to-end* dalam satu kerangka *multi-task learning*?
2. Bagaimana kinerja analisis sentimen model JDST berbasis xLSTM dibandingkan dengan JDST berbasis LSTM?
3. Bagaimana kinerja pendeteksian topik model JDST berbasis xLSTM dibandingkan dengan JDST berbasis LSTM?

1.3 Tujuan Penelitian

Berdasarkan masalah penelitian yang telah dirumuskan, maka tujuan dari penelitian ini adalah sebagai berikut:

1. Merancang model JDST berbasis xLSTM yang mengintegrasikan analisis sentimen dan pendeteksian topik dalam satu kerangka *multi-task learning*.
2. Membandingkan kinerja analisis sentimen model JDST berbasis xLSTM dengan JDST berbasis LSTM.
3. Membandingkan kualitas topik yang dihasilkan model JDST berbasis xLSTM dengan JDST berbasis LSTM.

1.4 Manfaat Penelitian

Manfaat yang diharapkan dari penelitian ini mencakup aspek teoretis dan praktis, sebagaimana dijelaskan berikut:

1. Manfaat Teoretis

Penelitian ini diharapkan dapat berkontribusi terhadap pengembangan penelitian NLP, terutama pada integrasi analisis sentimen dan pendeteksian topik dalam satu kerangka *multi-task learning*. Selain itu, penelitian ini juga menambah referensi terkait penggunaan xLSTM sebagai model representasi pada data teks. Hasil yang diperoleh diharapkan dapat menjadi acuan bagi penelitian selanjutnya dalam mengembangkan model analisis teks yang lebih efektif.

2. Manfaat Praktis

Penelitian ini diharapkan dapat membantu analisis data ulasan teks secara lebih menyeluruh dengan mengintegrasikan informasi sentimen dan topik dalam

satu model. Model yang diusulkan dapat dimanfaatkan untuk berbagai kebutuhan, seperti analisis opini pelanggan, pemantauan kepuasan pengguna, serta pengambilan keputusan berbasis data pada platform digital. Selain itu, pendekatan yang digunakan juga berpotensi meningkatkan efisiensi pemrosesan dibandingkan dengan pendekatan terpisah.

1.5 Ruang Lingkup Penelitian

Ruang lingkup pada penelitian ini dibatasi pada beberapa aspek berikut.

1. Analisis sentimen yang digunakan berupa klasifikasi biner dengan dua kategori, yaitu sentimen positif dan negatif.
2. *Dataset* yang digunakan untuk mengevaluasi performa model adalah *dataset* IMDB dan Yelp yang terdiri atas teks ulasan berbahasa Inggris.
3. Jumlah data ulasan yang digunakan dalam penelitian ini dibatasi sebanyak 50.000 ulasan untuk masing-masing *dataset*, baik IMDB maupun Yelp.

BAB 2

LANDASAN TEORI

Bab ini membahas landasan teoretis yang mendukung penelitian. Pembahasan meliputi konsep analisis sentimen, pendeteksian topik, metode pemrosesan teks, mekanisme *Multi-Task Learning* (MTL), serta metrik evaluasi yang digunakan dalam penelitian.

2.1 Analisis Sentimen

Penjelasan pada subbab ini mengacu pada Liu (2012), kecuali jika disebutkan dari sumber lain. Analisis sentimen merupakan bidang kajian dalam Natural Language Processing (NLP) yang bertujuan untuk mengidentifikasi dan mengevaluasi opini, sentimen, maupun emosi yang terkandung dalam data teks. Opini tersebut dapat diekspresikan terhadap berbagai entitas, seperti produk, layanan, organisasi, maupun isu tertentu. Oleh karena itu, analisis sentimen banyak digunakan untuk memahami persepsi dan tanggapan masyarakat berdasarkan data teks yang bersifat tidak terstruktur.

Salah satu pendekatan yang paling umum digunakan dalam analisis sentimen adalah *document-level sentiment analysis*. Pada pendekatan ini, setiap dokumen diasumsikan merepresentasikan satu sentimen utama secara keseluruhan, seperti positif, negatif, atau netral. Dengan demikian, proses analisis dilakukan pada tingkat dokumen tanpa mempertimbangkan sentimen pada bagian-bagian teks yang lebih rinci. Pendekatan ini sesuai digunakan ketika isi dokumen mencerminkan pandangan penulis terhadap suatu entitas secara umum dan konsisten.

Dalam penelitian ini, analisis sentimen dilakukan pada tingkat dokumen. Setiap ulasan pada dataset diperlakukan sebagai satu dokumen yang merepresentasikan satu sentimen dominan, yaitu positif atau negatif. Oleh karena itu, tugas klasifikasi sentimen yang dilakukan bertujuan untuk menentukan polaritas sentimen keseluruhan dari setiap ulasan.

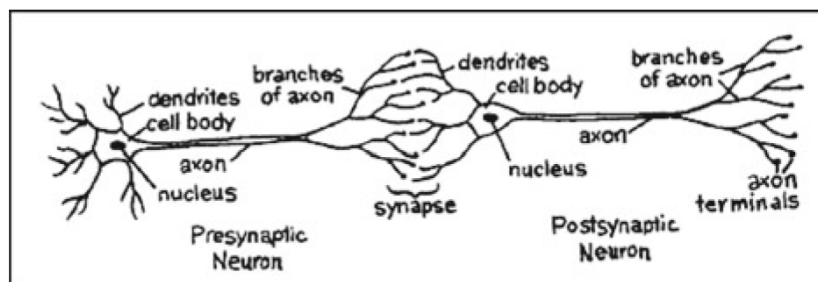
2.2 Pendeteksian Topik

Penjelasan pada subbab ini mengacu pada penelitian Zafarani-Moattar *et al.* (2024), kecuali jika disebutkan berasal dari sumber lain. Pendeteksian topik (*topic detection*) merupakan proses untuk mengidentifikasi dan mengelompokkan dokumen berdasarkan kesamaan tema atau topik yang dibahas. Pada data teks, proses ini bertujuan untuk menemukan pola kemunculan kata yang mencerminkan suatu topik tertentu.

Pendeteksian topik dipengaruhi oleh karakteristik bahasa serta hubungan semantik antar kata dan dokumen.

Metode pendeteksian topik dapat dikategorikan ke dalam pendekatan *frequent pattern mining* dan *clustering-based*. Pendekatan *frequent pattern mining* berfokus pada identifikasi kata atau kombinasi kata yang sering muncul bersama dalam dokumen sebagai representasi dari suatu topik. Sementara itu, pada pendekatan *clustering-based*, dokumen direpresentasikan dalam bentuk vektor dan kemudian dikelompokkan berdasarkan ukuran kemiripan atau jarak antardokumen. Setiap *cluster* yang terbentuk merepresentasikan suatu topik tertentu.

2.3 Artificial Neural Network



Gambar 2.1: Struktur neuron biologis (Aggarwal *et al.*, 2018).

Artificial Neural Network (ANN) atau bisa disebut sebagai jaringan saraf tiruan merupakan model komputasi dalam *machine learning* yang terinspirasi dari sistem saraf biologis, khususnya otak manusia (Aggarwal *et al.*, 2018). Sistem saraf biologis terdiri dari neuron yang berfungsi menerima, mengolah, dan meneruskan informasi melalui komponen utama seperti dendrit, *cell body*, akson, dan sinapsis. Berdasarkan Gambar 2.1, sinyal diterima melalui dendrit dan diproses pada *cell body*. Selanjutnya, sinyal diteruskan melalui akson ke neuron lain melalui sinapsis. Hubungan antarneuron tersebut memiliki tingkat pengaruh yang dapat berubah, sehingga memungkinkan terjadinya proses pembelajaran.

Neuron dalam ANN digambarkan sebagai unit komputasi yang menerima *input* numerik dan masing-masing dikalikan dengan bobot untuk menunjukkan kekuatan hubungan. Nilai ini kemudian diproses oleh fungsi aktivasi untuk menghasilkan *output*. Secara umum, ANN menggambarkan cara sistem biologis memproses dan meneruskan informasi dalam bentuk numerik (Aggarwal *et al.*, 2018).

ANN dapat mempelajari pola dan fitur penting dari data *input*. Proses ini dikenal

sebagai *representation learning*, yaitu pembelajaran representasi data agar informasi penting di dalam data dapat digunakan oleh model untuk menyelesaikan suatu tugas (Bengio *et al.*, 2012). Hasil dari proses tersebut disebut representasi laten, yaitu vektor numerik yang dibentuk oleh lapisan tersembunyi (*hidden layer*) untuk merangkum fitur penting dari data. Kumpulan representasi laten ini membentuk ruang laten, yaitu ruang vektor di mana setiap data dipetakan berdasarkan fitur yang dipelajari oleh model.

2.3.1 Arsitektur Artificial Neural Network

Arsitektur ANN terdiri atas beberapa lapisan atau *layer* neuron yang saling terhubung dan bekerja secara berurutan dalam memproses informasi (Qamar & Zardari, 2023). *Input layer* berfungsi sebagai titik masuk data ke dalam neuron, di mana *input* berupa fitur diterima dan kemudian diteruskan ke *layer* berikutnya untuk diproses lebih lanjut. Selanjutnya, *layer* yang berada di antara *input layer* dan *output layer*, yang dikenal sebagai *hidden layer*, mengekstraksi pola dan karakteristik penting dari data melalui kombinasi linier dengan bobot dan fungsi nonlinier. Sementara itu, *output layer* menghasilkan *output* akhir neuron sesuai dengan tujuan pemodelan, seperti nilai kontinu pada tugas regresi atau label kelas pada tugas klasifikasi. Perbedaan jumlah *layer*, neuron, serta pola konektivitas antar-neuron menghasilkan berbagai variasi arsitektur ANN, mulai dari jaringan *feedforward* yang sederhana hingga arsitektur yang lebih kompleks (Da Silva *et al.*, 2016).

2.3.1.1 Perceptron

Perceptron merupakan model dasar dari ANN yang terdiri atas satu *input layer* yang terhubung langsung ke satu neuron *output* tanpa *hidden layer*. Setiap *input* direpresentasikan sebagai sinyal numerik yang dikalikan dengan bobot tertentu. Hasilnya kemudian dijumlahkan dan diproses menggunakan fungsi aktivasi untuk menghasilkan *output* (Aggarwal *et al.*, 2018).

Arsitektur *perceptron* umumnya dilengkapi dengan komponen bias yang berfungsi menggeser ambang aktivasi neuron sehingga pengambilan keputusan tidak harus selalu melalui titik asal. Dengan adanya bias, model memiliki fleksibilitas yang lebih besar dalam membentuk batas keputusan yang sesuai dengan distribusi data. Secara matematis, *output perceptron* dirumuskan pada persamaan (2.1) sebagai

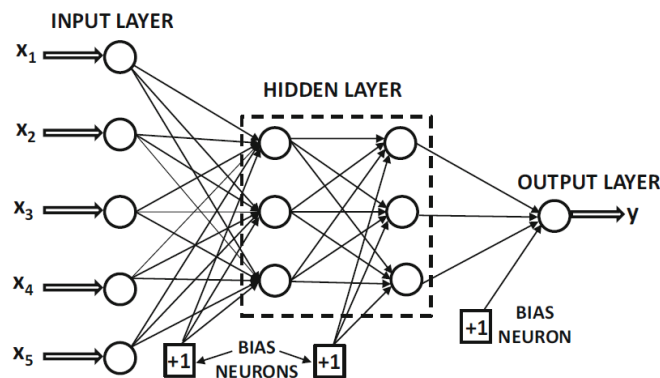
$$y = f \left(\sum_{i=1}^n w_i x_i + b \right), \quad (2.1)$$

dengan x_i menyatakan *input* ke- i , w_i adalah bobot yang merepresentasikan kontribusi relatif setiap *input* terhadap *output* neuron, b adalah bias, dan $f(\cdot)$ adalah fungsi aktivasi yang menentukan respons akhir neuron terhadap sinyal yang diterima. Ketika $b = 0$, persamaan ini menjadi bentuk *perceptron* tanpa bias.

2.3.1.2 Multi-Layer Perceptron

Multi-Layer Perceptron (MLP) merupakan bentuk pengembangan dari model *perceptron* dengan penambahan satu atau lebih *hidden layer* di antara *input layer* dan *output layer* (Aggarwal *et al.*, 2018). Keberadaan *hidden layer* memungkinkan jaringan untuk memodelkan hubungan nonlinier yang tidak dapat dijelaskan oleh *perceptron*. Pada arsitektur MLP, setiap neuron dalam suatu *layer* terhubung penuh dengan neuron pada *layer* berikutnya, sehingga proses pemrosesan informasi berlangsung secara berurutan dari *input layer* hingga *output layer*. Oleh karena itu, MLP diklasifikasikan sebagai *feed-forward neural network*, di mana aliran data hanya terjadi dalam satu arah tanpa koneksi umpan balik.

Ilustrasi arsitektur umum MLP ditunjukkan pada Gambar 2.2. Pada arsitektur tersebut, *input layer* menerima vektor *input* yang kemudian diproses secara bertahap oleh *hidden layer* melalui kombinasi linier berbobot dan fungsi aktivasi. Selanjutnya, *output* dari *hidden layer* terakhir diteruskan ke *output layer* untuk menghasilkan *output* akhir jaringan.



Gambar 2.2: Struktur MLP (Aggarwal *et al.*, 2018).

Alur pemrosesan data pada MLP dapat dinyatakan secara berurutan antar *layer*. Misalkan vektor aktivasi pada *layer* ke- r dinyatakan sebagai $\mathbf{h}_r$, maka proses dari *input layer* ke *hidden layer* pertama dapat dituliskan pada persamaan (2.2) sebagai

$$\mathbf{h}_1 = f(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1), \quad (2.2)$$

dengan $\mathbf{x}$ merupakan vektor *input*, $\mathbf{W}_1$ adalah matriks bobot yang menghubungkan *input layer* dengan *hidden layer* pertama, dan $\mathbf{b}_1$ menyatakan vektor bias. Untuk *hidden layer* selanjutnya, proses antar *layer* dapat dinyatakan pada persamaan (2.3) sebagai

$$\mathbf{h}_{r+1} = f(\mathbf{W}_{r+1}\mathbf{h}_r + \mathbf{b}_{r+1}), \quad r = 1, 2, \dots, l-1, \quad (2.3)$$

dengan l menyatakan jumlah *hidden layer*. Selanjutnya, *output* jaringan diperoleh melalui *output layer* dalam persamaan (2.4) sebagai

$$\mathbf{y} = f(\mathbf{W}_{l+1}\mathbf{h}_l + \mathbf{b}_{l+1}), \quad (2.4)$$

dengan $\mathbf{y}$ merupakan vektor *output* jaringan. Fungsi $f(\cdot)$ dalam setiap *layer* adalah fungsi aktivasi yang digunakan untuk menambahkan sifat nonlinier ke dalam model.

2.3.1.3 Fungsi Aktivasi

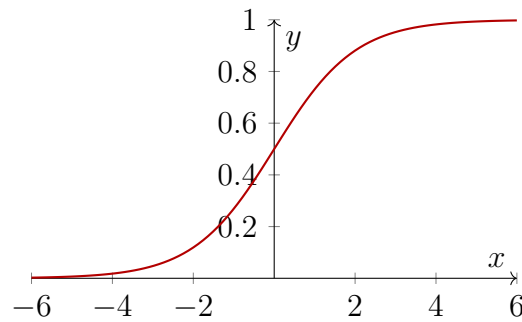
Fungsi aktivasi digunakan dalam *neural network* untuk menambahkan sifat nonlinier ke dalam model. Setelah *input* dikombinasikan dengan bobot dan bias, fungsi aktivasi memetakan hasil tersebut menjadi *output* neuron. Sifat nonlinieritas ini memungkinkan *neural network* mempelajari pola yang lebih kompleks dibandingkan dengan model linier sederhana (Dubey *et al.*, 2022). Pada penelitian ini, fungsi aktivasi diterapkan pada masing-masing komponen vektor *input* $\mathbf{x} \in \mathbb{R}^n$. *Neural network* memiliki berbagai jenis fungsi aktivasi yang dapat digunakan. Berikut adalah penjelasan tentang beberapa fungsi aktivasi yang digunakan dalam penelitian:

1. Fungsi Sigmoid

Fungsi aktivasi sigmoid menghasilkan *output* pada rentang $[0, 1]$ dan didefinisikan pada persamaan (2.5).

$$f(\mathbf{x}) = \frac{1}{1 + e^{-\mathbf{x}}}. \quad (2.5)$$

Berdasarkan sifat pemetaan tersebut, *output* fungsi sigmoid dapat diinterpretasikan sebagai probabilitas, sehingga fungsi ini banyak digunakan pada *layer output* untuk permasalahan klasifikasi biner. Nilai *output* fungsi sigmoid berubah signifikan pada kisaran nilai *input* di sekitar nol, serta secara bertahap mendekati nilai 0 dan 1 ketika *input* bernilai sangat kecil atau sangat besar (Nieradzic *et al.*, 2021). Grafik fungsi sigmoid ditunjukkan pada Gambar 2.3.



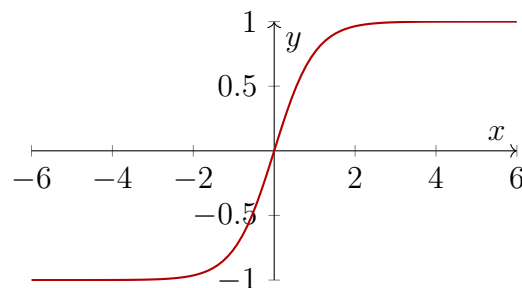
Gambar 2.3: Grafik fungsi sigmoid.

2. Fungsi *Hyperbolic Tangent*

Fungsi *Hyperbolic Tangent* ($\tanh$) merupakan fungsi aktivasi nonlinier yang menghasilkan *output* pada rentang $(-1, 1)$, sebagaimana dirumuskan pada persamaan (2.6).

$$f(x) = \tanh(x) = \varphi(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}. \quad (2.6)$$

Karena sifatnya yang simetris terhadap nol, fungsi $\tanh$ menghasilkan *output* yang terpusat di sekitar nol sehingga dapat membantu mempercepat konvergensi selama proses pelatihan *neural network* (Dubey *et al.*, 2022). Perubahan nilai *output* paling signifikan terjadi pada kisaran *input* di sekitar nol, sedangkan pada nilai *input* yang ekstrem, fungsi ini cenderung mendekati batas -1 dan 1 . Grafik fungsi $\tanh$ ditunjukkan pada Gambar 2.4.

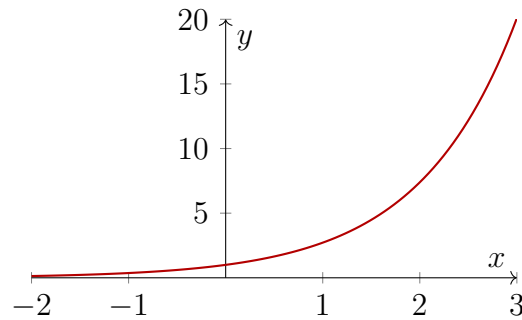
Gambar 2.4: Grafik fungsi $\tanh$.

3. Fungsi Eksponensial

Fungsi eksponensial merupakan fungsi nonlinier yang memetakan setiap nilai *input* ke *output* bernilai positif dan bersifat monoton meningkat, yang ditunjukkan pada persamaan (2.7).

$$f(x) = e^x. \quad (2.7)$$

Fungsi eksponensial memiliki domain bilangan riil dengan *range* $(0, \infty)$, sehingga perbedaan kecil pada nilai *input* dapat menghasilkan perbedaan *output* yang jauh lebih besar (Ho *et al.*, 2012). Grafik fungsi eksponensial ditunjukkan pada Gambar 2.5.



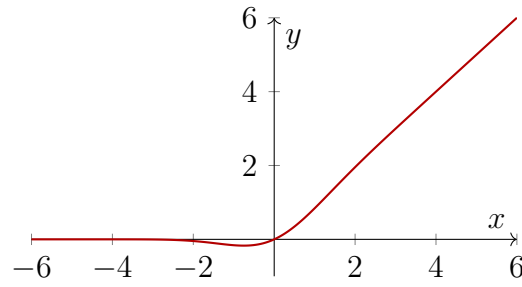
Gambar 2.5: Grafik fungsi eksponensial.

4. Fungsi *Gaussian Error Linear Unit*

Fungsi *Gaussian Error Linear Unit* (GeLU) merupakan fungsi aktivasi nonlinier yang menggabungkan nilai *input* dengan fungsi distribusi kumulatif Gaussian, sehingga aktivasi neuron ditentukan secara probabilistik berdasarkan besar kecilnya nilai *input* (Hendrycks & Gimpel, 2023). Secara matematis, fungsi GeLU didefinisikan pada persamaan (2.8).

$$f(\mathbf{x}) = \mathbf{x} \cdot P(X \leq \mathbf{x}) = \mathbf{x} \cdot \Phi(\mathbf{x}) = \mathbf{x} \cdot \frac{1}{2} \left[1 + \operatorname{erf} \left(\frac{\mathbf{x}}{\sqrt{2}} \right) \right], \quad (2.8)$$

dengan $\Phi(\mathbf{x})$ menyatakan fungsi distribusi kumulatif Gaussian standar dan $\operatorname{erf}(\mathbf{x}) = \frac{2}{\sqrt{\pi}} \int_0^{\mathbf{x}} e^{-t^2} dt$ merupakan fungsi *error* Gaussian. Berbeda dengan ReLU yang melakukan aktivasi berdasarkan tanda nilai *input*, GeLU memberikan pembobotan secara halus berdasarkan besar nilai *input*, sehingga perubahan nilai aktivasi menjadi lebih kontinu (Hendrycks & Gimpel, 2023). Grafik fungsi GeLU ditunjukkan pada Gambar 2.6.



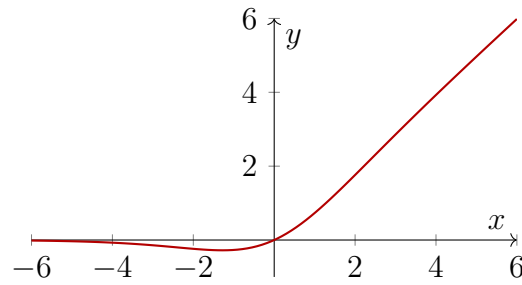
Gambar 2.6: Grafik fungsi GeLU.

5. Fungsi Swish

Fungsi Swish merupakan fungsi aktivasi nonlinier yang memanfaatkan fungsi sigmoid untuk mengatur besarnya aktivasi dari nilai *input*. Secara matematis, fungsi Swish didefinisikan pada persamaan (2.9).

$$f(\mathbf{x}) = \mathbf{x} \cdot \sigma(\mathbf{x}) = \frac{\mathbf{x}}{1 + e^{-\mathbf{x}}}, \quad (2.9)$$

dengan $\sigma(\mathbf{x})$ menyatakan fungsi sigmoid. Bentuk perkalian antara $\mathbf{x}$ dan $\sigma(\mathbf{x})$ menghasilkan mekanisme *self-gating*, sehingga kontribusi informasi dari suatu neuron dapat diatur secara adaptif (Ramachandran *et al.*, 2017). Grafik fungsi Swish ditunjukkan pada Gambar 2.7.



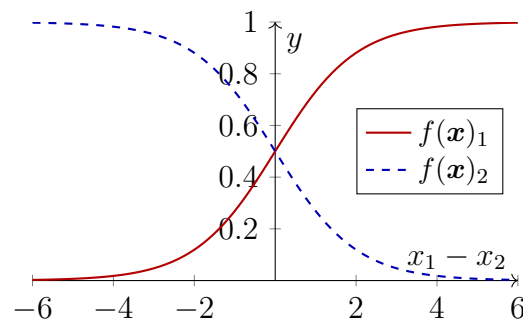
Gambar 2.7: Grafik fungsi Swish.

6. Fungsi Softmax

Fungsi *softmax* merupakan generalisasi dari fungsi sigmoid untuk kasus multikelas, yang memetakan vektor *input* $\mathbf{x} \in \mathbb{R}^n$ menjadi distribusi probabilitas pada rentang $(0, 1)$ dengan jumlah seluruh komponen sama dengan 1. Secara matematis, fungsi *softmax* untuk komponen ke- i didefinisikan pada persamaan (2.10).

$$f(\mathbf{x})_i = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}. \quad (2.10)$$

Fungsi *softmax* umum digunakan pada *output layer* untuk tugas klasifikasi multikelas, di mana setiap komponen *output* merepresentasikan probabilitas *input* termasuk ke dalam kelas tertentu. Fungsi eksponensial pada *numerator* memastikan seluruh nilai *output* bernilai positif, sedangkan normalisasi pada *denominator* memastikan jumlah seluruh probabilitas sama dengan 1 sehingga membentuk distribusi probabilitas (Goodfellow *et al.*, 2016). Grafik fungsi *softmax* untuk kasus dua kelas ($n = 2$) terhadap selisih *input* $x_1 - x_2$ ditunjukkan pada Gambar 2.8.



Gambar 2.8: Grafik fungsi *softmax* untuk kasus dua kelas.

2.3.1.4 Teknik Normalisasi

Normalisasi merupakan teknik yang digunakan dalam pelatihan *neural network* untuk menstabilkan distribusi aktivasi antar-*layer* selama proses pelatihan berlangsung (Huang *et al.*, 2020). Normalisasi dilakukan dengan menstandarisasi aktivasi menggunakan rata-rata dan variansi yang dihitung dari kelompok aktivasi tertentu. Hasil normalisasi kemudian disesuaikan kembali menggunakan parameter skala γ dan *shift* β yang dapat dilatih, sehingga model tetap memiliki fleksibilitas dalam mempelajari pola data. Terdapat dua metode normalisasi yang digunakan dalam penelitian ini, yaitu *Layer Normalization* dan *Group Normalization*, yang dijelaskan sebagai berikut:

1. *Layer Normalization*

Layer Normalization (LN) bekerja dengan menghitung rata-rata dan variansi dari seluruh unit dalam satu *layer* untuk setiap sampel secara independen (Ba *et al.*, 2016). Nilai-nilai tersebut kemudian digunakan untuk menstandarisasi aktivasi, sehingga distribusi aktivasi pada setiap *layer* tetap stabil selama pelatihan. Secara matematis, rata-rata μ^{LN} dan standar deviasi σ^{LN} pada LN dirumuskan pada

persamaan (2.11) sebagai

$$\mu^{\text{LN}} = \frac{1}{d_h} \sum_{i=1}^{d_h} a_i, \quad \sigma^{\text{LN}} = \sqrt{\frac{1}{d_h} \sum_{i=1}^{d_h} (a_i - \mu^{\text{LN}})^2}, \quad (2.11)$$

dengan d_h adalah jumlah unit dalam *layer* dan a_i adalah aktivasi unit ke- i . Selanjutnya, aktivasi yang telah dinormalisasi ditransformasikan menggunakan parameter yang dapat dipelajari yang dirumuskan pada persamaan (2.12) sebagai

$$\tilde{a}_i = \frac{\gamma^{\text{LN}}}{\sigma^{\text{LN}}} (a_i - \mu^{\text{LN}}) + \beta^{\text{LN}}, \quad (2.12)$$

dengan γ^{LN} dan β^{LN} masing-masing merupakan parameter skala dan bias yang dipelajari selama pelatihan.

2. *Group Normalization*

Group Normalization (GN) merupakan metode normalisasi yang membagi dimensi fitur dalam suatu *layer* ke dalam G kelompok berukuran sama (Wu & He, 2018). GN menghitung rata-rata dan variansi secara independen untuk setiap kelompok pada setiap sampel. Perhitungan tersebut tidak bergantung pada ukuran *batch*, sehingga GN tetap stabil meskipun *batch size* yang digunakan kecil.

Misalkan C menyatakan jumlah total dimensi fitur dan G adalah jumlah kelompok yang diinginkan, maka setiap kelompok memuat C/G dimensi. Untuk elemen ke- i dengan indeks *batch* i_N dan indeks dimensi i_C , GN mendefinisikan himpunan elemen yang dinormalisasi bersama sebagai

$$S_i^{\text{GN}} = \left\{ k \mid k_N = i_N, \left\lfloor \frac{k_C}{C/G} \right\rfloor = \left\lfloor \frac{i_C}{C/G} \right\rfloor \right\}, \quad (2.13)$$

dengan k_N dan k_C masing-masing adalah indeks *batch* dan indeks dimensi dari elemen k . Persamaan (2.13) menyatakan bahwa dua elemen berada dalam kelompok normalisasi yang sama apabila keduanya berasal dari sampel yang sama ($k_N = i_N$) dan dimensinya masuk dalam rentang kelompok yang sama ($\lfloor k_C/(C/G) \rfloor$).

Setelah himpunan S_i^{GN} terdefinisi, rata-rata dan standar deviasi dihitung dari seluruh elemen dalam kelompok tersebut, yang dirumuskan pada persamaan (2.14) sebagai

$$\mu_i^{\text{GN}} = \frac{1}{|S_i^{\text{GN}}|} \sum_{k \in S_i^{\text{GN}}} a_k, \quad \sigma_i^{\text{GN}} = \sqrt{\frac{1}{|S_i^{\text{GN}}|} \sum_{k \in S_i^{\text{GN}}} (a_k - \mu_i^{\text{GN}})^2}, \quad (2.14)$$

dengan a_k adalah nilai aktivasi elemen ke- k dan $|S_i^{\text{GN}}|$ adalah jumlah elemen dalam kelompok. Selanjutnya, setiap elemen dinormalisasi dan ditransformasikan menggunakan parameter yang dapat dipelajari, yang dirumuskan pada persamaan (2.15) sebagai

$$\tilde{a}_i = \frac{\gamma^{\text{GN}}}{\sigma_i^{\text{GN}}} (a_i - \mu_i^{\text{GN}}) + \beta^{\text{GN}}, \quad (2.15)$$

dengan γ^{GN} dan β^{GN} masing-masing merupakan parameter skala dan bias yang dipelajari selama pelatihan.

2.3.1.5 Fungsi Loss

Fungsi *loss* merupakan metode matematis untuk mengukur tingkat kesalahan antara hasil prediksi model dan nilai label sebenarnya. Dalam konteks *deep learning*, nilai *loss* menjadi dasar dalam proses pembaruan parameter agar model mampu menghasilkan prediksi yang lebih akurat (Bishop & Nasrabadi, 2006). Penelitian ini menerapkan dua jenis fungsi *loss*, yaitu *Binary Cross-Entropy* untuk tugas klasifikasi sentimen dan *Sum of Squared Errors* untuk pendeteksian topik berbasis *clustering*, yang dijelaskan sebagai berikut:

1. *Binary Cross-Entropy*

Binary Cross-Entropy (BCE) merupakan fungsi *loss* yang umum digunakan pada tugas klasifikasi biner. Dalam penelitian ini, BCE digunakan untuk mengukur kesalahan prediksi pada analisis sentimen dengan dua kelas, yaitu positif dan negatif. BCE sesuai digunakan karena *output* model berupa probabilitas, sehingga selisih antara probabilitas prediksi dan label sebenarnya dapat dihitung secara langsung (Elharrouss *et al.*, 2025). Secara matematis, BCE untuk satu observasi dirumuskan pada persamaan (2.16) sebagai

$$\mathcal{L}_{\text{BCE}}(y_i, \hat{y}_i) = -[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)], \quad (2.16)$$

dengan $y_i \in \{0, 1\}$ adalah label asli pada observasi ke- i , dan $\hat{y}_i$ adalah probabilitas prediksi model terhadap kelas positif. Nilai BCE akan semakin kecil apabila

probabilitas prediksi model semakin sesuai dengan label sebenarnya. Untuk N observasi, nilai BCE dapat dituliskan seperti pada persamaan (2.17).

$$\mathcal{L}_{\text{BCE}} = \frac{1}{N} \sum_{i=1}^N \mathcal{L}_{\text{BCE}}(y_i, \hat{y}_i). \quad (2.17)$$

2. *Sum of Squared Errors*

Sum of Squared Errors (SSE) merupakan fungsi *loss* yang menghitung total kuadrat selisih antara nilai target dan hasil prediksi model (Khoshvaght *et al.*, 2025). Secara matematis, SSE dinyatakan pada persamaan (2.18).

$$\mathcal{L}_{\text{SSE}} = \sum_{i=1}^N (y_i - \hat{y}_i)^2, \quad (2.18)$$

dengan y_i adalah nilai asli ke- i , $\hat{y}_i$ adalah nilai hasil prediksi ke- i , dan N adalah jumlah data. Semakin kecil nilai SSE, semakin kecil pula selisih antara nilai asli dan nilai hasil prediksi. Pada metode *clustering*, seperti *K-Means* dan *Deep Embedded K-Means* (DEKM), SSE digunakan untuk mengukur total kuadrat jarak antara setiap titik data dengan *centroid* dari *cluster* yang sesuai.

3. *Mean Squared Error*

Mean Squared Error (MSE) merupakan fungsi *loss* yang menghitung rata-rata kuadrat selisih antara nilai prediksi dan nilai target (Ciampiconi *et al.*, 2024). Secara matematis, MSE dirumuskan pada persamaan (2.19) sebagai

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2, \quad (2.19)$$

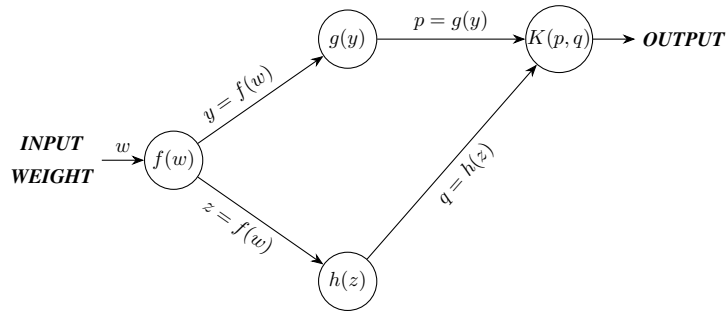
dengan y_i adalah nilai target pada observasi ke- i , $\hat{y}_i$ adalah nilai prediksi model, dan N adalah jumlah observasi. Nilai MSE selalu bernilai non-negatif dan bernilai nol apabila prediksi model identik dengan nilai target. Pada penelitian ini, MSE digunakan sebagai fungsi *loss* pada tahap pra-pelatihan *encoder* untuk mengukur selisih antara *embedding* hasil rekonstruksi decoder dengan *embedding* token asli.

2.3.1.6 *Backpropagation*

Pada *neural network*, proses pelatihan diawali dengan *forward propagation*, di mana *input* x diproses melalui beberapa *hidden layer* hingga menghasilkan *output* o . Berdasarkan *output* tersebut, nilai fungsi *loss* $\mathcal{L}$ dihitung untuk mengevaluasi kesesuaian

antara prediksi model dan label sebenarnya. Proses tersebut kemudian diakhiri dengan *backpropagation*, yaitu proses propagasi *error* dari *output layer* menuju *layer* sebelumnya untuk menghitung gradien fungsi *loss* terhadap seluruh bobot jaringan (Aggarwal *et al.*, 2018). Gradien menunjukkan arah dan besar perubahan fungsi *loss* akibat perubahan parameter jaringan, sehingga informasi tersebut dapat digunakan untuk memperbarui parameter secara bertahap menuju nilai *loss* yang lebih kecil.

Misalkan terdapat *neural network* dengan dua jalur yang menghubungkan bobot w ke *output* o sebagaimana ditunjukkan pada Gambar 2.9, dengan $f(w)$ menyatakan fungsi pertama yang menerima *input* w , serta $g(y)$ dan $h(z)$ masing-masing menyatakan fungsi pada jalur pertama dan jalur kedua, dengan $y = f(w)$ dan $z = f(w)$.



Gambar 2.9: Ilustrasi graf komputasi *MLP* dua jalur (Aggarwal *et al.*, 2018), telah diolah kembali.

Karena *output* jaringan bergantung pada bobot melalui fungsi komposisi berlapis, gradien fungsi *loss* terhadap bobot w dihitung menggunakan aturan rantai sebagaimana dinyatakan pada persamaan (2.20).

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial o} \frac{\partial o}{\partial w}. \quad (2.20)$$

Karena *output* o bergantung pada w melalui dua jalur komputasi secara bersamaan, yaitu melalui $p = g(y)$ dengan $y = f(w)$, dan $q = h(z)$ dengan $z = f(w)$, maka turunan $\partial o / \partial w$ dihitung menggunakan aturan rantai multivariabel (*multivariable chain*

rule) sebagai berikut.

$$\begin{aligned}
 \frac{\partial \mathcal{L}}{\partial w} &= \frac{\partial \mathcal{L}}{\partial o} \left(\frac{\partial o}{\partial p} \frac{\partial p}{\partial w} + \frac{\partial o}{\partial q} \frac{\partial q}{\partial w} \right) \\
 &= \frac{\partial \mathcal{L}}{\partial o} \left(\frac{\partial o}{\partial p} \frac{\partial p}{\partial y} \frac{\partial y}{\partial w} + \frac{\partial o}{\partial q} \frac{\partial q}{\partial z} \frac{\partial z}{\partial w} \right) \\
 &= \frac{\partial \mathcal{L}}{\partial o} \left(\underbrace{\frac{\partial K(p, q)}{\partial p} g'(y) f'(w)}_{\text{jalur pertama}} + \underbrace{\frac{\partial K(p, q)}{\partial q} h'(z) f'(w)}_{\text{jalur kedua}} \right).
 \end{aligned} \tag{2.21}$$

Persamaan (2.21) menunjukkan bahwa gradien total diperoleh dari penjumlahan kontribusi gradien pada setiap jalur komputasi. Pada *neural network* yang lebih kompleks, suatu bobot dapat memengaruhi *output* melalui lebih dari dua jalur. Dalam kondisi tersebut, gradien dihitung dengan mengakumulasi kontribusi dari seluruh jalur yang menghubungkan bobot tersebut ke *output* jaringan.

Misalkan $\mathcal{P}$ menyatakan himpunan seluruh jalur yang menghubungkan suatu *hidden unit* menuju *output* jaringan. Untuk *neural network* dengan *hidden unit* $h_1, h_2, \dots, h_k$ yang terhubung ke *output* o , dengan $w_{(h_{r-1}, h_r)}$ menyatakan bobot yang menghubungkan unit h_{r-1} ke h_r , gradien fungsi *loss* terhadap bobot tersebut dinyatakan pada persamaan (2.22).

$$\frac{\partial \mathcal{L}}{\partial w_{(h_{r-1}, h_r)}} = \frac{\partial \mathcal{L}}{\partial o} \left[\sum_{p \in \mathcal{P}} \frac{\partial o}{\partial h_k} \prod_{i=r}^{k-1} \frac{\partial h_{i+1}}{\partial h_i} \right] \frac{\partial h_r}{\partial w_{(h_{r-1}, h_r)}}. \tag{2.22}$$

Bagian dalam tanda kurung pada persamaan (2.22) menyatakan akumulasi pengaruh *hidden unit* h_r terhadap *loss* melalui seluruh jalur yang ada. Besaran ini disebut *error signal* dan didefinisikan pada persamaan (2.23).

$$\Delta(h_r, o) = \frac{\partial \mathcal{L}}{\partial h_r}. \tag{2.23}$$

Pada *neural network* dengan *hidden layer* mendalam, jumlah jalur tersebut dapat berjumlah sangat banyak sehingga perhitungan secara langsung tidak efisien dilakukan. Oleh karena itu, *backpropagation* menghitung *error signal* secara bertahap mulai dari *output layer* menuju *layer* sebelumnya. Informasi *error* yang telah dihitung pada suatu unit dapat digunakan kembali untuk menghitung *error signal* pada unit-unit yang berada lebih awal dalam jaringan. Rekursi tersebut dinyatakan pada persamaan (2.24), dengan a_h menyatakan nilai *pre-aktivasi* unit h sebelum fungsi aktivasi $f(\cdot)$ diterapkan.

$$\Delta(h_r, o) = \sum_{h: h_r \rightarrow h} f'(a_h) w_{(h_r, h)} \Delta(h, o), \quad (2.24)$$

dengan nilai awal $\Delta(o, o) = \partial \mathcal{L} / \partial o$ yang dihitung langsung dari fungsi *loss*. Gradien terhadap bobot $w_{(h_{r-1}, h_r)}$ kemudian diperoleh sebagai berikut.

$$\frac{\partial \mathcal{L}}{\partial w_{(h_{r-1}, h_r)}} = \Delta(h_r, o) \cdot h_{r-1} \cdot f'(a_{h_r}). \quad (2.25)$$

Setelah seluruh gradien diperoleh, parameter jaringan diperbarui menggunakan algoritma optimasi untuk meminimalkan nilai fungsi *loss*.

2.3.1.7 Optimasi *Adaptive Moment Estimation with Decoupled Weight Decay*

Proses optimasi merupakan tahapan dalam model *deep learning* yang bertujuan mencari parameter sehingga nilai fungsi *loss* selama pelatihan menjadi sekecil mungkin (Zhang *et al.*, 2023). Beberapa metode optimasi yang umum digunakan dalam pelatihan *neural network* antara lain *Stochastic Gradient Descent* (SGD), RMSProp, Adam, dan AdamW. Pada penelitian ini, metode optimasi yang digunakan adalah AdamW, yang merupakan pengembangan dari algoritma Adam.

Adaptive Moment Estimation (Adam) merupakan algoritma optimasi yang menggabungkan mekanisme *momentum* dengan penyesuaian *learning rate* secara adaptif (Kingma & Ba, 2017). Pada setiap iterasi ke- t , Adam menghitung *moving average* dari gradien orde pertama dan orde kedua untuk memperbarui parameter model. Misalkan $\mathbf{g}^{(t)} = \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}^{(t-1)})$ merepresentasikan gradien fungsi *loss* terhadap seluruh parameter $\mathbf{w}$ pada iterasi ke- t , maka estimasi momen pertama dan momen kedua dihitung masing-masing menggunakan persamaan (2.26) dan (2.27) sebagai

$$\mathbf{m}^{(t)} = \beta_1 \mathbf{m}^{(t-1)} + (1 - \beta_1) \mathbf{g}^{(t)}, \quad (2.26)$$

$$\mathbf{v}^{(t)} = \beta_2 \mathbf{v}^{(t-1)} + (1 - \beta_2) (\mathbf{g}^{(t)})^2, \quad (2.27)$$

dengan β_1 dan β_2 menyatakan koefisien peluruhan untuk masing-masing estimasi momen. Karena kedua estimasi diinisialisasi dengan nilai nol, koreksi bias diperlukan agar estimasi pada iterasi awal tidak cenderung mendekati nol. Koreksi bias tersebut dinyatakan pada persamaan (2.28).

$$\hat{\mathbf{m}}^{(t)} = \frac{\mathbf{m}^{(t)}}{1 - \beta_1^t}, \quad \hat{\mathbf{v}}^{(t)} = \frac{\mathbf{v}^{(t)}}{1 - \beta_2^t}. \quad (2.28)$$

Selanjutnya, parameter model diperbarui menggunakan persamaan (2.29) sebagai

$$\mathbf{w}^{(t)} = \mathbf{w}^{(t-1)} - \alpha \frac{\hat{\mathbf{m}}^{(t)}}{\sqrt{\hat{\mathbf{v}}^{(t)} + \varepsilon}}, \quad (2.29)$$

dengan α menyatakan *learning rate* dan ε merupakan konstanta bernilai kecil yang berfungsi menjaga stabilitas numerik.

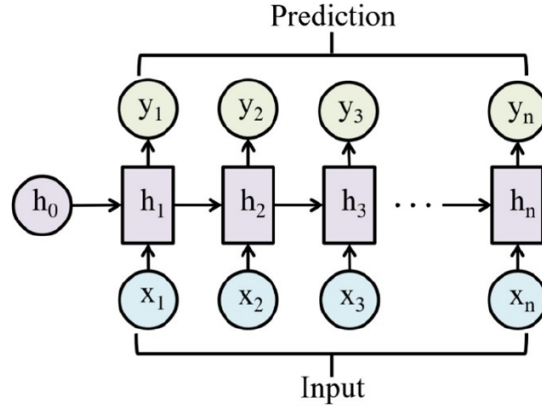
Meskipun Adam memiliki konvergensi yang relatif cepat, mekanisme regularisasinya memiliki kelemahan karena *weight decay* diterapkan bersamaan dengan pembaruan gradien, sehingga efek regularisasi menjadi kurang optimal. Untuk mengatasi hal tersebut, dikembangkan algoritma *Adaptive Moment Estimation with Decoupled Weight Decay* (AdamW) yang memisahkan mekanisme *weight decay* dari pembaruan gradien utama. Pemisahan ini memungkinkan regularisasi bekerja secara lebih konsisten sehingga kemampuan generalisasi model dapat meningkat. Pembaruan parameter pada AdamW dirumuskan pada persamaan (2.30) sebagai

$$\mathbf{w}^{(t)} = \mathbf{w}^{(t-1)} - \alpha \left(\frac{\hat{\mathbf{m}}^{(t)}}{\sqrt{\hat{\mathbf{v}}^{(t)} + \varepsilon}} + \lambda \mathbf{w}^{(t-1)} \right), \quad (2.30)$$

dengan α menyatakan *learning rate* dan λ merupakan koefisien *weight decay*. Pemisahan tersebut membuat proses regularisasi bekerja terpisah dari adaptasi gradien sehingga AdamW mampu memberikan kemampuan generalisasi yang lebih baik dibandingkan Adam.

2.4 Recurrent Neural Network

Recurrent Neural Network (RNN) merupakan arsitektur *neural network* untuk memodelkan data berurutan, seperti teks dan deret waktu (Mienye *et al.*, 2024). RNN memiliki koneksi berulang yang memungkinkan informasi dari langkah sebelumnya dipertahankan, sehingga mampu menangkap ketergantungan waktu antareleman (Schmidt, 2019). Pada setiap waktu t , *input* $\mathbf{x}_t$ diproses untuk menghasilkan *hidden state* $\mathbf{h}_t$ sebagai representasi informasi hingga waktu tersebut, yang kemudian digunakan untuk menghasilkan *output* $\hat{\mathbf{y}}_t$.



Gambar 2.10: Arsitektur RNN (Mienye *et al.*, 2024).

Arsitektur dasar RNN ditunjukkan pada Gambar 2.10. Koneksi dari h_{t-1} ke h_t menunjukkan bahwa pemrosesan pada setiap langkah waktu dipengaruhi oleh *input* saat ini dan informasi dari langkah sebelumnya. Misalkan diberi urutan *input* $\{\mathbf{x}_t\}_{t=1}^T$, *hidden state* $\{\mathbf{h}_t\}_{t=1}^T$, dan *output* $\{\hat{\mathbf{y}}_t\}_{t=1}^T$, maka pembaruan *hidden state* dirumuskan pada persamaan (2.31).

$$h_t = f(\mathbf{w}_x \mathbf{x}_t + \mathbf{w}_h h_{t-1} + b_h), \quad (2.31)$$

dengan $\mathbf{w}_x$ sebagai bobot dari *input* ke *hidden layer*, $\mathbf{w}_h$ sebagai bobot *recurrent*, b_h sebagai bias, dan $f(\cdot)$ sebagai fungsi aktivasi nonlinier. Nilai awal h_0 umumnya diinisialisasi sebagai vektor nol. Selanjutnya, *output* dihitung dari *hidden state* melalui persamaan (2.32).

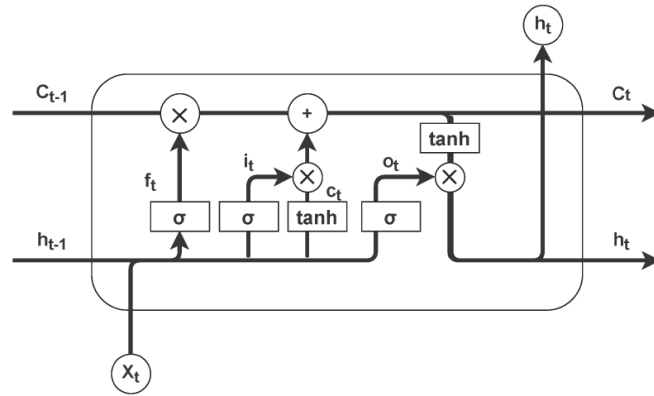
$$\hat{\mathbf{y}}_t = f(\mathbf{w}_o \mathbf{h}_t + b_o), \quad (2.32)$$

dengan $\mathbf{w}_o$ sebagai bobot dari *hidden layer* ke *output* dan b_o sebagai bias. Persamaan (2.31) dan (2.32) bersama-sama menggabungkan informasi saat ini dan konteks masa lalu untuk menghasilkan prediksi.

Meskipun efektif dalam memodelkan data berurutan, RNN menghadapi masalah *vanishing* dan *exploding gradients*, terutama pada ketergantungan jangka panjang. Masalah ini muncul akibat propagasi gradien yang melibatkan perkalian berulang sepanjang urutan waktu, sehingga nilai gradien dapat mengecil atau membesar secara drastis. Akibatnya, model kesulitan mempelajari hubungan jangka panjang, sehingga dikembangkan varian seperti LSTM (Schmidt, 2019).

2.5 Long Short-Term Memory

Long Short-Term Memory (LSTM) merupakan arsitektur pengembangan dari RNN yang dirancang untuk mengatasi keterbatasan RNN dalam mempelajari dependensi jangka panjang. LSTM memperkenalkan *cell state* sebagai jalur utama penyimpanan informasi jangka panjang, serta *gating mechanism* untuk mengendalikan informasi yang perlu disimpan, diperbarui, atau dihapus pada setiap langkah waktu (Mienye *et al.*, 2024). Melalui ketiga *gate* tersebut, LSTM mengatur informasi yang masuk, dihapus, dan diteruskan sebagai *output*, sehingga model dapat mempertahankan informasi yang relevan meskipun urutan inputnya panjang. Ilustrasi arsitektur LSTM pada satu langkah waktu ditunjukkan pada Gambar 2.11. Pada ilustrasi tersebut, *input* x_t , *hidden state* sebelumnya h_{t-1} , dan *cell state* sebelumnya c_{t-1} digunakan bersama untuk menghasilkan *cell state* c_t dan *hidden state* h_t pada waktu t (Mienye *et al.*, 2024).



Gambar 2.11: Arsitektur dasar LSTM (Mienye *et al.*, 2024).

LSTM memiliki tiga *gate* utama yang bekerja pada setiap langkah waktu t . *Forget gate* f_t memutuskan informasi mana dari *cell state* sebelumnya yang perlu dibuang, *input gate* i_t mengatur seberapa besar informasi baru z_t yang dimasukkan ke memori, dan *output gate* o_t menentukan bagian mana dari *cell state* yang diteruskan sebagai *hidden state*. Operasi keseluruhan LSTM pada waktu ke- t dirumuskan pada persamaan (2.33) sampai (2.38).

$$f_t = \sigma(\tilde{f}_t), \quad \tilde{f}_t = \mathbf{w}_f^\top \mathbf{x}_t + r_f h_{t-1} + b_f, \quad (2.33)$$

$$i_t = \sigma(\tilde{i}_t), \quad \tilde{i}_t = \mathbf{w}_i^\top \mathbf{x}_t + r_i h_{t-1} + b_i, \quad (2.34)$$

$$z_t = \tanh(\tilde{z}_t), \quad \tilde{z}_t = \mathbf{w}_z^\top \mathbf{x}_t + r_z h_{t-1} + b_z, \quad (2.35)$$

$$c_t = f_t c_{t-1} + i_t z_t, \quad (2.36)$$

$$o_t = \sigma(\tilde{o}_t), \quad \tilde{o}_t = \mathbf{w}_o^\top \mathbf{x}_t + r_o h_{t-1} + b_o, \quad (2.37)$$

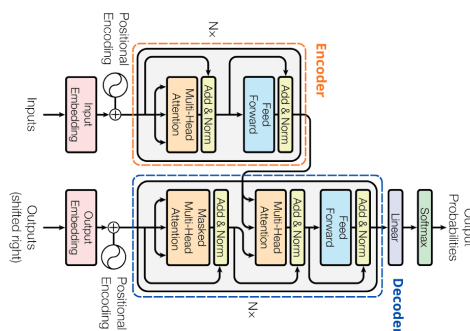
$$h_t = o_t \tilde{h}_t, \quad \tilde{h}_t = \tanh(c_t). \quad (2.38)$$

Pada setiap persamaan *gate*, pra-aktivasi ($\tilde{\cdot}$) dihitung dari kombinasi linier antara *input* saat ini x_t , *hidden state* sebelumnya h_{t-1} , dan bias b , yang kemudian diproses dengan fungsi aktivasi sigmoid $\sigma(\cdot)$. Persamaan (2.36) merupakan bagian utama LSTM dalam memperbarui memori. *Cell state* diperbarui dengan mempertahankan sebagian informasi lama yang dikendalikan oleh *forget gate* f_t , sekaligus menyerap informasi baru yang diatur oleh *input gate* i_t . Selanjutnya, *hidden state* h_t dihasilkan dengan memfilter *cell state* melalui *output gate* o_t .

2.6 Transformer

Penjelasan pada subbab ini mengacu pada penelitian (Vaswani *et al.*, 2023), kecuali jika bersumber dari referensi lain. Transformer merupakan arsitektur *neural network* untuk memodelkan data berurutan yang memanfaatkan mekanisme *attention* tanpa menggunakan koneksi *recurrent* ataupun konvolusional. Mekanisme ini memungkinkan model menilai hubungan antareleman dalam suatu urutan melalui pemberian bobot yang berbeda. Informasi yang relevan kemudian dapat diproses secara selektif, sementara setiap elemen dapat berinteraksi secara langsung tanpa bergantung pada pemrosesan berurutan. Transformer menunjukkan keunggulan pada berbagai bidang seperti *language modeling* (Radford & Narasimhan, 2018), peramalan deret waktu (Nie *et al.*, 2023), dan *computer vision* (Dosovitskiy *et al.*, 2021).

2.6.1 Arsitektur Transformer



Gambar 2.12: Arsitektur dasar Transformer (Vaswani *et al.*, 2023), telah diolah kembali.

Arsitektur Transformer tersusun atas *encoder* dan *decoder*, yang masing-masing terdiri dari tumpukan beberapa *layer* identik, sebagaimana ditunjukkan pada

Gambar 2.12. *Encoder* berfungsi memetakan urutan *input* ke dalam representasi laten yang bersifat kontekstual, sedangkan *decoder* menghasilkan urutan *output* secara autoregresif berdasarkan representasi tersebut. Setiap *layer* pada *encoder* dan *decoder* dilengkapi dengan mekanisme *attention*, transformasi nonlinier, *skip connection*, dan *layer normalization* untuk menjaga kestabilan aliran gradien selama pelatihan.

2.6.2 Word Embedding

Dalam NLP, teks tidak dapat digunakan secara langsung dalam bentuk simbol diskrit, sehingga diperlukan representasi numerik agar model *deep learning* dapat membaca *input*. Pendekatan yang umum digunakan adalah *word embedding*, yaitu pemetaan token ke dalam vektor kontinu berdimensi tetap yang mampu menangkap informasi semantik berdasarkan kemunculan kata dalam konteks tertentu (Almeida & Xexéo, 2023). Representasi vektor ini memungkinkan kata-kata dengan makna atau penggunaan yang serupa memiliki representasi yang saling berdekatan dalam ruang vektor.

Pada arsitektur Transformer, setiap token pada posisi ke- t direpresentasikan sebagai vektor *embedding* $e_t \in \mathbb{R}^{d_{\text{model}}}$. Urutan *input* $(x_1, x_2, \dots, x_S)$ kemudian diubah menjadi urutan vektor kontinu $(e_1, e_2, \dots, e_S)$. Representasi ini selanjutnya digunakan sebagai *input* ke bagian *encoder*.

2.6.3 Encoder

Secara umum, *encoder* merupakan komponen *neural network* yang berfungsi memetakan data *input* ke dalam suatu representasi laten yang mempertahankan informasi penting dari data tersebut (Goodfellow *et al.*, 2016). Pada arsitektur Transformer, *encoder* berfungsi membentuk representasi kontekstual dari urutan token *input*. Struktur *encoder* terdiri atas L *layer* yang identik, dengan setiap *layer* memuat dua sub-*layer* utama, yaitu *multi-head self-attention* dan *position-wise feed-forward network*. Setiap sub-*layer* diikuti oleh mekanisme *skip connection* dan *layer normalization*, sehingga keluaran setiap sub-*layer* dinyatakan sebagai $\text{LayerNorm}(\mathbf{x} + \text{Sublayer}(\mathbf{x}))$. Seluruh sub-*layer* menghasilkan representasi dengan dimensi yang sama, yaitu d_{model} , agar *skip connection* dapat diterapkan secara konsisten. *Output* dari *encoder* pada *layer* terakhir berupa sekumpulan vektor representasi kontekstual yang kemudian digunakan oleh *decoder* untuk menghasilkan keluaran.

2.6.4 Mekanisme *Attention*

Mekanisme *attention* memungkinkan model memusatkan pemrosesan pada bagian-bagian *input* yang paling relevan terhadap suatu tugas tertentu. Model sekuensial umumnya memadatkan informasi dari suatu urutan ke dalam satu vektor tetap. Sementara itu, *attention* memberikan bobot adaptif pada setiap elemen *input*, sehingga setiap token dapat berinteraksi langsung dengan token lain dalam urutan (Luong *et al.*, 2015).

2.6.4.1 *Scaled Dot-Product Attention*

Mekanisme *scaled dot-product attention* digunakan untuk mengukur hubungan relevansi antar token melalui tiga komponen utama, yaitu *query* ($\mathbf{Q}$), *key* ($\mathbf{K}$), dan *value* ($\mathbf{V}$). Komponen *query* merepresentasikan informasi yang digunakan untuk pencarian konteks, *key* berfungsi sebagai representasi pembanding, sedangkan *value* memuat informasi yang akan dikombinasikan berdasarkan tingkat relevansi antara *query* dan *key*. Ketiga matriks tersebut diperoleh dari hasil proyeksi linier terhadap matriks *input* $\mathbf{X} \in \mathbb{R}^{S \times d_{\text{model}}}$ sebagaimana dinyatakan pada persamaan (2.39).

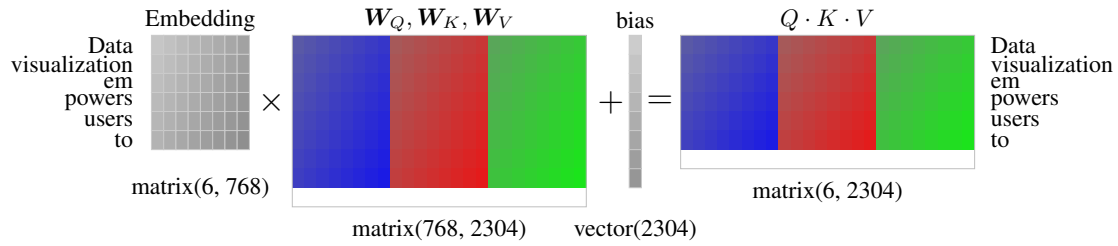
$$\mathbf{Q} = \mathbf{X}\mathbf{W}^Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}^K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}^V, \quad (2.39)$$

dengan $\mathbf{W}^Q, \mathbf{W}^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ dan $\mathbf{W}^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ menyatakan matriks bobot yang dipelajari selama proses pelatihan. Tingkat relevansi antar token dihitung melalui operasi perkalian titik (*dot product*) antara *query* dan *key*, kemudian diskalakan menggunakan $\sqrt{d_k}$ untuk menjaga kestabilan numerik pada proses pelatihan. Hasil perhitungan tersebut selanjutnya dinormalisasi menggunakan fungsi *softmax* dan digunakan untuk memberikan bobot pada matriks *value*, sebagaimana dirumuskan pada persamaan (2.40) sebagai

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}. \quad (2.40)$$

2.6.4.2 *Multi-Head Attention*

Multi-head attention merupakan pengembangan dari mekanisme *scaled dot-product attention* yang memungkinkan model memproses informasi secara paralel melalui beberapa *head*. Setiap *head* menggunakan parameter proyeksi yang berbeda sehingga dapat menangkap pola hubungan yang berbeda antar token secara simultan. Ilustrasi proses pembentukan $\mathbf{Q}$, $\mathbf{K}$, dan $\mathbf{V}$ dari *input embedding* ditunjukkan pada Gambar 2.13.



Gambar 2.13: Ilustrasi proses pembentukan *query*, *key*, dan *value* dari *input embedding* (Pradipta Hardinatha *et al.*, 2025), telah diolah kembali.

Secara matematis, pembentukan *query*, *key*, dan *value* untuk setiap *head* ke-*h* dinyatakan pada persamaan (2.41) sebagai

$$Q^{(h)} = QW_h^Q, \quad K^{(h)} = KW_h^K, \quad V^{(h)} = VW_h^V, \quad (2.41)$$

dengan *output* setiap *head* diperoleh melalui $\text{head}_h = \text{Attention}(Q^{(h)}, K^{(h)}, V^{(h)})$. Setelah itu, seluruh *head* digabungkan dan diproyeksikan kembali sebagaimana dirumuskan pada persamaan (2.42).

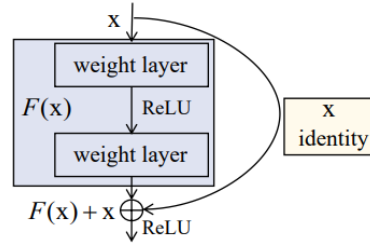
$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_H) W^O. \quad (2.42)$$

2.6.5 Skip Connection

Skip connection atau *residual connection* merupakan mekanisme yang memungkinkan *input* suatu *layer* diteruskan secara langsung ke *layer* berikutnya tanpa harus sepenuhnya melalui transformasi nonlinier. Secara umum, jika suatu sub-*layer* direpresentasikan sebagai fungsi $\mathcal{F}(\cdot)$ dengan *input* x , maka *output* residual dapat dirumuskan pada persamaan (2.43) sebagai berikut.

$$y = \mathcal{F}(x) + x. \quad (2.43)$$

Pada *skip connection*, model mempelajari representasi baru $\mathcal{F}(x)$ dan mempertahankan informasi awal dari x . Jaringan hanya akan mempelajari perubahan atau koreksi terhadap *input* awal, sehingga proses pelatihan menjadi lebih stabil dan mengurangi masalah *vanishing gradient* pada jaringan yang dalam (Xu *et al.*, 2025). Ilustrasi mekanisme *skip connection* ditunjukkan pada Gambar 2.14.



Gambar 2.14: Ilustrasi *skip connection* (Xu *et al.*, 2025).

2.6.6 Decoder

Secara umum, *decoder* merupakan komponen jaringan saraf yang bertugas mentransformasikan representasi laten menjadi *output* yang sesuai dengan tujuan pembelajaran (Goodfellow *et al.*, 2016). Pada arsitektur Transformer, *decoder* bertugas menghasilkan urutan *output* berdasarkan representasi laten yang diperoleh dari *encoder*. *Decoder* bekerja secara autoregresif, yaitu setiap prediksi token bergantung pada token-token *output* sebelumnya serta informasi yang diberikan oleh *encoder*. Struktur *decoder* terdiri atas tumpukan L layer identik. Setiap layer memuat tiga sub-layer utama, yaitu *masked multi-head self-attention*, *encoder-decoder attention*, dan *position-wise feed-forward network*. Masing-masing sub-layer dilengkapi dengan mekanisme *skip connection* dan *layer normalization*.

Pada sub-layer *masked self-attention*, pembatasan akses terhadap token masa depan diterapkan menggunakan matriks *look-ahead mask* M_{mask} , sehingga prediksi token ke- t , dengan $t = 1, 2, \dots, S$, hanya bergantung pada token-token sebelumnya. Secara matematis, mekanisme ini dapat dirumuskan pada persamaan (2.44) sebagai

$$\text{MaskedAttention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{M}_{\text{mask}}\right) \mathbf{V}, \quad (2.44)$$

dengan M_{mask} berisi nilai 0 pada posisi yang diperbolehkan dan $-\infty$ pada posisi yang dibatasi, sehingga token masa depan tidak berkontribusi dalam perhitungan *attention* setelah normalisasi *softmax*.

2.7 Causal Convolution

Causal convolution merupakan varian dari konvolusi satu dimensi yang dirancang untuk memproses data berurutan dengan mempertahankan urutan temporal (van den Oord *et al.*, 2016). Pada konvolusi standar satu dimensi, *output* pada posisi t dihitung

berdasarkan informasi dari posisi-posisi di sekitarnya, baik sebelum maupun sesudah posisi tersebut. Secara matematis, operasi konvolusi standar dengan *kernel* $w \in \mathbb{R}^k$ terhadap *input* $x \in \mathbb{R}^L$ pada posisi t dirumuskan pada persamaan (2.45) sebagai

$$y_t = \sum_{i=0}^{k-1} w_i \cdot x_{t+i-\lfloor k/2 \rfloor}, \quad (2.45)$$

dengan k menyatakan ukuran *kernel* dan $\lfloor k/2 \rfloor$ merupakan setengah lebar *kernel* yang memungkinkan konvolusi mengakses posisi sebelum dan sesudah t . Pada data berurutan seperti teks, akses terhadap posisi sesudah t artinya model menggunakan informasi dari token-token yang belum diamati, sehingga melanggar asumsi kausalitas.

Untuk mengatasi hal tersebut, *causal convolution* membatasi jangkauan konvolusi hanya pada posisi saat ini dan posisi-posisi sebelumnya (van den Oord *et al.*, 2016). Operasi *causal convolution* pada posisi t dirumuskan pada persamaan (2.46) sebagai

$$y_t = \sum_{i=0}^{k-1} w_i \cdot x_{t-i}, \quad (2.46)$$

sehingga *output* y_t hanya bergantung pada *input* dari posisi $t, t-1, \dots, t-k+1$. Pembatasan ini diimplementasikan melalui penambahan *zero-padding* sebanyak $k-1$ elemen di sisi kiri *input*, sehingga panjang *output* tetap sama dengan panjang *input* tanpa menggunakan informasi dari posisi masa depan. Pada arsitektur xLSTM, *causal convolution* diterapkan di dalam blok sLSTM dan blok mLSTM sebelum pemrosesan *recurrent* untuk menangkap hubungan lokal antar token yang berdekatan (Beck *et al.*, 2024).

2.8 K-Means Clustering

Clustering merupakan metode dalam *unsupervised learning* yang bertujuan mengelompokkan data ke dalam beberapa *cluster* berdasarkan tingkat kemiripan tanpa menggunakan label kelas (Jain *et al.*, 1999). Setiap *cluster* diharapkan memiliki kemiripan yang tinggi antar anggotanya dan perbedaan yang jelas dengan *cluster* lain, sehingga metode ini mampu menemukan pola tersembunyi pada data berdimensi tinggi (Bishop & Nasrabadi, 2006). Berdasarkan keanggotaan data, *clustering* dibedakan menjadi *hard clustering* dan *soft clustering* (Petrus *et al.*, 2019). Pada *hard clustering*, setiap data hanya menjadi anggota satu *cluster*, sedangkan pada *soft clustering*, data dapat memiliki derajat keanggotaan pada lebih dari satu *cluster*.

Salah satu metode *hard clustering* yang umum digunakan adalah *K-Means clustering*.

Metode ini membagi data ke dalam K *cluster* dengan meminimalkan jarak antara setiap titik data dan *centroid* yang merepresentasikannya. Kedekatan data terhadap *centroid* umumnya diukur menggunakan jarak Euclidean (Bishop & Nasrabadi, 2006). Jarak Euclidean antara dua vektor $\mathbf{p}$ dan $\mathbf{q}$ pada ruang berdimensi D didefinisikan pada persamaan (2.47) sebagai

$$\|\mathbf{p} - \mathbf{q}\| = \sqrt{\sum_{d=1}^D (p_d - q_d)^2}, \quad (2.47)$$

dengan $\mathbf{p} = (p_1, p_2, \dots, p_D)$ dan $\mathbf{q} = (q_1, q_2, \dots, q_D)$. Ukuran ini menjadi dasar penugasan setiap data ke *cluster* dengan *centroid* terdekat.

Misalkan diberikan himpunan data $\mathcal{X} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\}$ dengan $\mathbf{x}_i \in \mathbb{R}^D$, dan pusat *cluster* $\boldsymbol{\mu}_k \in \mathbb{R}^D$ untuk $k = 1, 2, \dots, K$ yang diinisialisasi secara acak. Tujuan utama *K-Means* adalah meminimalkan fungsi objektif J yang dirumuskan pada persamaan (2.48) sebagai

$$J = \sum_{i=1}^N \sum_{k=1}^K r_{ik} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2, \quad (2.48)$$

dengan $r_{ik} \in \{0, 1\}$ sebagai indikator keanggotaan *cluster*, di mana $r_{ik} = 1$ apabila $\mathbf{x}_i$ termasuk ke dalam *cluster* ke- k , dan berlaku kendala $\sum_{k=1}^K r_{ik} = 1$ untuk setiap i .

Optimisasi dilakukan secara bergantian antara dua tahap. Pertama, dengan $\boldsymbol{\mu}_k$ tetap, setiap data ditugaskan ke *cluster* dengan *centroid* terdekat sebagaimana dirumuskan pada persamaan (2.49).

$$r_{ik} = \begin{cases} 1, & \text{jika } k = \arg \min_j \|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2, \\ 0, & \text{lainnya.} \end{cases} \quad (2.49)$$

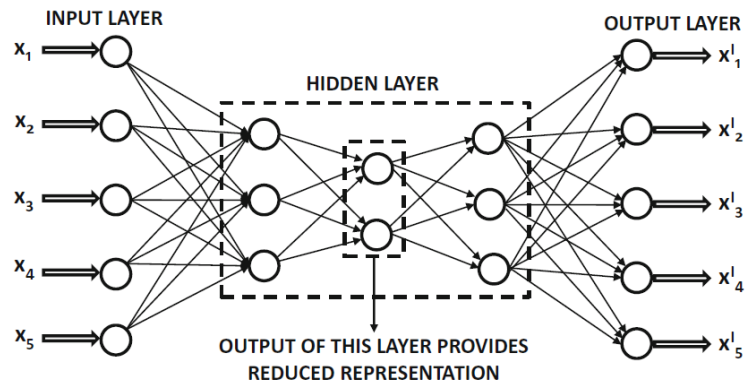
Kedua, dengan r_{ik} tetap, *centroid* diperbarui dengan menetapkan $\frac{\partial J}{\partial \boldsymbol{\mu}_k} = 0$, sehingga diperoleh:

$$\boldsymbol{\mu}_k = \frac{\sum_{i=1}^N r_{ik} \mathbf{x}_i}{\sum_{i=1}^N r_{ik}}. \quad (2.50)$$

Persamaan (2.50) menunjukkan bahwa *centroid* setiap *cluster* merupakan rata-rata dari seluruh anggotanya. Kedua tahap ini diulang sampai perubahan nilai fungsi objektif lebih kecil dari batas toleransi ε atau jumlah iterasi mencapai *maxiter*. Pada penelitian ini, *K-Means* diperluas melalui metode *Deep Embedded K-Means* (DEKM), yang terlebih dahulu mempelajari representasi laten yang lebih informatif melalui *autoencoder* sebelum melakukan pengelompokan.

2.9 Autoencoder

Autoencoder merupakan salah satu arsitektur ANN yang termasuk ke dalam *feed-forward network*. Model ini dilatih untuk menghasilkan kembali data masukan pada bagian *output* sehingga hasil rekonstruksinya sedekat mungkin dengan data asli (Goodfellow *et al.*, 2016). Secara umum, *autoencoder* terdiri atas dua komponen utama, yaitu *encoder* dan *decoder* (Aggarwal *et al.*, 2018). Arsitektur umum *autoencoder* ditunjukkan pada Gambar 2.15.



Gambar 2.15: Ilustrasi *autoencoder* (Aggarwal *et al.*, 2018).

Misalkan terdapat vektor *input* $\mathbf{x} = (x_1, x_2, \dots, x_D)^T$ dengan dimensi D dan vektor laten $\mathbf{z} = (z_1, z_2, \dots, z_d)^T$ dengan dimensi d , di mana $d < D$. Bagian *encoder* bertugas mengubah vektor masukan menjadi representasi berdimensi lebih rendah pada ruang laten melalui suatu fungsi nonlinier (Vincent *et al.*, 2010). Proses tersebut dinyatakan sebagai fungsi $f_\theta: X \rightarrow Z$, dengan θ menyatakan parameter atau bobot pada bagian *encoder*. Hubungan tersebut dapat dituliskan pada persamaan (2.51).

$$\mathbf{z} = f_\theta(\mathbf{x}) \quad (2.51)$$

Selanjutnya, *decoder* menggunakan representasi laten $\mathbf{z}$ untuk membangun kembali data sehingga diperoleh vektor keluaran $\mathbf{y} = (y_1, y_2, \dots, y_D)^T$ yang memiliki dimensi sama dengan data *input* (Vincent *et al.*, 2010). Proses rekonstruksi ini dinyatakan sebagai fungsi $g_{\theta'}: Z \rightarrow Y$, dengan θ' menyatakan parameter pada bagian *decoder*. Hubungan tersebut dituliskan pada persamaan (2.52).

$$\mathbf{y} = g_{\theta'}(\mathbf{z}) \quad (2.52)$$

Proses pembelajaran pada *autoencoder* dilakukan dengan mengoptimalkan parameter

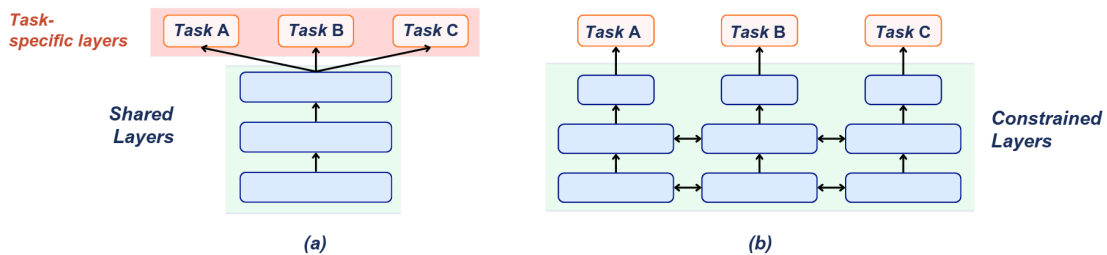
pada *encoder* dan *decoder* agar hasil rekonstruksi mendekati data *input*. Hal tersebut dicapai dengan meminimalkan *reconstruction loss*, yaitu selisih antara data asli dan data hasil rekonstruksi. Melalui proses tersebut, *autoencoder* dapat mempelajari representasi laten yang tetap mempertahankan informasi penting dari data *input* (Vincent *et al.*, 2010). Formulasi *reconstruction loss* dinyatakan pada persamaan (2.53).

$$\mathcal{L} = \frac{1}{n} \sum_{i=1}^n \|x_i - g_{\theta'}(f_{\theta}(x_i))\|^2 \quad (2.53)$$

Pada persamaan (2.53), x_i menyatakan data ke- i , sedangkan $g_{\theta'}(f_{\theta}(x_i))$ merupakan hasil rekonstruksi dari data tersebut. Nilai *reconstruction loss* mengukur seberapa dekat hasil rekonstruksi dengan data asli. Semakin kecil nilai *loss*, semakin baik kemampuan *autoencoder* dalam mempertahankan informasi penting dari data *input*.

2.10 Multi-Task Learning

Penjelasan pada subbab ini mengacu pada penelitian (Ruder, 2017), kecuali jika bersumber dari referensi lain. *Multi-Task Learning* (MTL) merupakan pendekatan pembelajaran yang melatih beberapa tugas secara simultan dengan menggunakan representasi data yang sama. Tugas-tugas yang saling berkaitan dapat saling berkontribusi selama proses pembelajaran, sehingga informasi dari satu tugas membantu pembentukan representasi yang lebih baik untuk tugas lainnya.



Gambar 2.16: Ilustrasi pendekatan MTL: (a) *hard parameter sharing* dan (b) *soft parameter sharing* (Ruder, 2017), telah diolah kembali.

MTL dalam *deep learning* umumnya diterapkan melalui *hard parameter sharing* dan *soft parameter sharing*, sebagaimana diilustrasikan pada Gambar 2.16. Pada *hard parameter sharing*, beberapa *hidden layer* dibagikan secara langsung antar tugas, sementara setiap tugas tetap memiliki *output layer*-nya masing-masing. Pemisahan antara *layer* bersama dan *layer* spesifik memungkinkan model mempelajari informasi umum sekaligus menyesuaikan pembelajaran untuk masing-masing tugas. Sementara itu,

soft parameter sharing menggunakan model terpisah untuk setiap tugas, tetapi menjaga keterkaitan antar model melalui regularisasi parameter. Parameter antar model tidak dibagikan secara langsung, melainkan dibatasi agar memiliki kemiripan tertentu sehingga informasi dari setiap tugas tetap dapat dimanfaatkan.

Penelitian ini menerapkan *hard parameter sharing* untuk mengintegrasikan analisis sentimen dan pendeteksian topik dalam satu model terpadu. xLSTM digunakan sebagai model pembentuk representasi teks yang menghasilkan *embedding* token, kemudian diproses oleh *fully connected layer* untuk memperoleh representasi bersama. Selanjutnya, representasi bersama tersebut dimanfaatkan secara simultan oleh dua tugas, yaitu klasifikasi sentimen melalui MLP dan pendeteksian topik melalui DEKM.

2.11 Metrik Evaluasi

Metrik evaluasi merupakan ukuran kuantitatif yang digunakan untuk menilai kinerja suatu model dalam menyelesaikan tugas tertentu berdasarkan kriteria yang telah ditetapkan. Jenis metrik yang digunakan bergantung pada karakteristik tugas dan model yang diterapkan. Pada model klasifikasi, seperti analisis sentimen, evaluasi umumnya dilakukan menggunakan ukuran berbasis *confusion matrix* untuk menilai ketepatan prediksi terhadap kelas positif maupun negatif (Sokolova & Lapalme, 2009). Sementara itu, evaluasi pendeteksian topik yang bersifat *unsupervised* dilakukan menggunakan *topic coherence* dan *topic diversity*. *Topic coherence* menilai keterkaitan semantik antar kata dalam satu topik (O’Callaghan *et al.*, 2015), sedangkan *topic diversity* menilai keberagaman kata antar topik yang dihasilkan (Dieng *et al.*, 2019).

2.11.1 Metrik Evaluasi Analisis Sentimen

Pada tugas analisis sentimen, evaluasi model dilakukan untuk mengukur kemampuan klasifikasi dalam mengenali sentimen positif dan negatif pada teks. *Confusion matrix* digunakan untuk membandingkan hasil prediksi model dengan label aktual. Matriks tersebut menjadi dasar dalam perhitungan berbagai metrik evaluasi klasifikasi, seperti *accuracy*, *precision*, *recall*, dan *F1-score* (Sokolova & Lapalme, 2009). Ilustrasi *confusion matrix* untuk klasifikasi biner ditunjukkan pada Tabel 2.1.

Tabel 2.1: Ilustrasi *Confusion Matrix*

	Prediksi Positif	Prediksi Negatif
Aktual Positif	<i>True Positive (TP)</i>	<i>False Negative (FN)</i>
Aktual Negatif	<i>False Positive (FP)</i>	<i>True Negative (TN)</i>

Setiap elemen pada *confusion matrix* memiliki makna yang penting dalam menentukan performa model klasifikasi. Penjelasan mengenai masing-masing komponen pada *confusion matrix* disajikan sebagai berikut:

- *True Positive (TP)* : jumlah data positif yang berhasil diprediksi benar sebagai positif oleh model.
- *True Negative (TN)* : jumlah data negatif yang berhasil diprediksi benar sebagai negatif oleh model.
- *False Positive (FP)* : jumlah data negatif yang salah diprediksi sebagai positif (dikenal sebagai *Type I error*).
- *False Negative (FN)* : jumlah data positif yang salah diprediksi sebagai negatif (dikenal sebagai *Type II error*).

Berdasarkan keempat komponen tersebut, sejumlah metrik evaluasi dapat digunakan untuk menilai performa model dalam melakukan klasifikasi sentimen secara kuantitatif. Metrik yang paling umum digunakan meliputi *accuracy*, *precision*, *recall*, dan *F1-score*, yang masing-masing dijelaskan sebagai berikut:

1. *Accuracy*

Accuracy mengukur proporsi total prediksi yang benar terhadap seluruh data uji. Metrik ini memberikan gambaran umum mengenai kinerja model klasifikasi, namun kurang representatif pada kondisi distribusi kelas yang tidak seimbang. Hal ini disebabkan prediksi terhadap kelas mayoritas dapat mendominasi nilai akurasi (Sokolova & Lapalme, 2009). Secara matematis, *accuracy* dirumuskan pada persamaan (2.54).

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}. \quad (2.54)$$

2. *Precision*

Precision menggambarkan tingkat ketepatan prediksi positif yang dihasilkan model terhadap seluruh prediksi positif. Nilai *precision* yang tinggi menunjukkan bahwa model mampu menekan jumlah kesalahan *false positive*, terutama pada kasus ketika kesalahan prediksi positif memiliki dampak yang signifikan (Sokolova & Lapalme, 2009). Nilai *precision* dirumuskan pada persamaan (2.55).

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}. \quad (2.55)$$

3. *Recall*

Recall merepresentasikan kemampuan model dalam mengidentifikasi seluruh data yang benar-benar termasuk ke dalam kelas positif. Nilai *recall* yang tinggi menunjukkan sensitivitas model terhadap kelas positif, meskipun dalam praktiknya peningkatan *recall* sering kali diikuti oleh bertambahnya nilai *false positive* (Sokolova & Lapalme, 2009). Nilai *recall* dinyatakan pada persamaan (2.56).

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}. \quad (2.56)$$

4. *F1-score*

F1-score merupakan rata-rata harmonik antara *precision* dan *recall* yang digunakan untuk mengevaluasi keseimbangan performa model klasifikasi. Metrik ini umum digunakan pada kondisi data tidak seimbang karena mampu memberikan gambaran yang lebih proporsional antara ketepatan prediksi positif dan kemampuan model dalam mengenali kelas positif (Sokolova & Lapalme, 2009). Nilai *F1-score* dirumuskan pada persamaan (2.57).

$$\text{F1-score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}. \quad (2.57)$$

2.11.2 Metrik Evaluasi Pendeteksian Topik

Penelitian ini menggunakan metrik evaluasi berupa *topic coherence* dan *topic diversity* untuk menilai kualitas topik yang dihasilkan oleh model. *Topic coherence* digunakan untuk mengevaluasi keterkaitan semantik antar kata dalam satu topik, sedangkan *topic diversity* digunakan untuk mengevaluasi keberagaman kata antar topik. Penjelasan mengenai kedua metrik tersebut disajikan sebagai berikut:

1. *Topic Coherence*

Topic coherence merupakan ukuran yang digunakan untuk menilai sejauh mana kata-kata dalam satu topik saling berkaitan secara semantik dan membentuk makna yang mudah diinterpretasikan (Röder *et al.*, 2015).

Dalam penelitian ini, digunakan *topic coherence* berbasis *word embedding*, yaitu TC-W2V, yang mengukur kesamaan semantik antar kata menggunakan representasi vektor kata (O’Callaghan *et al.*, 2015). Perhitungan TC-W2V dilakukan dengan merepresentasikan setiap kata dalam topik ke dalam bentuk vektor menggunakan model *word embedding* seperti *word2vec*.

Selanjutnya, untuk setiap pasangan kata dalam himpunan topik $W = \{\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_M\}$, dihitung kesamaan semantik menggunakan *cosine similarity*, sebagaimana dirumuskan pada persamaan (2.58).

$$\text{similarity}(\mathbf{w}_i, \mathbf{w}_j) = \frac{\mathbf{w}_i^\top \mathbf{w}_j}{\|\mathbf{w}_i\| \|\mathbf{w}_j\|}. \quad (2.58)$$

Nilai *topic coherence* kemudian diperoleh dengan menghitung rata-rata kesamaan semantik dari seluruh pasangan kata dalam topik, sebagaimana dirumuskan pada persamaan (2.59) sebagai

$$TC-W2V = \frac{2}{M(M-1)} \sum_{j=2}^M \sum_{i=1}^{j-1} \text{similarity}(\mathbf{w}_i, \mathbf{w}_j), \quad (2.59)$$

dengan $\text{similarity}(\mathbf{w}_i, \mathbf{w}_j)$ menyatakan similaritas antara vektor kata $\mathbf{w}_i$ dan $\mathbf{w}_j$, sedangkan M adalah jumlah kata dalam satu topik. Nilai TC-W2V berada pada rentang $[-1, 1]$ karena merupakan rata-rata dari nilai *cosine similarity*. Nilai yang semakin mendekati 1 menunjukkan bahwa kata-kata dalam topik memiliki keterkaitan semantik yang semakin kuat, sehingga topik yang dihasilkan lebih koheren dan mudah diinterpretasikan.

2. *Topic Diversity*

Topic Diversity (TD) merupakan ukuran yang digunakan untuk menilai keberagaman kata antar topik. Metrik ini menghitung proporsi kata unik dari M kata teratas pada seluruh K topik (Dieng *et al.*, 2019). Misalkan U menyatakan jumlah kata unik dari seluruh daftar M kata teratas pada setiap topik, maka *topic diversity* dirumuskan pada persamaan (2.60) sebagai

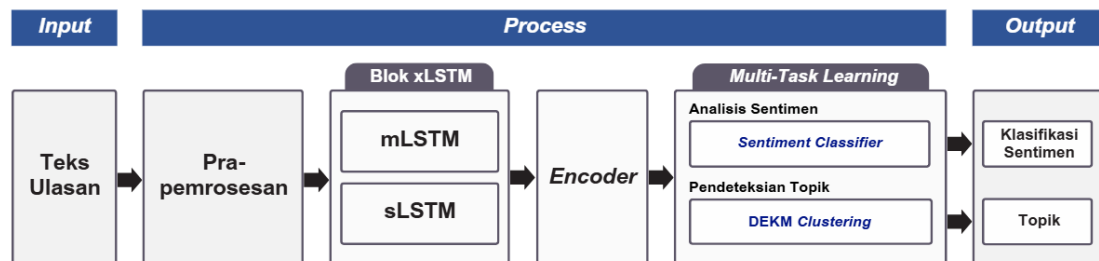
$$\text{Topic Diversity} = \frac{U}{K \times M}, \quad (2.60)$$

dengan U menyatakan jumlah kata unik dari gabungan seluruh daftar kata teratas, K menyatakan jumlah topik, dan M menyatakan jumlah kata teratas yang digunakan pada setiap topik. Nilai *topic diversity* berada pada rentang $(0, 1]$. Nilai yang semakin mendekati 1 menunjukkan bahwa kata-kata yang muncul pada setiap topik semakin beragam sehingga topik yang dihasilkan cenderung tidak redundan, sedangkan nilai yang lebih rendah menunjukkan semakin banyak kata yang berulang pada berbagai topik.

BAB 3

METODE JOINT DOCUMENT LEVEL SENTIMENT-TOPIC MODEL BERBASIS EXTENDED LONG SHORT-TERM MEMORY UNTUK ANALISIS SENTIMEN

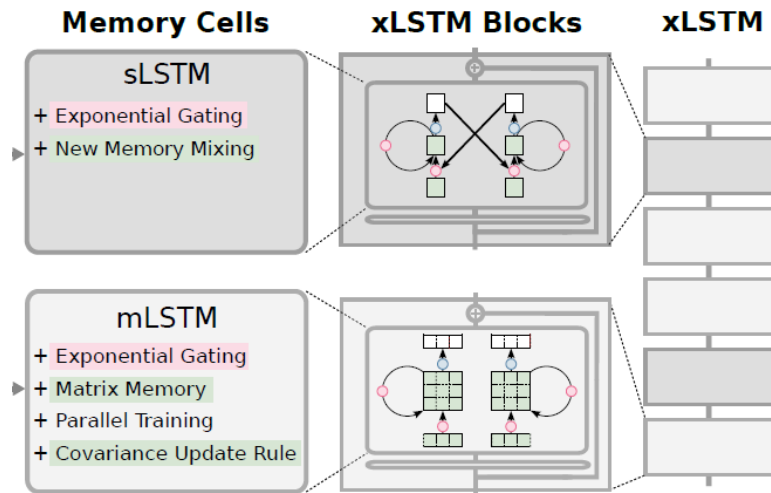
Bab ini menjelaskan arsitektur dan metode yang digunakan dalam pengembangan model *Joint Document Level Sentiment-Topic Model* (JDST) berbasis *Extended Long Short-Term Memory* (xLSTM). Sebelum membahas model JDST, terlebih dahulu dijelaskan dua komponen utama yang mendasarinya, yaitu xLSTM sebagai model representasi teks pada Subbab 3.1 dan DEKM sebagai metode pendeteksian topik pada Subbab 3.2. Selanjutnya, arsitektur model JDST dijelaskan pada Subbab 3.3, sedangkan alur umum proses model dalam melakukan analisis sentimen dan pendeteksian topik secara simultan ditunjukkan pada Gambar 3.1.



Gambar 3.1: Alur proses model JDST berbasis xLSTM

3.1 *Extended Long Short-Term Memory*

Extended Long Short-Term Memory (xLSTM) merupakan pengembangan dari arsitektur LSTM yang bertujuan meningkatkan kapasitas memori dan stabilitas pelatihan pada jaringan *recurrent*, sekaligus mempertahankan kemampuan LSTM dalam mempelajari dependensi jangka panjang (Beck *et al.*, 2024). xLSTM terdiri dari dua komponen utama, yaitu *Scalar Long Short-Term Memory* (sLSTM) dan *Matrix Long Short-Term Memory* (mLSTM), yang membentuk arsitektur xLSTM secara keseluruhan sebagaimana ditunjukkan pada Gambar 3.2.



Gambar 3.2: Arsitektur xLSTM (Beck *et al.*, 2024).

Peningkatan kinerja xLSTM dicapai melalui mekanisme *exponential gating*, yang memungkinkan pembaruan memori dilakukan lebih fleksibel. sLSTM memanfaatkan mekanisme ini bersama dengan *new memory mixing* untuk menggabungkan informasi lama dan baru dalam bentuk memori skalar. Adapun mLSTM menggunakan *matrix memory* yang memungkinkan komputasi dilakukan secara paralel, sehingga lebih efisien untuk data berurutan yang panjang (Alharthi & Mahmood, 2024). Kedua varian ini diintegrasikan dalam satu *residual block backbone* untuk membentuk arsitektur xLSTM secara keseluruhan.

3.1.1 Scalar Long Short-Term Memory

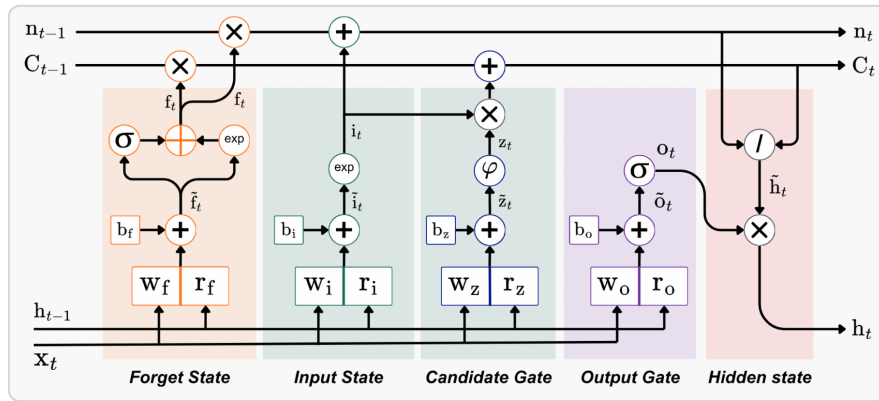
Scalar Long Short-Term Memory (sLSTM) adalah varian xLSTM yang memproses memori dalam bentuk skalar pada setiap unitnya. sLSTM dikembangkan untuk meningkatkan stabilitas gradien serta efisiensi pembaruan memori pada jaringan *recurrent* (Beck *et al.*, 2024). Komponen utama sLSTM adalah mekanisme *exponential gating*, yaitu penggunaan fungsi eksponensial pada *gate* untuk mengontrol aliran informasi yang masuk ke dalam memori.

Pada LSTM, fungsi sigmoid membatasi nilai *gate* dalam rentang $(0, 1)$, sehingga kemampuannya membedakan informasi penting dari yang kurang penting menjadi terbatas. Melalui fungsi eksponensial, *gate* dapat memberikan bobot yang lebih besar pada informasi yang dianggap penting, sehingga model mampu mempertahankan informasi relevan dalam urutan yang panjang. Selain itu, sLSTM memperkenalkan konsep *new memory mixing*, yaitu mekanisme yang menggabungkan informasi dari langkah sebelumnya dan informasi saat ini secara bersamaan saat memperbarui memori.

(Alharthi & Mahmood, 2024). Mekanisme ini berbeda dengan LSTM yang hanya memanfaatkan *forget gate* untuk menentukan seberapa banyak memori lama yang dipertahankan.

3.1.1.1 Arsitektur Sel sLSTM

sLSTM dirancang untuk menangkap hubungan antareleman dalam data berurutan dengan lebih stabil melalui mekanisme pembaruan memori yang teratur. Pada arsitekturnya, sLSTM memperbarui memori skalar pada setiap langkah waktu t melalui mekanisme *recurrent* yang melibatkan *forget gate*, *input gate*, kandidat memori, dan *output gate*, sebagaimana ditunjukkan pada Gambar 3.3.



Gambar 3.3: Arsitektur sLSTM (Alharthi & Mahmood, 2024), telah diolah kembali.

Pada seluruh persamaan berikut, $w \in \mathbb{R}^d$ merepresentasikan vektor bobot *input*, $r \in \mathbb{R}$ merepresentasikan bobot rekuren skalar, dan $b \in \mathbb{R}$ merupakan bias. Proses komputasi sLSTM diawali dengan menghitung pre-aktivasi seluruh *gate* dan kandidat memori secara bersamaan berdasarkan *input* saat ini $x_t \in \mathbb{R}^d$ dan *hidden state* sebelumnya $h_{t-1} \in \mathbb{R}$, karena keempatnya tidak saling bergantung satu sama lain. Hasil pre-aktivasi kemudian digunakan secara berurutan untuk menstabilkan *gate*, memperbarui *cell state* beserta *normalizer*-nya, dan menghasilkan *hidden state* yang baru.

Forget gate $f_t \in \mathbb{R}$ mengatur seberapa besar informasi dari *cell state* sebelumnya yang dipertahankan. Nilai f_t dapat dihitung menggunakan fungsi sigmoid maupun fungsi eksponensial, sebagaimana ditunjukkan pada persamaan (3.1).

$$f_t = \sigma(\tilde{f}_t) \quad \text{atau} \quad \exp(\tilde{f}_t), \quad \tilde{f}_t = w_f^\top x_t + r_f h_{t-1} + b_f. \quad (3.1)$$

Input gate $i_t \in \mathbb{R}$ mengatur seberapa besar informasi baru z_t yang dimasukkan ke dalam memori. sLSTM menggunakan fungsi eksponensial sehingga model dapat

memberikan bobot yang lebih besar pada informasi yang dianggap penting, sebagaimana dirumuskan pada persamaan (3.2).

$$i_t = \exp(\tilde{i}_t), \quad \tilde{i}_t = \mathbf{w}_i^\top \mathbf{x}_t + r_i h_{t-1} + b_i. \quad (3.2)$$

Kandidat memori $z_t \in \mathbb{R}$ merepresentasikan informasi baru yang akan ditambahkan ke dalam memori. Nilai z_t diperoleh melalui fungsi aktivasi non-linier $\varphi(\cdot)$ pada kombinasi linier antara *input* $\mathbf{x}_t$ dan *hidden state* sebelumnya h_{t-1} , sebagaimana dirumuskan pada persamaan (3.3).

$$z_t = \varphi(\tilde{z}_t), \quad \tilde{z}_t = \mathbf{w}_z^\top \mathbf{x}_t + r_z h_{t-1} + b_z. \quad (3.3)$$

Output gate $o_t \in \mathbb{R}$ menentukan besarnya informasi memori yang diteruskan ke *output*. Pre-aktivasinya dihitung pada fase yang sama dengan *gate* lainnya, sebagaimana ditunjukkan pada persamaan (3.4).

$$o_t = \sigma(\tilde{o}_t), \quad \tilde{o}_t = \mathbf{w}_o^\top \mathbf{x}_t + r_o h_{t-1} + b_o. \quad (3.4)$$

Penggunaan fungsi eksponensial pada *forget gate* dan *input gate* berpotensi menyebabkan ketidakstabilan numerik akibat nilai yang meningkat sangat signifikan. Maka, sLSTM menerapkan mekanisme stabilisasi melalui variabel $m_t \in \mathbb{R}$, $i'_t \in \mathbb{R}$, dan $f'_t \in \mathbb{R}$ pada persamaan (3.5)–(3.7) sebelum *gate* digunakan dalam pembaruan memori.

$$m_t = \max(\log(f_t) + m_{t-1}, \log(i_t)), \quad (3.5)$$

$$i'_t = \exp(\log(i_t) - m_t) = \exp(\tilde{i}_t - m_t), \quad (3.6)$$

$$f'_t = \exp(\log(f_t) + m_{t-1} - m_t). \quad (3.7)$$

Cell state $c_t \in \mathbb{R}$ diperbarui menggunakan *gate* yang telah distabilkan, yaitu i'_t dan f'_t , melalui kombinasi antara memori dari langkah sebelumnya c_{t-1} dan kandidat memori baru z_t , sebagaimana dirumuskan pada persamaan (3.8).

$$c_t = f'_t c_{t-1} + i'_t z_t. \quad (3.8)$$

Untuk menjaga kestabilan skala nilai *cell state* pada urutan yang panjang, sLSTM menggunakan variabel normalisasi $n_t \in \mathbb{R}$ yang mengakumulasi pengaruh kumulatif *gate*

sepanjang urutan, sebagaimana dirumuskan pada persamaan (3.9).

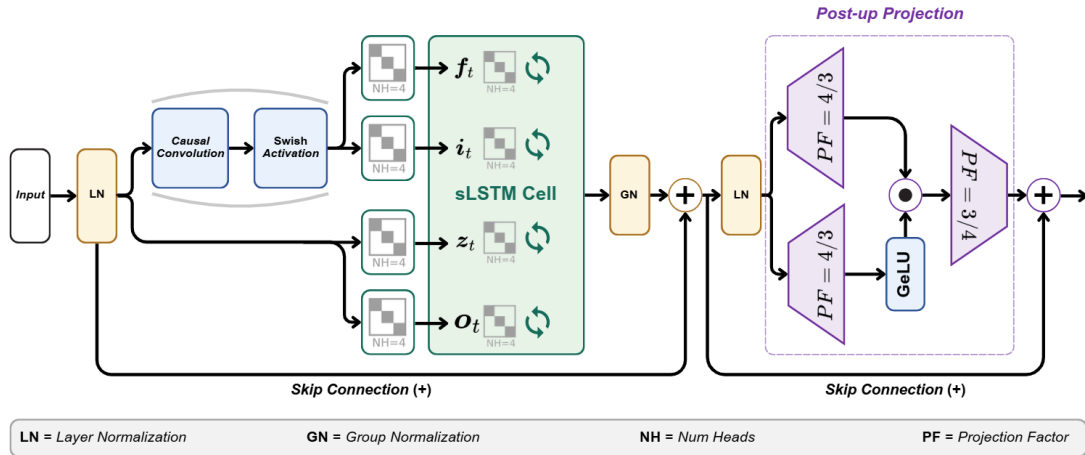
$$n_t = f'_t n_{t-1} + i'_t. \quad (3.9)$$

Terakhir, *hidden state* $h_t \in \mathbb{R}$ diperoleh dengan menormalkan *cell state* menggunakan n_t agar nilainya tetap dalam rentang yang terkendali, kemudian mengalikannya dengan *output gate* o_t , sebagaimana dirumuskan pada persamaan (3.10).

$$h_t = o_t \tilde{h}_t, \quad \tilde{h}_t = \frac{c_t}{n_t}. \quad (3.10)$$

3.1.1.2 Blok sLSTM

Blok sLSTM mengintegrasikan sel sLSTM ke dalam struktur *residual block* untuk meningkatkan kapasitas representasi dan stabilitas pelatihan. Blok ini memproses *input* secara bertahap, mulai dari ekstraksi informasi kontekstual awal, pemrosesan *recurrent* oleh sel sLSTM, hingga penggabungan kembali dengan *input* melalui *skip connection* dan *Layer Normalization*. Struktur internal blok sLSTM disajikan pada Gambar 3.4.



Gambar 3.4: Struktur blok sLSTM (Beck *et al.*, 2024), telah diolah kembali.

Sebelum memasuki sel sLSTM, *input* blok terlebih dahulu dilewatkan melalui lapisan *causal convolution*, yaitu lapisan konvolusi yang memproses setiap token menggunakan informasi dari token tersebut dan token-token sebelumnya, tanpa memanfaatkan token-token setelahnya. Lapisan ini membantu menangkap hubungan antar token yang berdekatan. Fungsi aktivasi Swish pada persamaan (2.9) diterapkan setelah konvolusi tersebut. Hasil konvolusi kemudian diproyeksikan menggunakan blok linier diagonal ke dalam beberapa bagian terpisah yang disebut *head*. Representasi yang terbentuk dalam

setiap *head* digunakan dalam mekanisme sel sLSTM untuk menghitung *input gate*, *forget gate*, *output gate*, serta pembaruan pada *cell state*. *Output* dari setiap *head* sel sLSTM selanjutnya diterapkan *Group Normalization* pada persamaan (2.13).

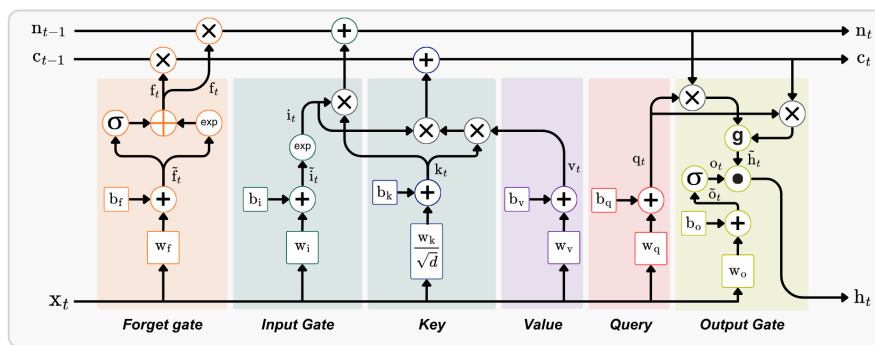
Hasil normalisasi tersebut selanjutnya dipetakan ke ruang berdimensi lebih tinggi menggunakan *gated* MLP dengan fungsi aktivasi GeLU pada persamaan (2.8), lalu dikembalikan ke dimensi semula melalui *down-projection*. Pada tahap akhir, hasil tersebut dikombinasikan dengan *input* awal blok melalui *skip connection* pada persamaan (2.43) dan dinormalisasi menggunakan *Layer Normalization* pada persamaan (2.11).

3.1.2 Matrix Long Short-Term Memory

Matrix Long Short-Term Memory (mLSTM) merupakan varian dari arsitektur xLSTM yang menggunakan representasi memori berbentuk matriks berdimensi $\mathbb{R}^{d \times d}$, berbeda dengan LSTM yang menggunakan vektor. Dengan struktur matriks ini, model dapat menangkap hubungan antar dimensi fitur lebih mendalam karena setiap pasangan dimensi dapat berinteraksi langsung di dalam memori (Beck *et al.*, 2024). Selain itu, karena mLSTM tidak memiliki koneksi *recurrent* pada *gate*-nya, pembaruan memori dapat dilakukan secara paralel sehingga proses pelatihan menjadi lebih efisien dibandingkan LSTM (Alharthi & Mahmood, 2024).

3.1.2.1 Arsitektur Sel mLSTM

mLSTM menggunakan matriks memori dua dimensi $C_t \in \mathbb{R}^{d \times d}$ sebagai pengganti *cell state* berbentuk vektor pada LSTM konvensional, serta menghilangkan koneksi *recurrent* pada seluruh *gate* sehingga komputasi dapat dilakukan secara paralel selama pelatihan. Arsitektur mLSTM ditunjukkan pada Gambar 3.5.



Gambar 3.5: Arsitektur mLSTM (Alharthi & Mahmood, 2024), telah diolah kembali.

Pada seluruh persamaan berikut, $\mathbf{W} \in \mathbb{R}^{d \times d}$ merepresentasikan matriks bobot, $\mathbf{w} \in \mathbb{R}^d$ merepresentasikan vektor bobot, dan $\mathbf{b} \in \mathbb{R}^d$ merupakan vektor bias. Karena seluruh *gate* dan proyeksi *input* pada mLSTM tidak memiliki koneksi *recurrent*, semua pre-aktivasi hanya bergantung pada *input* $\mathbf{x}_t \in \mathbb{R}^d$ sehingga dapat dihitung secara bersamaan. Hasil pre-aktivasi kemudian digunakan secara berurutan untuk menstabilkan *gate*, memperbarui matriks memori beserta *normalizer*-nya, dan menghasilkan *hidden state* yang baru.

Forget gate $f_t \in \mathbb{R}$ mengatur proporsi matriks memori lama yang dipertahankan dan dapat dihitung menggunakan fungsi sigmoid maupun fungsi eksponensial sebagaimana ditunjukkan pada persamaan (3.11).

$$f_t = \sigma(\tilde{f}_t) \quad \text{atau} \quad \exp(\tilde{f}_t), \quad \tilde{f}_t = \mathbf{w}_f^\top \mathbf{x}_t + b_f. \quad (3.11)$$

Input gate $i_t \in \mathbb{R}$ menentukan besarnya informasi baru yang dimasukkan ke dalam memori matriks. *Gate* ini hanya bergantung pada *input* saat ini $\mathbf{x}_t$ dan dihitung menggunakan fungsi eksponensial sebagaimana ditunjukkan pada persamaan (3.12).

$$i_t = \exp(\tilde{i}_t), \quad \tilde{i}_t = \mathbf{w}_i^\top \mathbf{x}_t + b_i. \quad (3.12)$$

Selanjutnya, *input* $\mathbf{x}_t$ diproyeksikan ke dalam tiga vektor representasi yang mengadaptasi prinsip serupa dengan Transformer. Vektor *query* $\mathbf{q}_t \in \mathbb{R}^d$ menentukan informasi yang ingin diambil kembali dari memori, vektor *key* $\mathbf{k}_t \in \mathbb{R}^d$ menentukan letak informasi disimpan dalam memori, dan vektor *value* $\mathbf{v}_t \in \mathbb{R}^d$ adalah informasi yang akan disimpan ke dalam memori. Faktor $\frac{1}{\sqrt{d}}$ berfungsi mencegah nilai *dot-product* yang terlalu besar, serupa dengan prinsip *scaled dot-product attention*. Ketiga vektor tersebut dirumuskan pada persamaan (3.13).

$$\mathbf{k}_t = \frac{1}{\sqrt{d}} \mathbf{W}_k \mathbf{x}_t + \mathbf{b}_k, \quad \mathbf{v}_t = \mathbf{W}_v \mathbf{x}_t + \mathbf{b}_v, \quad \mathbf{q}_t = \mathbf{W}_q \mathbf{x}_t + \mathbf{b}_q. \quad (3.13)$$

Output gate $\mathbf{o}_t \in \mathbb{R}^d$ mengatur aliran informasi dari memori ke *output* jaringan. Seperti *gate* lainnya pada mLSTM, *gate* ini tidak memiliki koneksi *recurrent* dan pre-aktivasinya dihitung bersamaan dengan seluruh komponen lain pada fase pertama, sebagaimana ditunjukkan pada persamaan (3.14).

$$\mathbf{o}_t = \sigma(\tilde{\mathbf{o}}_t), \quad \tilde{\mathbf{o}}_t = \mathbf{W}_o \mathbf{x}_t + \mathbf{b}_o. \quad (3.14)$$

Penggunaan fungsi eksponensial pada *forget gate* dan *input gate* berpotensi

menimbulkan ketidakstabilan numerik. Oleh karena itu, mLSTM menerapkan mekanisme stabilisasi melalui variabel $m_t \in \mathbb{R}$, $i'_t \in \mathbb{R}$, dan $f'_t \in \mathbb{R}$ pada persamaan (3.15)–(3.17) sebelum *gate* digunakan dalam pembaruan memori.

$$m_t = \max(\log(f_t) + m_{t-1}, \log(i_t)), \quad (3.15)$$

$$i'_t = \exp(\log(i_t) - m_t) = \exp(\tilde{i}_t - m_t), \quad (3.16)$$

$$f'_t = \exp(\log(f_t) + m_{t-1} - m_t). \quad (3.17)$$

Matriks memori $\mathbf{C}_t \in \mathbb{R}^{d \times d}$ diperbarui dengan *gate* yang telah distabilkan sebagaimana ditunjukkan pada persamaan (3.18), dengan cara mempertahankan sebagian memori lama dan menambahkan informasi baru dalam bentuk perkalian luar (*outer product*) antara $\mathbf{v}_t$ dan $\mathbf{k}_t$.

$$\mathbf{C}_t = f'_t \mathbf{C}_{t-1} + i'_t \mathbf{v}_t \mathbf{k}_t^\top. \quad (3.18)$$

dengan $f'_t \mathbf{C}_{t-1}$ merupakan memori lama yang dipertahankan, sedangkan $i'_t \mathbf{v}_t \mathbf{k}_t^\top$ adalah informasi baru yang ditambahkan ke dalam memori.

Untuk menjaga kestabilan numerik selama proses pembaruan, mLSTM menggunakan variabel normalisasi $\mathbf{n}_t \in \mathbb{R}^d$ yang mengakumulasi pengaruh *gate* terhadap vektor *key* sepanjang urutan, sebagaimana dirumuskan pada persamaan (3.19).

$$\mathbf{n}_t = f'_t \mathbf{n}_{t-1} + i'_t \mathbf{k}_t. \quad (3.19)$$

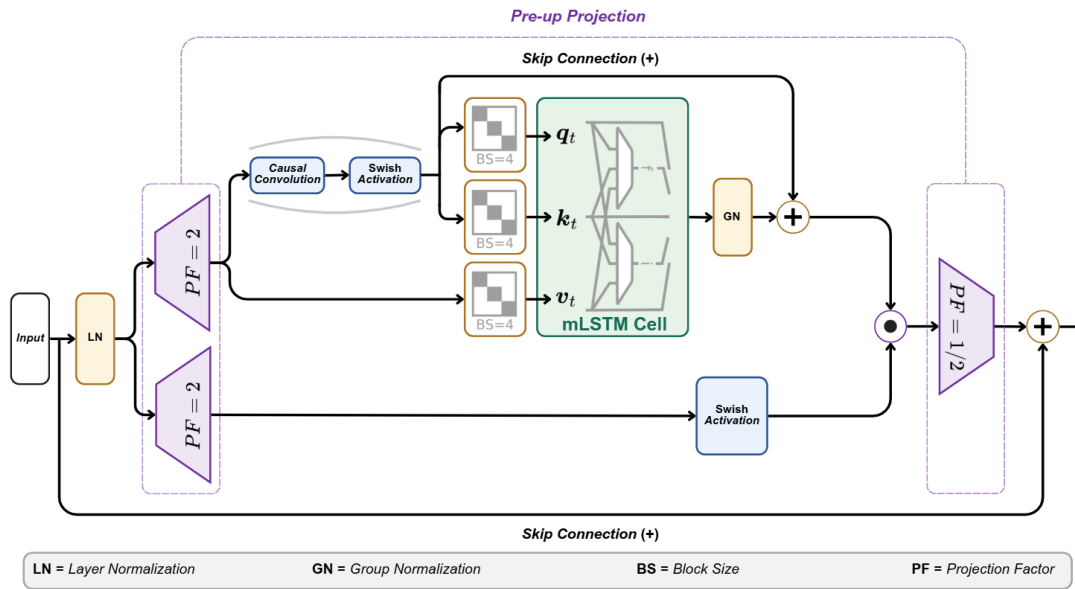
Terakhir, *hidden state* $\mathbf{h}_t \in \mathbb{R}^d$ diperoleh dengan mengambil informasi dari matriks memori $\mathbf{C}_t$ menggunakan *query* $\mathbf{q}_t$ melalui perkalian matriks-vektor $\mathbf{C}_t \mathbf{q}_t \in \mathbb{R}^d$, kemudian hasilnya dinormalisasi agar nilainya tetap dalam rentang yang terkendali, dan dikontrol oleh *output gate* $\mathbf{o}_t$ melalui perkalian *element-wise* (Hadamard) $\odot$, sebagaimana ditunjukkan pada persamaan (3.20).

$$\mathbf{h}_t = \mathbf{o}_t \odot \tilde{\mathbf{h}}_t, \quad \tilde{\mathbf{h}}_t = \frac{\mathbf{C}_t \mathbf{q}_t}{\max(|\mathbf{n}_t^\top \cdot \mathbf{q}_t|, 1)}. \quad (3.20)$$

Penyebut $\max(|\mathbf{n}_t^\top \cdot \mathbf{q}_t|, 1)$ merupakan nilai absolut dari *dot-product* antara $\mathbf{n}_t$ dan $\mathbf{q}_t$, yang memastikan pembagi tidak mendekati nol sehingga hasil normalisasi tetap stabil secara numerik.

3.1.2.2 Blok mLSTM

Blok mLSTM mengintegrasikan sel mLSTM ke dalam struktur *residual block* dengan pendekatan *pre-up projection*. Pada pendekatan ini, *input* blok terlebih dahulu diproyeksikan ke dimensi yang lebih tinggi melalui dua jalur linier sebelum diproses lebih lanjut. Jalur pertama digunakan sebagai *input* utama bagi sel mLSTM, sedangkan jalur kedua menghasilkan *external output gate* yang berfungsi mengontrol *output* akhir blok. Struktur blok mLSTM secara keseluruhan ditunjukkan pada Gambar 3.6.



Gambar 3.6: Arsitektur blok mLSTM (Beck *et al.*, 2024), telah diolah kembali.

Input dari jalur pertama dapat dilewatkan melalui *causal convolution* dengan ukuran *kernel* 4 untuk menangkap hubungan antar token yang berdekatan. *Input* kemudian diproyeksikan melalui proyeksi linier blok-diagonal ke dalam beberapa *head*. Representasi yang terbentuk dalam setiap *head* digunakan dalam sel mLSTM untuk menghitung setiap *gate* dan vektor *query*, *key*, dan *value* yang diproses secara paralel. Seluruh *output head* kemudian digabungkan dan dinormalisasi menggunakan *Group Normalization* pada persamaan (2.13).

Hasil tersebut selanjutnya digabungkan dengan *skip connection* pada persamaan (2.43) dari *input* awal yang dapat dipelajari, lalu dikontrol oleh *external output gate* melalui perkalian elemen per elemen. Pada tahap akhir, *output* diproyeksikan kembali ke dimensi semula melalui *down-projection* dan dinormalisasi menggunakan *Layer Normalization* pada persamaan (2.11). Dengan demikian, dimensi *output* blok tetap konsisten dengan dimensi *input* awal sehingga dapat diteruskan ke blok berikutnya melalui *skip connection* pada persamaan (2.43).

3.2 Deep Embedded K-Means

Deep Embedded K-Means (DEKM) merupakan metode *deep clustering* yang dikembangkan untuk mengatasi keterbatasan *clustering* konvensional pada data berdimensi tinggi (Guo *et al.*, 2021). Berbeda dengan *K-Means* yang mengelompokkan data langsung pada ruang fitur asli, DEKM terlebih dahulu mempelajari representasi laten yang lebih informatif melalui *autoencoder*, kemudian melakukan pengelompokan pada ruang laten tersebut. Proses DEKM terdiri dari tiga tahapan utama yang dilakukan secara bergantian, yaitu pembentukan ruang laten melalui *autoencoder*, pendeteksian *cluster* menggunakan *K-Means*, dan optimasi representasi untuk meningkatkan kualitas struktur *cluster*. Tabel 3.1 menunjukkan notasi yang digunakan dalam metode ini.

Tabel 3.1: Notasi dan interpretasi simbol pada DEKM (Guo *et al.*, 2021).

Simbol	Interpretasi	Simbol	Interpretasi
$d \in \mathbb{N}$	Dimensi ruang <i>input</i>	$\mathbf{X} = [\mathbf{x}_1; \dots; \mathbf{x}_n]$	Matriks data
$e \in \mathbb{N}$	Dimensi ruang laten	$\mathbf{I} \in \mathbb{R}^{e \times e}$	Matriks identitas
$n \in \mathbb{N}$	Jumlah titik data	$\mathbf{V} \in \mathbb{R}^{e \times e}$	Matriks transformasi ortonormal
$K \in \mathbb{N}$	Jumlah <i>cluster</i>	$\mathbf{H} = [\mathbf{h}_1; \dots; \mathbf{h}_n]$	Matriks representasi laten
$\mathbf{x} \in \mathbb{R}^{d \times 1}$	Sebuah titik data	$\mathcal{U} = \{\boldsymbol{\mu}_i\}_{i=1}^k$	Himpunan seluruh <i>centroid</i>
$\boldsymbol{\mu} \in \mathbb{R}^{e \times 1}$	<i>Centroid</i> sebuah <i>cluster</i>	$\mathcal{C} = \{C_1, \dots, C_k\}$	Himpunan <i>cluster</i>
$\mathbf{h} \in \mathbb{R}^{e \times 1}$	Representasi laten sebuah titik data	$f(\cdot)$	Fungsi <i>encoder</i>
		$g(\cdot)$	Fungsi <i>decoder</i>

Pada tahap pertama, *autoencoder* dilatih untuk memperoleh representasi laten dari data *input*. Mekanisme kerja *encoder*, *decoder*, serta proses rekonstruksi pada *autoencoder* serupa dengan mekanisme *autoencoder* yang telah dijelaskan pada Subbab 2.9. Pada tahap ini, *encoder* digunakan untuk memetakan data $\mathbf{x} \in \mathbb{R}^{d \times 1}$ menjadi representasi laten $\mathbf{h} = f(\mathbf{x}) \in \mathbb{R}^{e \times 1}$, sedangkan *decoder* merekonstruksi kembali data melalui $\hat{\mathbf{x}} = g(f(\mathbf{x}))$. Parameter *autoencoder* dipelajari dengan meminimalkan *reconstruction loss* pada Persamaan (3.21).

$$L_1 = \sum_{i=1}^n \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2. \quad (3.21)$$

Setelah proses *pre-training* selesai, *decoder* tidak lagi digunakan dan hanya *encoder* yang dipertahankan untuk menghasilkan representasi laten $\mathbf{H} = f(\mathbf{X})$.

Pada tahap kedua, representasi laten $\mathbf{H}$ digunakan sebagai *input* bagi algoritma *K-Means* untuk memperoleh himpunan *cluster* $C = \{C_1, \dots, C_K\}$ beserta centroid μ_i dari masing-masing *cluster*. Proses ini dilakukan dengan meminimalkan fungsi objektif pada persamaan (3.22) sebagai

$$L_2 = \sum_{i=1}^K \sum_{\mathbf{h} \in C_i} \|\mathbf{h} - \mu_i\|^2, \quad (3.22)$$

dengan K menyatakan jumlah *cluster*. Berdasarkan hasil pengelompokan tersebut, dihitung matriks sebaran dalam-*cluster* (*within-cluster scatter matrix*) sebagai

$$\mathbf{S}_w = \sum_{i=1}^K \sum_{\mathbf{h} \in C_i} (\mathbf{h} - \mu_i)(\mathbf{h} - \mu_i)^\top. \quad (3.23)$$

Matriks $\mathbf{S}_w$ kemudian didekomposisi menggunakan dekomposisi eigen untuk memperoleh matriks ortonormal $\mathbf{V}$ yang tersusun atas vektor-vektor eigen dari $\mathbf{S}_w$. Vektor eigen diurutkan berdasarkan nilai eigennya secara menaik (*ascending*). Selanjutnya, representasi laten ditransformasikan ke ruang baru melalui persamaan (3.24).

$$\mathbf{Y} = \mathbf{V}\mathbf{H}, \quad \mathbf{y} = \mathbf{V}\mathbf{h}, \quad \mathbf{m}_i = \mathbf{V}\mu_i \quad (3.24)$$

Nilai eigen yang lebih kecil menunjukkan sebaran data dalam-*cluster* yang lebih rapat sehingga mengandung informasi *cluster* yang lebih kuat. Oleh karena itu, setelah transformasi, komponen awal pada $\mathbf{Y}$ memuat informasi *cluster* yang lebih kuat dibandingkan komponen akhirnya.

Pada tahap ketiga, DEKM melakukan optimasi representasi pada komponen terakhir dari $\mathbf{Y}$ yang memiliki informasi *cluster* paling lemah. Untuk setiap titik data $\mathbf{y} \in C_i$, dibentuk representasi target $\mathbf{y}'$ yang identik dengan $\mathbf{y}$, kecuali pada komponen terakhirnya yang digantikan oleh komponen terakhir centroid hasil transformasi $\mathbf{m}_i = \mathbf{V}\mu_i$. Optimasi dilakukan dengan meminimalkan fungsi *loss* pada persamaan (3.25).

$$\mathcal{L}_{\text{DEKM}} = \sum_{i=1}^K \sum_{\mathbf{y} \in C_i} \|\mathbf{y} - \mathbf{y}'\|^2. \quad (3.25)$$

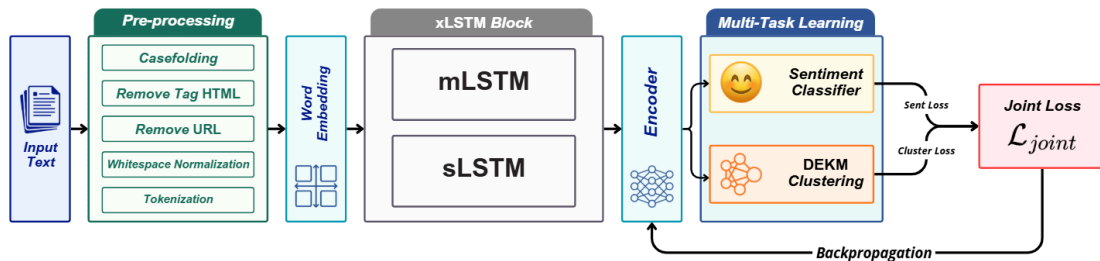
Loss tersebut digunakan untuk memperbarui parameter *encoder* sehingga representasi laten menjadi semakin sesuai terhadap struktur *cluster*. Setelah representasi diperbarui, proses pembentukan *cluster*, perhitungan $\mathbf{S}_w$, dekomposisi eigen, dan optimasi representasi diulang secara bergantian hingga jumlah iterasi maksimum tercapai atau kondisi konvergensi terpenuhi. Prosedur lengkap DEKM dirangkum pada Algoritma 1.

Algoritma 1 DEKM (Guo *et al.*, 2021)**Input:** Matriks data $\mathbf{X}$, jumlah *cluster* K , jumlah iterasi maksimum *Iter***Output:** Himpunan *cluster* $\mathcal{C} = \{C_1, \dots, C_K\}$

- 1: Latih *autoencoder* menggunakan persamaan (3.21)
- 2: **for** $i \leftarrow 1$ **to** *Iter* **do**
- 3: Generate ruang laten melalui *encoder*: $\mathbf{H} = f(\mathbf{X})$
- 4: $(\mathcal{C}, \mathcal{U}) \leftarrow \mathbf{KMeans}(\mathbf{H}, K)$
- 5: Hitung matriks sebaran dalam-*cluster*: $\mathbf{S}_w \leftarrow \sum_{i=1}^K \sum_{\mathbf{h} \in C_i} (\mathbf{h} - \boldsymbol{\mu}_i)(\mathbf{h} - \boldsymbol{\mu}_i)^\top$
- 6: $\mathbf{V} \leftarrow \mathbf{Eig}(\mathbf{S}_w)$ ▷ Vektor eigen diurutkan *ascending* terhadap nilai eigennya
- 7: Optimasi representasi menggunakan persamaan (3.25)
- 8: **end for**
- 9: **return** $\mathcal{C}$

3.3 Model Joint Document Level Sentiment-Topic Model Berbasis Extended Long Short-Term Memory

Model JDST pada penelitian ini menggunakan pendekatan MTL dengan skema *hard parameter sharing* sebagaimana dijelaskan pada subbab 2.10. Arsitektur lengkap model JDST berbasis xLSTM ditunjukkan pada Gambar 3.7.



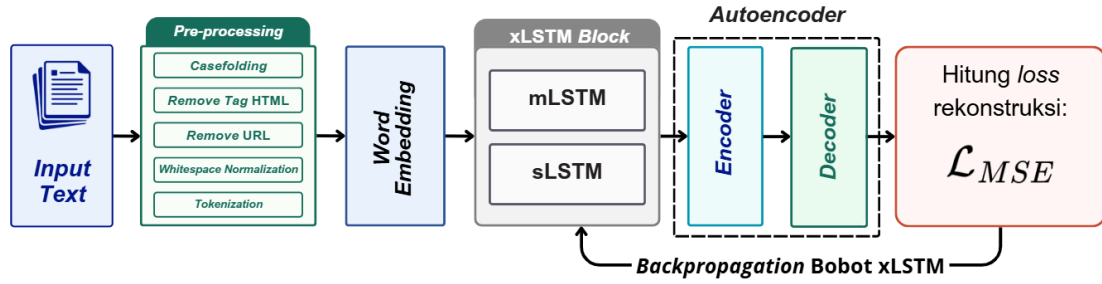
Gambar 3.7: Arsitektur lengkap model JDST berbasis xLSTM

xLSTM digunakan sebagai model representasi teks yang memproses dokumen ulasan menjadi representasi token $\mathbf{h}_t$. Representasi token tersebut kemudian diagregasi dan diproses oleh *encoder* berupa MLP 2 *layer* untuk menghasilkan representasi bersama $\mathbf{z}$ berdimensi lebih rendah, yang selanjutnya digunakan secara bersamaan oleh dua jalur tugas, yaitu analisis sentimen dan pendeteksian topik. xLSTM terdiri atas dua jenis blok, yaitu blok mLSTM dan blok sLSTM, yang dapat dikombinasikan dalam berbagai konfigurasi. Konfigurasi xLSTM ditulis dalam bentuk xLSTM[$a:b$], dengan a menunjukkan jumlah blok mLSTM dan b menunjukkan jumlah blok sLSTM. Penelitian ini menguji empat konfigurasi, yaitu:

1. xLSTM[1:0] (satu blok mLSTM),
2. xLSTM[0:1] (satu blok sLSTM),
3. xLSTM[1:1] – ms (satu blok mLSTM dan satu blok sLSTM dengan urutan mLSTM→sLSTM),
4. xLSTM[1:1] – sm (satu blok mLSTM dan satu blok sLSTM dengan urutan sLSTM→mLSTM).

Selain empat konfigurasi tersebut, penelitian ini juga menggunakan LSTM satu *layer* (LSTM-1L) dan LSTM dua *layer* (LSTM-2L) sebagai model pembanding. LSTM digunakan sebagai model representasi teks dalam arsitektur JDST yang sama, sehingga perbandingan antar model hanya membedakan jenis model representasi teks yang digunakan sementara komponen lainnya tetap sama. Perbandingan ini bertujuan mengukur sejauh mana xLSTM memberikan perbedaan performa dibandingkan LSTM dalam model JDST. Untuk memperjelas penamaan model, notasi JDST-xLSTM[*a:b*] merujuk pada model JDST dengan xLSTM berkonfigurasi [*a:b*], sedangkan JDST-LSTM merujuk pada model JDST dengan LSTM. Notasi ini digunakan secara konsisten pada bab selanjutnya untuk membedakan antar model dalam analisis hasil.

Sebelum digunakan pada tahap *joint training*, xLSTM terlebih dahulu menjalani tahap pra-pelatihan (*pre-training*) bersama dengan komponen *encoder* dan *decoder* dalam kerangka *autoencoder* sebagaimana dijelaskan pada subbab 2.9. Tahap ini dilakukan karena bobot awal xLSTM masih diinisialisasi secara acak, sehingga representasi token yang dihasilkan belum mampu menggambarkan informasi semantik dan topik dokumen dengan baik. Pra-pelatihan bertujuan menghasilkan representasi yang lebih informatif sebelum memasuki tahap *joint training*. Pendekatan ini sejalan dengan penelitian Guo *et al.* (2021) yang menunjukkan bahwa representasi yang dilatih terlebih dahulu dapat menghasilkan kualitas *clustering* yang lebih baik. Bobot xLSTM dan *encoder* yang diperoleh dari tahap ini kemudian digunakan sebagai inisialisasi parameter pada tahap *joint training*, sedangkan *decoder* tidak digunakan lagi. Alur pra-pelatihan secara umum ditunjukkan pada Gambar 3.8.



Gambar 3.8: Alur pra-pelatihan xLSTM dengan *autoencoder*

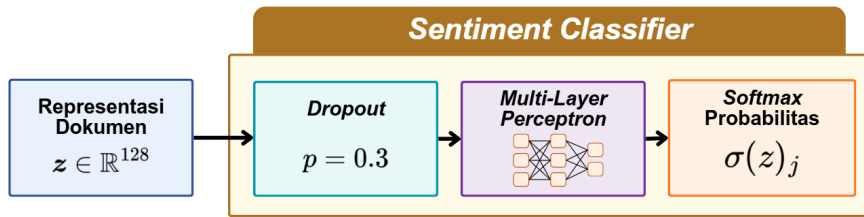
Pada tahap pra-pelatihan, dokumen diproses melalui xLSTM untuk menghasilkan *hidden state* $\mathbf{h}_t \in \mathbb{R}^{256}$ pada setiap posisi token t . Setiap token direpresentasikan menggunakan *word embedding* GloVe berdimensi 300 dan diproyeksikan ke dimensi 256 melalui lapisan linier yang diikuti *Layer Normalization* pada persamaan (2.11), kemudian diproses oleh xLSTM. *Hidden state* $\mathbf{h}_t$ selanjutnya diproses oleh *encoder* berupa MLP dengan *layer* $(\mathbb{R}^{256} \rightarrow \mathbb{R}^{512})$ ber-aktivasi ReLU, diikuti *layer* $(\mathbb{R}^{512} \rightarrow \mathbb{R}^{128})$ untuk menghasilkan representasi terkompresi, yang kemudian direkonstruksi oleh *decoder* berupa MLP dengan *layer* $(\mathbb{R}^{128} \rightarrow \mathbb{R}^{512})$ ber-aktivasi ReLU, diikuti *layer* $(\mathbb{R}^{512} \rightarrow \mathbb{R}^{256})$. Hasil rekonstruksi dibandingkan dengan representasi asli menggunakan fungsi *loss* $\mathcal{L}_{MSE}$ pada persamaan (2.19). Pada tahap ini, bobot xLSTM, *encoder*, dan *decoder* dioptimasi secara bersamaan melalui *backpropagation*. Setelah pra-pelatihan selesai, *decoder* tidak digunakan dan hanya bobot xLSTM serta *encoder* yang dipertahankan.

Pada tahap *joint training*, bobot xLSTM yang diperoleh dari pra-pelatihan dibekukan (*frozen*) sehingga tidak diperbarui selama proses pelatihan. Dengan demikian, xLSTM berfungsi sebagai *feature extractor* yang menghasilkan representasi token $\mathbf{h}_t$ secara tetap. Pembekuan bobot xLSTM dilakukan agar representasi token yang telah dipelajari pada tahap pra-pelatihan tidak terganggu oleh gradien dari tugas sentimen dan *clustering* pada tahap *joint training*. Setiap dokumen diproses menggunakan xLSTM untuk menghasilkan *hidden state* $\mathbf{h}_t \in \mathbb{R}^{256}$ pada setiap posisi token. Untuk memperoleh satu representasi per dokumen, $\mathbf{h}_t$ diagregasi menggunakan *masked mean pooling* pada persamaan (3.26).

$$\mathbf{z}_{pool} = \frac{\sum_{t=1}^L a_t \mathbf{h}_t}{\sum_{t=1}^L a_t}, \quad (3.26)$$

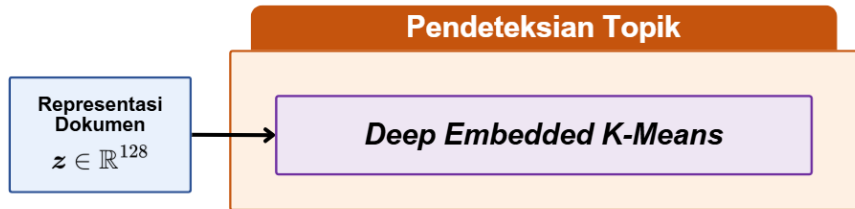
dengan $a_t \in \{0, 1\}$ merupakan nilai *attention mask* yang bernilai 1 untuk token valid dan 0 untuk token *padding*, serta L menyatakan panjang urutan token. Hasil *masked mean pooling* $\mathbf{z}_{pool} \in \mathbb{R}^{256}$ selanjutnya diproses oleh *encoder* yang sama dari tahap pra-pelatihan untuk menghasilkan representasi bersama $\mathbf{z} \in \mathbb{R}^{128}$ yang digunakan oleh jalur analisis sentimen dan jalur pendeteksian topik.

Pada jalur analisis sentimen, representasi bersama z dilewatkan ke *sentiment classifier* berupa MLP tiga *layer* yang berfungsi sebagai pengklasifikasi biner. *sentiment classifier* terdiri atas *layer* $\mathbb{R}^{128} \rightarrow \mathbb{R}^{128}$, $\mathbb{R}^{128} \rightarrow \mathbb{R}^{32}$, dan $\mathbb{R}^{32} \rightarrow \mathbb{R}^2$, dengan *Layer Normalization*, fungsi aktivasi GELU, dan *dropout* ($p = 0,3$) pada setiap *layer* kecuali *layer* terakhir yang menggunakan fungsi *softmax* untuk menghasilkan distribusi probabilitas kelas positif dan negatif. Alur lengkap jalur analisis sentimen ditunjukkan pada Gambar 3.9.



Gambar 3.9: Alur lengkap jalur analisis sentimen

Pada jalur pendeteksian topik, representasi bersama z digunakan sebagai *input* bagi komponen DEKM sebagaimana dijelaskan pada subbab 3.2 untuk melakukan pengelompokan dokumen berdasarkan topik. Alur lengkap jalur pendeteksian topik ditunjukkan pada Gambar 3.10.



Gambar 3.10: Alur lengkap jalur pendeteksian topik

Jalur analisis sentimen dan pendeteksian topik dilatih secara simultan dalam satu kerangka MTL. Masing-masing jalur menghasilkan nilai *loss* yang digabungkan ke dalam satu fungsi objektif bersama (*joint loss*). Fungsi *joint loss* terdiri dari *sentiment loss* $\mathcal{L}_{\text{BCE}}$ pada persamaan (2.17) dan *clustering loss* $\mathcal{L}_{\text{DEKM}}$ pada persamaan (3.25). Proses *joint training* dibagi menjadi dua fase, yaitu fase *warmup* dan fase pelatihan penuh. Pada fase *warmup*, optimasi hanya menggunakan *loss* sentimen agar *encoder* terlebih dahulu membentuk representasi bersama yang stabil sebelum komponen *clustering* diaktifkan. Fungsi objektif pada fase ini dirumuskan pada persamaan (3.27) sebagai

$$\mathcal{L}_{\text{warmup}} = \mathcal{L}_{\text{BCE}}, \quad (3.27)$$

Setelah fase *warmup* selesai, pelatihan dilanjutkan ke fase pelatihan penuh. Pada skema ini, bobot *loss* sentimen ditetapkan secara tetap bernilai 1 karena tugas klasifikasi sentimen bersifat *supervised* sehingga gradiennya relatif stabil, sedangkan *clustering loss* diberi bobot adaptif melalui parameter η yang diinisialisasi bernilai 1 pada awal pelatihan dan dioptimasi bersama parameter model lainnya menggunakan AdamW. Selama proses pelatihan, nilai η disesuaikan berdasarkan gradien yang diterima dari fungsi *loss* gabungan, sehingga keseimbangan kontribusi antara tugas sentimen dan pendeteksian topik tidak perlu ditentukan secara manual. Fungsi objektif pada fase pelatihan penuh dirumuskan pada persamaan (3.28) sebagai

$$\mathcal{L}_{\text{joint}} = \mathcal{L}_{\text{BCE}} + \eta \mathcal{L}_{\text{DEKM}}, \quad (3.28)$$

Pada proses optimasi, fungsi *loss* gabungan pada Persamaan (3.28) memungkinkan sinyal kesalahan dari tugas sentimen maupun topik mengalir secara bersamaan ke komponen model yang sama. Proses *backpropagation* dari $\mathcal{L}_{\text{joint}}$ memperbarui parameter *encoder* (MLP 2 layer), *sentiment classifier* (MLP 3 layer), serta bobot adaptif η , sedangkan parameter xLSTM tetap dibekukan sebagaimana dijelaskan sebelumnya.

Karena *encoder* menerima sinyal perbaikan dari kedua tugas secara simultan, representasi bersama z yang dihasilkannya secara bertahap menyesuaikan diri terhadap kebutuhan klasifikasi sentimen maupun pendeteksian topik selama proses pelatihan berlangsung. Mekanisme ini menjadikan model bersifat *joint* dan *end-to-end*. Kedua tugas tidak dilatih secara terpisah dalam tahap yang berbeda, melainkan dioptimasi bersama melalui satu fungsi objektif sehingga setiap pembaruan parameter memperhitungkan kebutuhan analisis sentimen dan pendeteksian topik secara bersamaan.

BAB 4

APLIKASI

Bab ini berisikan hasil simulasi dan analisis kinerja model JDST berbasis xLSTM yang diusulkan. Data yang digunakan untuk menghasilkan performa model menggunakan *dataset* IMDB dan Yelp. Kinerja model dianalisis dari dua aspek utama, yaitu analisis sentimen dan pendeteksian topik. Pada tugas analisis sentimen, performa model dievaluasi menggunakan metrik *accuracy*, *precision*, *recall*, dan *F1-score*. Sementara itu, pada pendeteksian topik, kualitas hasil *cluster* dianalisis menggunakan metrik skor *Topic Coherence-W2V* (TC-W2V) dan *Topic Diversity* (TD).

4.1 Spesifikasi Mesin dan Perangkat Lunak

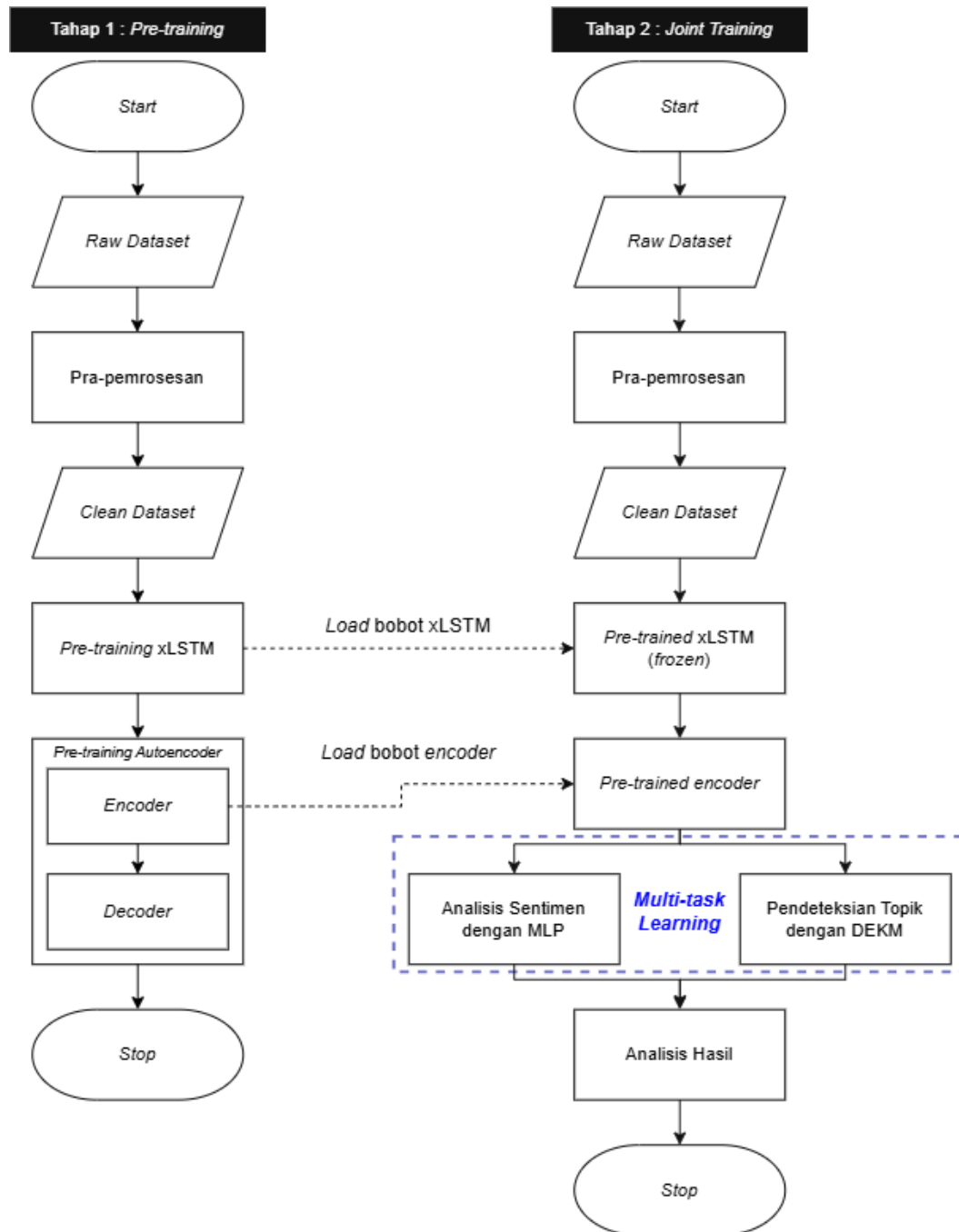
Spesifikasi mesin dan perangkat lunak yang digunakan dalam proses simulasi pada penelitian ini dapat dilihat pada Tabel 4.1.

Tabel 4.1: Spesifikasi mesin dan perangkat lunak

Mesin	
<i>Central Processing Unit</i> (CPU)	Intel Core Ultra 7 155H
Memori	32 GB
<i>Graphic Processing Unit</i> (GPU)	NVIDIA GeForce RTX 4050 Laptop GPU
Perangkat Lunak	
Perangkat lunak pemrograman	Google Colaboratory, Visual Studio Code
Bahasa pemrograman	Python 3.13.3
Pustaka yang digunakan	os, math, random, json, re, numpy, PyTorch, gensim, transformers, datasets, tqdm, xlstm

4.2 Tahapan Umum Simulasi

Tahapan umum simulasi penelitian ini digambarkan dalam diagram alur pada Gambar 4.1.



Gambar 4.1: Diagram alur penelitian.

Proses penelitian terdiri atas dua tahap utama. Pada tahap pertama (pra-pelatihan), data ulasan yang telah melalui pra-pemrosesan digunakan untuk melatih xLSTM bersama *encoder* dan *decoder* dalam kerangka *autoencoder* untuk mempelajari representasi token yang informatif melalui rekonstruksi *embedding*.

Pada tahap kedua (*joint training*), bobot xLSTM dan *encoder* yang diperoleh dari pra-pelatihan dimuat ke dalam model JDST, dengan bobot xLSTM dibekukan

sedangkan *encoder* tetap dapat diperbarui. Representasi bersama yang dihasilkan *encoder* selanjutnya digunakan secara bersamaan oleh dua jalur tugas utama, yaitu analisis sentimen melalui MLP dan pendeteksian topik melalui DEKM. Hasil dari kedua jalur tersebut kemudian dievaluasi menggunakan metrik evaluasi yang sesuai.

4.3 Data

Subbab ini membahas *dataset* yang digunakan pada penelitian beserta karakteristiknya, termasuk tahapan persiapan data sebelum diterapkan pada model JDST. Tahapan pengolahan data meliputi pengumpulan data ulasan, pra-pemrosesan teks, pembagian data latih dan data uji, sampai pembentukan representasi teks yang digunakan sebagai *input* model.

4.3.1 Pengumpulan Data

Penelitian ini menggunakan dua *dataset* ulasan berbahasa Inggris, yaitu IMDB Movie Reviews dan Yelp Review Dataset. Kedua *dataset* dipilih karena umum digunakan sebagai *benchmark* dalam penelitian analisis sentimen dan pendeteksian topik, serta dilengkapi dengan label sentimen biner.

Dataset IMDB Movie Reviews terdiri dari 50.000 ulasan film yang diunggah oleh Maas *et al.* (2011) dan tersedia secara publik melalui platform Hugging Face dalam format *Apache Arrow*. *Dataset* ini memuat label sentimen positif dan negatif yang relatif seimbang, sehingga sesuai digunakan untuk tugas klasifikasi biner.

Yelp Review Dataset terdiri dari 598.000 ulasan pengguna terhadap berbagai layanan dan bisnis, seperti restoran dan toko. *Dataset* ini tersedia secara publik melalui platform Hugging Face yang diunggah oleh Zhang *et al.* (2015) dalam format *Apache Arrow* dengan label sentimen positif dan negatif yang seimbang. Dalam penelitian ini, jumlah data Yelp yang digunakan disesuaikan agar proporsional dengan jumlah data IMDB, yaitu sebanyak 50.000 ulasan.

Beberapa cuplikan dokumen ulasan dari masing-masing *dataset* disajikan pada Tabel 4.2 dan Tabel 4.3. Kedua *dataset* menggunakan label sentimen biner, dengan nilai 1 menunjukkan sentimen positif dan nilai 0 menunjukkan sentimen negatif. Contoh data ulasan IMDB ditampilkan pada Tabel 4.2 berikut.

Tabel 4.2: Contoh data ulasan IMDB (Maas *et al.*, 2011).

No	Text	Label
1	 This is without a doubt the funniest comedy of the year. Everybody is brilliant ... laugh your face off. 9 - 10.	1
2	It's highly stylized, but this movie shows that real people appear on these shows ... Yes, it's mostly comedy, but there are some sad moments.	1
⋮	⋮	⋮
50.000	The plot is about the death of little children. Hopper is the one who has to investigate the killings ... Just don't watch the movie.	0

Selanjutnya, contoh data ulasan Yelp disajikan pada Tabel 4.3 berikut:

Tabel 4.3: Contoh data ulasan Yelp (Zhang *et al.*, 2015).

No	Text	Label
1	http://www... Continuing to screw people over, and the threaten to beat them up like tough guys.	0
2	Horrible service... Food is gross! Pretty much sums it up. Decorations inside are nice to look at. But I will never go back after eating the food.	0
⋮	⋮	⋮
50.000	This is by far the best dentist I have ever been to. She is honest and never tries to sell you a bunch of stuff that you don't need. We are very grateful ... worth the drive from Florida.	1

4.3.2 Prapengolahan Data

Tahap prapengolahan data dilakukan untuk membersihkan teks agar dapat digunakan secara optimal dalam proses pembentukan representasi dengan xLSTM. Karena xLSTM merupakan model sekuensial yang dirancang untuk menangkap hubungan kontekstual antartoken secara berurutan, penelitian ini menerapkan prapengolahan yang minim. Tahapan seperti *stemming*, penghapusan *stop words*, dan normalisasi tanda baca tidak

diterapkan, karena kata-kata fungsional seperti *stop words* masih mengandung informasi yang relevan bagi model dalam membedakan sentimen maupun topik ulasan.

Pada tahap selanjutnya, teks akan diproses menggunakan tokenisasi berbasis *WordPiece* yang mampu menangani kata-kata di luar kosakata dengan memecahnya menjadi unit *subword*, sehingga tidak diperlukan normalisasi kata secara manual. Oleh karena itu, prapengolahan pada penelitian ini difokuskan pada pembersihan elemen-elemen yang tidak memiliki makna bahasa, tanpa menghilangkan kata-kata yang masih mengandung informasi kontekstual. Adapun tahapan prapengolahan yang dilakukan adalah sebagai berikut:

1. *Casefolding*, yaitu mengubah seluruh teks menjadi huruf kecil untuk menghindari perbedaan representasi kata akibat variasi kapitalisasi.
2. Penghapusan tag HTML seperti `<br />`, `<b>`, dan elemen HTML lain yang tidak memiliki kandungan bahasa.
3. Penghapusan URL, yaitu menghapus tautan atau alamat situs web dengan pola `http://...` atau `www....` karena tidak mengandung informasi sentimen maupun topik.
4. Penghapusan karakter non-ASCII, yaitu menghapus emoji, simbol *unicode*, dan karakter khusus yang tidak termasuk dalam karakter teks standar yang berpotensi mengganggu proses tokenisasi.
5. Normalisasi *whitespace*, yaitu menggabungkan spasi berlebih, karakter *tab*, dan baris baru menjadi satu spasi tunggal agar teks memiliki format yang konsisten.

Hasil dari tahap prapengolahan ini berupa teks yang telah dibersihkan dan distandarisasi, tetapi tetap mempertahankan sebagian besar struktur kalimat asli. Teks yang telah melalui tahap ini selanjutnya digunakan pada proses tokenisasi dan pembentukan representasi teks menggunakan xLSTM. Contoh perbandingan data teks sebelum dan sesudah melalui tahap prapengolahan dapat dilihat pada Tabel 4.4.

Tabel 4.4: Contoh hasil prapengolahan pada *dataset* IMDB dan Yelp

IMDB	
Sebelum Prapengolahan	Sesudah Prapengolahan

 This is without a doubt the funniest comedy of the year. Everybody is brilliant. The acting is superb. You can see that the actors enjoyed making this film. it's a shame to spoil the film with give aways, so rent it and laugh your face off.

 9 - 10.	this is without a doubt the funniest comedy of the year. everybody is brilliant. the acting is superb. you can see that the actors enjoyed making this film. its a shame to spoil the film with give aways, so rent it and laugh your face off. 9 - 10.
It's highly stylized, but this movie shows that real people appear on these shows and what seems like good fun and a chance to appear on television can have serious consequences.

 Yes, i's mostly comedy, but there are some sad moments.	it's highly stylized, but this movie shows that real people appear on these shows and what seems like good fun and a chance to appear on television can have serious consequences. yes, i's mostly comedy, but there are some sad moments.
Yelp	
Sebelum Prapengolahan	Sesudah Prapengolahan
http://www... Continuing to screw people over, and the threaten to beat them up like tough guys.	continuing to screw people over, and the threaten to beat them up like tough guys.
Horrible service... Food is gross! Pretty much sums it up. Decorations inside are nice to look at. But I will never go back after eating the food.	horrible service... food is gross! pretty much sums it up. decorations inside are nice to look at. but i will never go back after eating the food.

Hasil prapengolahan pada Tabel 4.4 menunjukkan bahwa perubahan utama yang dilakukan adalah pembersihan elemen non-linguistik dan penyeragaman format teks. Dengan demikian, informasi kontekstual dalam ulasan tetap dipertahankan sehingga dapat dimanfaatkan lebih lanjut pada tahap tokenisasi dan pembentukan *embedding*.

4.3.3 Representasi Teks

Setelah melalui tahap prapengolahan data, teks selanjutnya diubah menjadi representasi numerik melalui proses tokenisasi dan pembentukan *embedding*. Tahap ini bertujuan untuk menghasilkan representasi yang dapat diproses oleh model xLSTM. Kualitas representasi teks yang dihasilkan akan memengaruhi kemampuan model dalam menangkap pola sentimen dan topik pada data ulasan.

Tokenisasi dilakukan menggunakan BERT *Tokenizer* dengan skema *WordPiece* yang mampu memecah kata menjadi unit *subword*, sehingga dapat menangani kata di luar kosakata dengan lebih baik. Setiap teks yang telah diprapengolahan dikonversi menjadi urutan token dengan panjang maksimum 128 token. Apabila panjang teks melebihi batas tersebut, maka dilakukan *truncation* atau pemotongan, sedangkan teks yang lebih pendek akan ditambahkan *padding* agar memiliki panjang yang seragam. Contoh hasil tokenisasi ditunjukkan pada Tabel 4.5.

Tabel 4.5: Contoh proses tokenisasi pada *dataset* IMDB dan Yelp

Teks (Hasil Prapengolahan)	Token (<i>WordPiece</i>)	Token ID
IMDB		
this is without a doubt the funniest comedy ... laugh your face off. 9 - 10.	[CLS], this, is, without, a, doubt, the, fun, ##nies, ##t, comedy, ..., 9, -, 10, ., [SEP]	[101, 2023, ..., 102]
it's highly stylized, but this movie shows ... yes, i's mostly comedy, but there are some sad moments.	[CLS], it, ', s, highly, stylized, but, this, movie, ..., sad, moments, ., [SEP]	[101, 2009, ..., 102]
Yelp		
continuing to screw people over, and the threaten to beat them up like tough guys.	[CLS], nc, ##ont, ##in, ##uing, to, screw, people, ..., tough, guys, ., [SEP]	[101, 13316, ..., 102]

Teks (Hasil Prapengolahan)	Token (<i>WordPiece</i>)	Token ID
horrible service... food is gross! pretty much sums it up. decorations inside are nice to look at. but i will never go back after eating the food.	[CLS], horrible, service, ., ., ., ., food, is, gross, ..., eating, the, food, ., [SEP]	[101, 9202, ..., 102]

Hasil tokenisasi selanjutnya dipetakan menjadi representasi vektor melalui proses *word embedding*. Pada penelitian ini, *embedding* yang digunakan adalah *Global Vector for Word Representation* (GloVe) berdimensi 300. Setiap token dalam kosakata dipetakan ke dalam vektor *embedding* GloVe yang bersesuaian. Untuk token yang tidak terdapat dalam kamus GloVe, *embedding* diinisialisasi secara acak menggunakan distribusi normal agar tetap dapat dipelajari selama proses pelatihan. Contoh ilustrasi hasil pemetaan token ke *embedding* ditunjukkan pada Tabel 4.6.

Tabel 4.6: Contoh proses *embedding* pada *dataset* IMDB dan Yelp

Token	<i>Embedding</i> (5 dari 300 dimensi)
IMDB	
[CLS]	[−0.0097, +0.0412, −0.0059, −0.0039, +0.0064, ...]
it	[−0.3066, +0.1682, +0.9851, −0.3361, −0.2416, ...]
,	[−0.3456, −0.2499, +0.5868, −0.8912, −1.0954, ...]
⋮	⋮
[SEP]	[−0.0343, +0.0056, +0.0141, −0.0173, +0.0262, ...]
Yelp	
[CLS]	[−0.0097, +0.0412, −0.0059, −0.0039, +0.0064, ...]
horrible	[+0.1894, +0.0167, +0.3780, −0.5666, −1.1490, ...]
service	[−0.4224, −0.1331, −0.4142, −0.2368, +0.1904, ...]
⋮	⋮
[SEP]	[−0.0343, +0.0056, +0.0141, −0.0173, +0.0262, ...]

4.3.4 Pembagian Data

Setelah melalui tahap prapengolahan, data ulasan dari *dataset* IMDB dan Yelp dibagi ke dalam dua tahap pembagian sesuai dengan kebutuhan pelatihan model. Pada tahap pra-pelatihan, seluruh data latih dari kedua *dataset* digunakan tanpa label sentimen dengan proporsi 90% sebagai data latih pra-pelatihan dan 10% sebagai data validasi, untuk meninjau konvergensi rekonstruksi *embedding* selama pelatihan berlangsung.

Pada tahap *joint training*, data dibagi dengan proporsi 80% untuk data latih dan 20% untuk data uji. Data latih digunakan sebagai *input* utama dalam proses pelatihan model pada tugas klasifikasi sentimen dan pendeteksian topik. Selanjutnya, data uji digunakan untuk mengevaluasi performa model.

4.4 Implementasi *Joint Document Level Sentiment-Topic Model*

Implementasi model dilakukan melalui tahap pra-pelatihan untuk memperoleh representasi awal yang informatif dan tahap *joint training* untuk memperoleh sentimen dan topik. Seluruh implementasi dilakukan menggunakan bahasa pemrograman *Python* dengan *library* yang tercantum pada Tabel 4.1.

Tahap pra-pelatihan dilakukan menggunakan *dataset* IMDB dan Yelp untuk membentuk representasi awal, baik pada seluruh konfigurasi xLSTM maupun LSTM sebagai model pembanding, melalui rekonstruksi *embedding* GloVe dengan fungsi *loss* $\mathcal{L}_{\text{MSE}}$ pada persamaan (2.19) dan optimisasi AdamW pada persamaan (2.30). Seluruh model dilatih melalui proses pra-pelatihan yang identik. *Hyperparameter* yang digunakan pada tahap pra-pelatihan ditunjukkan pada Tabel 4.7.

Tabel 4.7: *Hyperparameter* pra-pelatihan

<i>Hyperparameter</i>	Nilai
Dimensi <i>embedding</i> GloVe	300
Maksimum kosakata	30.000
Panjang maksimum token	256
Dimensi <i>hidden state</i> xLSTM / LSTM	256
Dimensi <i>encoder</i> (MLP 2 layer)	256 $\rightarrow$ 512 $\rightarrow$ 128
Dimensi <i>decoder</i> (MLP 2 layer)	128 $\rightarrow$ 512 $\rightarrow$ 256
<i>Batch size</i>	64
<i>Learning rate</i>	3×10^{-4}
<i>Weight decay</i>	1×10^{-3}
Jumlah <i>epoch</i> maksimum	15

Pada tahap *joint training*, bobot xLSTM yang diperoleh dari pra-pelatihan dibekukan,

sedangkan *encoder* tetap dapat diperbarui bersama komponen lainnya. Representasi bersama $z \in \mathbb{R}^{128}$ yang dihasilkan *encoder* dimanfaatkan secara bersamaan oleh kedua jalur tugas pada model JDST-xLSTM dan JDST-LSTM. Pelatihan kedua model menggunakan fungsi *joint loss* yang sama pada persamaan (3.28) dengan optimisasi AdamW pada persamaan (2.30) dan *shceduling learning rate* menggunakan *Cosine Annealing*.

Konfigurasi pelatihan dibuat identik untuk seluruh kombinasi model yang diuji, yaitu empat konfigurasi xLSTM pada JDST-xLSTM ([1:0], [0:1], [1:1] ($m \rightarrow s$), dan [1:1] ($s \rightarrow m$)) serta satu konfigurasi JDST-LSTM satu *layer* dan dua *layer*. Pada pendeteksian topik, ditetapkan tiga variasi jumlah topik, yaitu $K = 30, 50$, dan 80 , mengacu pada penelitian Gui *et al.* (2022). Selain itu, variasi jumlah topik dari kecil hingga besar bertujuan untuk mengamati bagaimana performa model berubah seiring peningkatan granularitas topik, yaitu dari topik yang bersifat umum pada $K = 30$ hingga topik yang lebih spesifik pada $K = 80$. Setiap kombinasi konfigurasi model dan jumlah topik dijalankan sebanyak 3 *run* dengan *seed* berbeda, yaitu 42, 123, dan 456, untuk mengurangi pengaruh inisialisasi acak terhadap hasil. *Hyperparameter* utama yang digunakan pada tahap *joint training* ditunjukkan pada Tabel 4.8.

Tabel 4.8: *Hyperparameter joint training* model JDST

<i>Hyperparameter</i>	<i>Nilai</i>
<i>Batch size</i>	64
<i>Learning rate</i>	3×10^{-4}
<i>Weight decay</i>	1×10^{-3}
Jumlah <i>epoch</i> maksimum	60
Jumlah <i>warmup epoch</i>	15
Jumlah topik (K)	30, 50, 80
Jumlah <i>run</i>	3
<i>Seed</i>	42, 123, 456
Bobot <i>loss</i> sentimen	1 (<i>fixed</i>)
Bobot <i>loss clustering</i> (η)	<i>learnable</i>
<i>Dropout</i>	0,3
<i>Early stopping patience</i>	10 <i>epoch</i>
Pembaruan <i>centroid</i>	setiap 5 <i>epoch</i>

4.5 Analisis Kinerja Model *Joint Document Level Sentiment-Topic Model*

Bagian ini membahas kinerja model JDST dalam dua tugas utama, yaitu klasifikasi sentimen dan pendeteksian topik. Sebagai evaluasi pendahuluan, kualitas representasi yang dihasilkan pada tahap pra-pelatihan juga dianalisis untuk menilai kemampuan

encoder dalam membentuk representasi laten yang informatif.

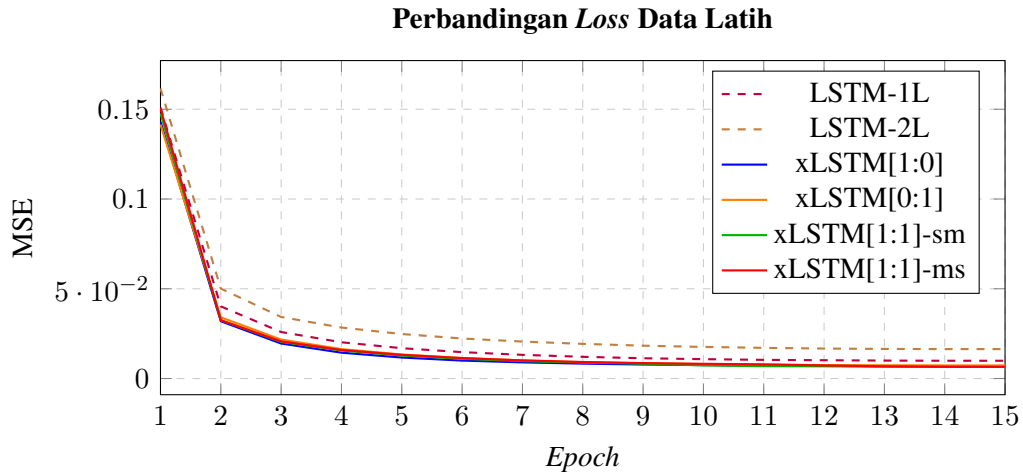
4.5.1 Analisis Kinerja Hasil Pra-pelatihan

Kinerja pra-pelatihan dievaluasi menggunakan nilai *loss* pada data latih dan data validasi. Pembagian ini dilakukan untuk meninjau apakah xLSTM dan *encoder* mampu mempelajari representasi yang umum dan tidak hanya menghafalkan pola pada data latih. Hasil perbandingan *loss* pra-pelatihan ditunjukkan pada Tabel 4.9.

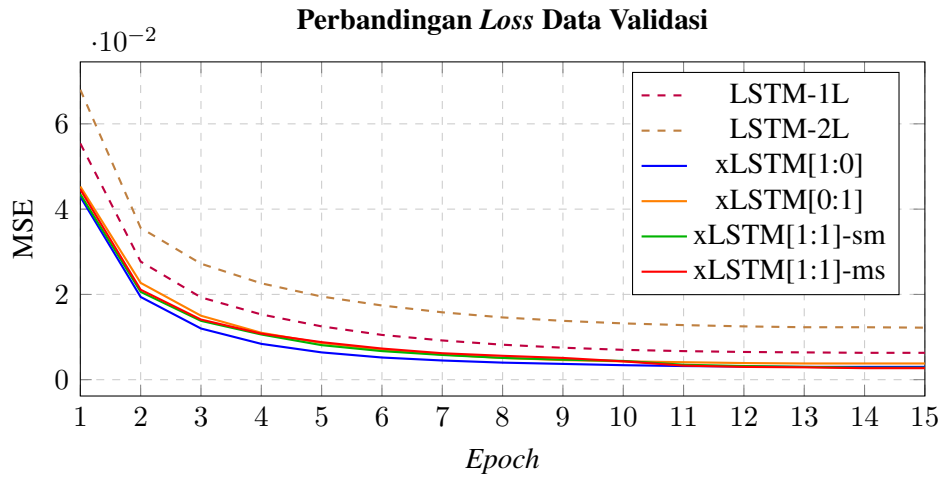
Tabel 4.9: Perbandingan *loss* pelatihan dan *loss* validasi pra-pelatihan

<i>Epoch</i>	LSTM-1L		LSTM-2L		xLSTM[1:0]		xLSTM[0:1]		xLSTM[1:1]-sm		xLSTM[1:1]-ms	
	latih	val	latih	val	latih	val	latih	val	latih	val	latih	val
1	0,1509	0,0554	0,1616	0,0680	0,1431	0,0429	0,1415	0,0453	0,1477	0,0435	0,1506	0,0446
2	0,0402	0,0277	0,0501	0,0356	0,0319	0,0194	0,0341	0,0227	0,0325	0,0206	0,0325	0,0211
3	0,0259	0,0193	0,0343	0,0272	0,0195	0,0120	0,0217	0,0150	0,0204	0,0138	0,0207	0,0141
4	0,0202	0,0153	0,0284	0,0226	0,0144	0,0084	0,0164	0,0110	0,0156	0,0106	0,0159	0,0108
5	0,0169	0,0125	0,0248	0,0195	0,0117	0,0064	0,0134	0,0086	0,0128	0,0081	0,0132	0,0088
6	0,0147	0,0105	0,0223	0,0174	0,0100	0,0052	0,0114	0,0070	0,0110	0,0067	0,0114	0,0073
7	0,0132	0,0092	0,0206	0,0158	0,0090	0,0045	0,0101	0,0059	0,0097	0,0058	0,0101	0,0062
8	0,0121	0,0082	0,0193	0,0146	0,0083	0,0040	0,0091	0,0051	0,0089	0,0051	0,0092	0,0056
9	0,0113	0,0075	0,0183	0,0138	0,0078	0,0037	0,0085	0,0046	0,0083	0,0047	0,0086	0,0051
10	0,0108	0,0070	0,0176	0,0132	0,0075	0,0034	0,0080	0,0043	0,0073	0,0044	0,0082	0,0043
11	0,0104	0,0067	0,0171	0,0128	0,0072	0,0032	0,0077	0,0041	0,0069	0,0035	0,0079	0,0033
12	0,0102	0,0065	0,0167	0,0125	0,0071	0,0031	0,0075	0,0039	0,0068	0,0032	0,0073	0,0030
13	0,0100	0,0064	0,0165	0,0123	0,0069	0,0030	0,0074	0,0038	0,0066	0,0030	0,0068	0,0029
14	0,0099	0,0063	0,0164	0,0123	0,0069	0,0030	0,0073	0,0038	0,0066	0,0028	0,0065	0,0027
15	0,0099	0,0063	0,0164	0,0122	0,0069	0,0030	0,0073	0,0038	0,0065	0,0028	0,0065	0,0027

Berdasarkan Tabel 4.9, seluruh konfigurasi xLSTM dengan LSTM menunjukkan tren penurunan *loss* pelatihan dan *loss* validasi yang konsisten seiring bertambahnya *epoch*. Model xLSTM[1:1]-ms memperoleh nilai *loss* validasi akhir terendah sebesar 0,0027, diikuti oleh xLSTM[1:1]-sm sebesar 0,0028, xLSTM[1:0] sebesar 0,0030, dan xLSTM[0:1] sebesar 0,0038. Sementara itu, LSTM-1L dan LSTM-2L masing-masing mencatat *loss* validasi akhir sebesar 0,0063 dan 0,0122, dengan LSTM-2L merupakan model yang memiliki nilai *loss* validasi akhir tertinggi di antara seluruh *encoder*. Tren penurunan *loss* tersebut juga dapat diamati secara visual pada Gambar 4.2 dan Gambar 4.3.



Gambar 4.2: Perbandingan *loss* data latih pra-pelatihan seluruh *encoder*.



Gambar 4.3: Perbandingan *loss* data validasi pra-pelatihan seluruh *encoder*.

Semua konfigurasi xLSTM dan LSTM menunjukkan konvergensi yang stabil tanpa indikasi *overfitting*, yang ditunjukkan oleh penurunan *loss* data latih dan *loss* data validasi secara konsisten serta jarak antarkeduanya yang relatif kecil pada setiap *epoch*. Secara keseluruhan, seluruh varian xLSTM mencapai nilai *loss* validasi akhir yang lebih rendah dibandingkan kedua varian LSTM. Model xLSTM[1:1]-ms memperoleh *loss* validasi akhir terendah sebesar 0,0027, diikuti oleh xLSTM[1:1]-sm sebesar 0,0028, sedangkan LSTM-2L mencatat nilai *loss* validasi akhir tertinggi sebesar 0,0122.

4.5.2 Analisis Kinerja Hasil *Joint Training*

Bagian ini membahas kinerja model JDST-xLSTM dan JDST-LSTM sebagai pembanding pada tahap *joint training*. Evaluasi dilakukan pada kinerja analisis sentimen

dan kualitas pendeteksian topik, menggunakan metrik yang telah dijelaskan pada Subbab 2.11.

4.5.2.1 Analisis Kinerja Hasil Analisis Sentimen

Tabel 4.10: Hasil evaluasi analisis sentimen model pada *dataset* IMDB

<i>Model</i>	<i>K</i>	<i>Accuracy</i>	<i>F1-Macro</i>	<i>Precision</i>	<i>Recall</i>
JDST-LSTM-1L	30	$0,8637 \pm 0,0104$	$0,8637 \pm 0,0104$	$0,8642 \pm 0,0104$	$0,8637 \pm 0,0104$
	50	$0,8599 \pm 0,0057$	$0,8597 \pm 0,0056$	$0,8612 \pm 0,0064$	$0,8599 \pm 0,0057$
	80	$0,8696 \pm 0,0029$	$0,8696 \pm 0,0029$	$0,8701 \pm 0,0032$	$0,8696 \pm 0,0029$
JDST-LSTM-2L	30	$0,8680 \pm 0,0031$	$0,8678 \pm 0,0033$	$0,8710 \pm 0,0009$	$0,8680 \pm 0,0031$
	50	$0,8616 \pm 0,0027$	$0,8610 \pm 0,0029$	$0,8671 \pm 0,0015$	$0,8616 \pm 0,0027$
	80	$0,8561 \pm 0,0063$	$0,8557 \pm 0,0061$	$0,8603 \pm 0,0082$	$0,8561 \pm 0,0063$
JDST-xLSTM[0:1]	30	$0,8760 \pm 0,0007$	$0,8759 \pm 0,0008$	$0,8773 \pm 0,0004$	$0,8760 \pm 0,0007$
	50	$0,8733 \pm 0,0028$	$0,8732 \pm 0,0029$	$0,8740 \pm 0,0023$	$0,8733 \pm 0,0028$
	80	$0,8754 \pm 0,0011$	$0,8754 \pm 0,0011$	$0,8755 \pm 0,0012$	$0,8754 \pm 0,0011$
JDST-xLSTM[1:0]	30	$0,8704 \pm 0,0025$	$0,8703 \pm 0,0025$	$0,8710 \pm 0,0029$	$0,8704 \pm 0,0025$
	50	$0,8772 \pm 0,0007$	$0,8772 \pm 0,0007$	$0,8776 \pm 0,0008$	$0,8772 \pm 0,0007$
	80	$0,8783 \pm 0,0027$	$0,8783 \pm 0,0027$	$0,8786 \pm 0,0026$	$0,8783 \pm 0,0027$
JDST-xLSTM[1:1] – sm	30	$0,8761 \pm 0,0060$	$0,8760 \pm 0,0062$	$0,8773 \pm 0,0046$	$0,8761 \pm 0,0060$
	50	$0,8795 \pm 0,0049$	$0,8794 \pm 0,0049$	$0,8798 \pm 0,0047$	$0,8795 \pm 0,0049$
	80	$0,8830 \pm 0,0018$	$0,8830 \pm 0,0018$	$0,8833 \pm 0,0016$	$0,8830 \pm 0,0018$
JDST-xLSTM[1:1] – ms	30	$0,8827 \pm 0,0016$	$0,8827 \pm 0,0016$	$0,8833 \pm 0,0015$	$0,8827 \pm 0,0016$
	50	$0,8855 \pm 0,0026$	$0,8855 \pm 0,0027$	$0,8862 \pm 0,0023$	$0,8855 \pm 0,0026$
	80	$0,8875 \pm 0,0013$	$0,8875 \pm 0,0013$	$0,8879 \pm 0,0009$	$0,8875 \pm 0,0013$

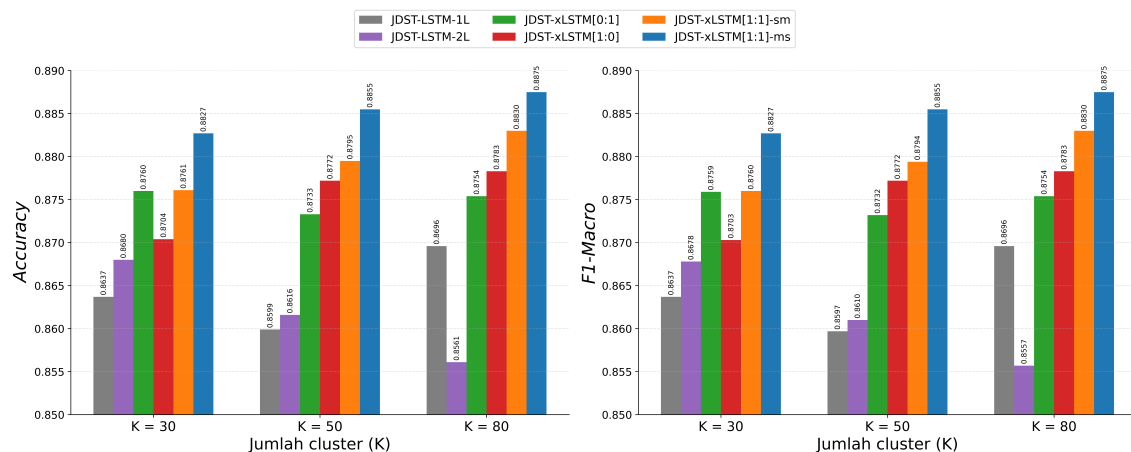
Keterangan: $K = 30$, $K = 50$, $K = 80$, (warna menandai model dengan kinerja terbaik pada nilai K dan metrik tersebut)

Berdasarkan hasil evaluasi sentimen pada *dataset* IMDB, JDST-xLSTM[1:1]-ms memperoleh performa terbaik pada seluruh skenario pengujian. Model ini menghasilkan nilai *accuracy*, *F1-macro*, *precision*, dan *recall* tertinggi pada seluruh nilai K yang diuji. Nilai *accuracy* meningkat dari $0,8827 \pm 0,0016$ pada $K = 30$ menjadi $0,8855 \pm 0,0026$ pada $K = 50$, dan mencapai nilai tertinggi sebesar $0,8875 \pm 0,0013$ pada $K = 80$. Peningkatan yang konsisten seiring bertambahnya jumlah topik memperkuat klasifikasi sentimen.

Di antara konfigurasi JDST-xLSTM lainnya, JDST-xLSTM[1:1]-sm menunjukkan performa yang kompetitif dan berada di posisi kedua. Model ini memperoleh *accuracy* sebesar $0,8761 \pm 0,0060$ pada $K = 30$ dan meningkat secara konsisten menjadi

$0,8830 \pm 0,0018$ pada $K = 80$. Kedua konfigurasi [1:1] yang mengombinasikan blok mLSTM dan sLSTM secara konsisten mengungguli konfigurasi blok tunggal, yang mengindikasikan bahwa kombinasi kedua jenis blok menghasilkan representasi yang lebih kaya dibandingkan penggunaan satu jenis blok saja. Sementara itu, JDST-xLSTM[1:0] memperoleh *accuracy* sebesar $0,8704 \pm 0,0025$ pada $K = 30$ dan meningkat menjadi $0,8783 \pm 0,0027$ pada $K = 80$, sedangkan JDST-xLSTM[0:1] menghasilkan performa yang relatif stabil dengan *accuracy* berkisar antara $0,8733 \pm 0,0028$ hingga $0,8760 \pm 0,0007$.

Sementara itu, kedua konfigurasi JDST-LSTM sebagai *baseline* secara konsisten menghasilkan performa yang lebih rendah dibandingkan seluruh konfigurasi JDST-xLSTM. JDST-LSTM-2L memperoleh *accuracy* tertinggi sebesar $0,8680 \pm 0,0031$ pada $K = 30$, namun mengalami penurunan menjadi $0,8561 \pm 0,0063$ pada $K = 80$. JDST-LSTM-1L menunjukkan pola serupa dengan *accuracy* berkisar antara $0,8599 \pm 0,0057$ hingga $0,8696 \pm 0,0029$. Secara keseluruhan, seluruh konfigurasi xLSTM mengungguli kedua konfigurasi LSTM pada tugas analisis sentimen di *dataset* IMDB, yang menunjukkan bahwa mekanisme *exponential gating* dan *matrix memory* pada xLSTM menghasilkan representasi token yang lebih informatif untuk klasifikasi sentimen dibandingkan arsitektur LSTM konvensional. Perbandingan nilai *accuracy* antar model untuk setiap nilai K dapat dilihat pada Gambar 4.4.



Gambar 4.4: Perbandingan *Accuracy* dan *F1-Macro* analisis sentimen model pada *dataset* IMDB

Tabel 4.11: Hasil evaluasi analisis sentimen model pada *dataset* Yelp

Model	K	Accuracy	F1-Macro	Precision	Recall
JDST-LSTM-1L	30	$0,8967 \pm 0,0019$	$0,8967 \pm 0,0019$	$0,8977 \pm 0,0020$	$0,8967 \pm 0,0019$

<i>Model</i>	<i>K</i>	<i>Accuracy</i>	<i>F1-Macro</i>	<i>Precision</i>	<i>Recall</i>
JDST-LSTM-2L	50	$0,9051 \pm 0,0095$	$0,9050 \pm 0,0096$	$0,9057 \pm 0,0089$	$0,9051 \pm 0,0095$
	80	$0,9031 \pm 0,0076$	$0,9031 \pm 0,0076$	$0,9035 \pm 0,0074$	$0,9031 \pm 0,0076$
	30	$0,9019 \pm 0,0080$	$0,9019 \pm 0,0081$	$0,9024 \pm 0,0076$	$0,9019 \pm 0,0080$
	50	$0,8957 \pm 0,0107$	$0,8957 \pm 0,0107$	$0,8960 \pm 0,0109$	$0,8957 \pm 0,0107$
	80	$0,9114 \pm 0,0016$	$0,9114 \pm 0,0016$	$0,9118 \pm 0,0013$	$0,9114 \pm 0,0016$
JDST-xLSTM[0:1]	30	$0,9105 \pm 0,0029$	$0,9105 \pm 0,0029$	$0,9108 \pm 0,0027$	$0,9105 \pm 0,0029$
	50	$0,9065 \pm 0,0079$	$0,9064 \pm 0,0079$	$0,9069 \pm 0,0081$	$0,9065 \pm 0,0079$
	80	$0,9096 \pm 0,0023$	$0,9096 \pm 0,0023$	$0,9101 \pm 0,0022$	$0,9096 \pm 0,0023$
JDST-xLSTM[1:0]	30	$0,9106 \pm 0,0006$	$0,9105 \pm 0,0006$	$0,9110 \pm 0,0008$	$0,9106 \pm 0,0006$
	50	$0,9102 \pm 0,0009$	$0,9102 \pm 0,0009$	$0,9106 \pm 0,0007$	$0,9102 \pm 0,0009$
	80	$0,9127 \pm 0,0009$	$0,9126 \pm 0,0009$	$0,9130 \pm 0,0010$	$0,9127 \pm 0,0009$
JDST-xLSTM[1:1]-sm	30	$0,9191 \pm 0,0013$	$0,9191 \pm 0,0013$	$0,9192 \pm 0,0014$	$0,9191 \pm 0,0013$
	50	$0,9117 \pm 0,0099$	$0,9117 \pm 0,0099$	$0,9122 \pm 0,0096$	$0,9117 \pm 0,0099$
	80	$0,9065 \pm 0,0124$	$0,9064 \pm 0,0124$	$0,9071 \pm 0,0122$	$0,9065 \pm 0,0124$
JDST-xLSTM[1:1]-ms	30	$0,9191 \pm 0,0004$	$0,9191 \pm 0,0004$	$0,9192 \pm 0,0004$	$0,9191 \pm 0,0004$
	50	$0,9201 \pm 0,0010$	$0,9201 \pm 0,0010$	$0,9202 \pm 0,0010$	$0,9201 \pm 0,0010$
	80	$0,9193 \pm 0,0017$	$0,9193 \pm 0,0017$	$0,9195 \pm 0,0016$	$0,9193 \pm 0,0017$

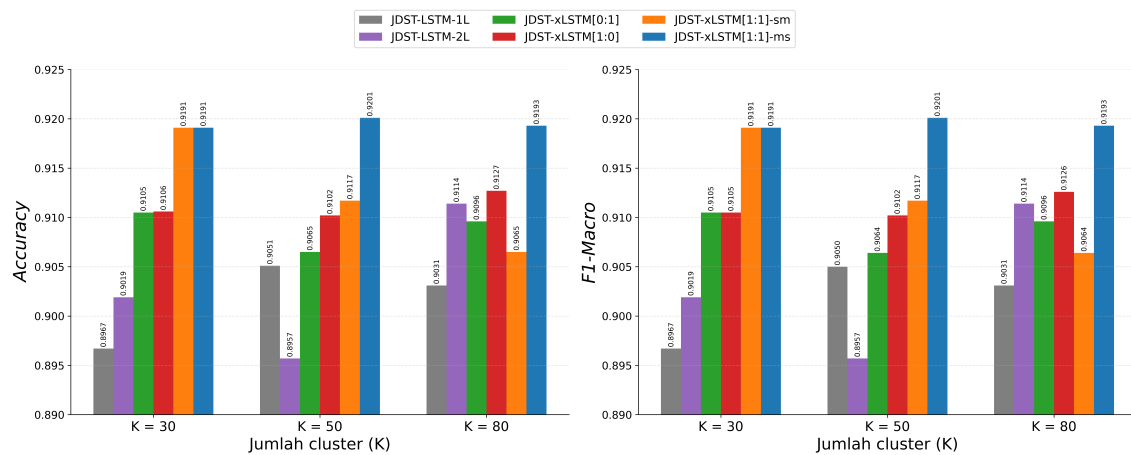
Keterangan: $K = 30$, $K = 50$, $K = 80$, (warna menandai model dengan kinerja terbaik pada nilai K dan metrik tersebut)

Berdasarkan hasil evaluasi sentimen pada *dataset* Yelp, kinerja terbaik tidak didominasi oleh satu model tunggal, tetapi tersebar di antara beberapa konfigurasi xLSTM. Pada $K = 30$, JDST-xLSTM[1:1]-sm dan JDST-xLSTM[1:1]-ms memperoleh *accuracy* yang sama, yaitu masing-masing $0,9191 \pm 0,0013$ dan $0,9191 \pm 0,0004$, dengan JDST-xLSTM[1:1]-ms menunjukkan stabilitas yang lebih tinggi berdasarkan nilai standar deviasi yang lebih kecil. Pada $K = 50$, JDST-xLSTM[1:1]-ms mengungguli seluruh model dengan *accuracy* tertinggi sebesar $0,9201 \pm 0,0010$, yang juga merupakan nilai tertinggi di seluruh skenario pengujian pada *dataset* Yelp. Sementara itu, pada $K = 80$, JDST-xLSTM[1:0] memperoleh performa terbaik dengan *accuracy* sebesar $0,9127 \pm 0,0009$.

Di antara konfigurasi blok tunggal, JDST-xLSTM[1:0] dan JDST-xLSTM[0:1] menunjukkan performa yang stabil di seluruh nilai K . JDST-xLSTM[1:0] memperoleh *accuracy* berkisar antara $0,9102 \pm 0,0009$ hingga $0,9127 \pm 0,0009$, sedangkan JDST-xLSTM[0:1] berkisar antara $0,9065 \pm 0,0079$ hingga $0,9105 \pm 0,0029$. Stabilitas ini tercermin dari nilai standar deviasi yang relatif kecil pada kedua konfigurasi tersebut.

Sementara itu, kedua konfigurasi JDST-LSTM sebagai *baseline* secara umum

menghasilkan performa yang lebih rendah dibandingkan konfigurasi xLSTM. JDST-LSTM-2L memperoleh *accuracy* tertinggi sebesar $0,9114 \pm 0,0016$ pada $K = 80$, namun pada $K = 50$ justru mengalami penurunan menjadi $0,8957 \pm 0,0107$ dengan standar deviasi yang cukup besar. JDST-LSTM-1L menunjukkan pola yang serupa dengan *accuracy* berkisar antara $0,8967 \pm 0,0019$ hingga $0,9051 \pm 0,0095$. Secara keseluruhan, hasil pada *dataset* Yelp memperkuat temuan pada IMDB bahwa penggunaan xLSTM sebagai model representasi teks menghasilkan performa analisis sentimen yang lebih baik dan lebih stabil dibandingkan arsitektur LSTM konvensional. Perbandingan nilai *accuracy* antar model pada setiap nilai K ditunjukkan pada Gambar 4.5.



Gambar 4.5: Perbandingan *Accuracy* dan *F1-Macro* analisis sentimen model pada *dataset* Yelp

4.5.2.2 Analisis Kinerja Hasil Pendeteksian Topik

Tabel 4.12: Hasil evaluasi pendeteksian topik model pada *dataset* IMDB

<i>Model</i>	<i>K</i>	TC-W2V	<i>Topic Diversity</i>
JDST-LSTM-1L	30	$0,3128 \pm 0,1256$	$0,3228 \pm 0,2029$
	50	$0,2413 \pm 0,1307$	$0,2841 \pm 0,1542$
	80	$0,3121 \pm 0,0556$	$0,2174 \pm 0,1149$
JDST-LSTM-2L	30	$0,3885 \pm 0,0131$	$0,2467 \pm 0,0386$
	50	$0,3285 \pm 0,0590$	$0,3242 \pm 0,1328$
	80	$0,2742 \pm 0,1490$	$0,3645 \pm 0,2166$
JDST-xLSTM[0:1]	30	$0,3827 \pm 0,0207$	$0,2689 \pm 0,0707$
	50	$0,3541 \pm 0,0395$	$0,3100 \pm 0,1143$
	80	$0,3543 \pm 0,0235$	$0,2521 \pm 0,0737$
JDST-xLSTM[1:0]	30	$0,3488 \pm 0,0462$	$0,2792 \pm 0,1257$

<i>Model</i>	<i>K</i>	<i>TC-W2V</i>	<i>Topic Diversity</i>
JDST-xLSTM [1:1]-sm	50	$0,3410 \pm 0,0477$	$0,2566 \pm 0,1338$
	80	$0,3289 \pm 0,0163$	$0,1780 \pm 0,0106$
	30	$0,3356 \pm 0,0669$	$0,2599 \pm 0,0822$
	50	$0,3853 \pm 0,0025$	$0,1894 \pm 0,0149$
	80	$0,3669 \pm 0,0088$	$0,1772 \pm 0,0309$
	30	$0,3946 \pm 0,0177$	$0,3798 \pm 0,0240$
JDST-xLSTM [1:1]-ms	50	$0,3867 \pm 0,0035$	$0,3646 \pm 0,0100$
	80	$0,3723 \pm 0,0097$	$0,2890 \pm 0,0314$

Keterangan: $K = 30$, $K = 50$, $K = 80$, (warna menandai model dengan kinerja terbaik) pada nilai K dan metrik tersebut.

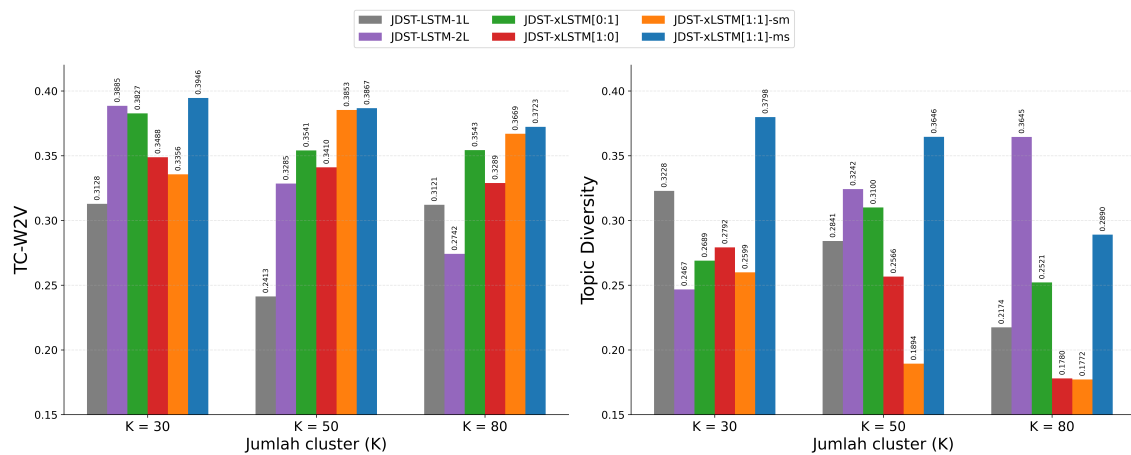
Berdasarkan hasil evaluasi pendeteksian topik pada *dataset* IMDB, JDST-xLSTM[1:1]-ms memperoleh performa terbaik pada metrik TC-W2V di seluruh nilai K yang diuji. Model ini menghasilkan nilai TC-W2V sebesar $0,3946 \pm 0,0177$ pada $K = 30$, $0,3867 \pm 0,0035$ pada $K = 50$, dan $0,3723 \pm 0,0097$ pada $K = 80$. Selain unggul pada koherensi topik, model ini juga memperoleh nilai *topic diversity* tertinggi pada $K = 30$ dan $K = 50$, masing-masing sebesar $0,3798 \pm 0,0240$ dan $0,3646 \pm 0,0100$. Kombinasi nilai TC-W2V dan TD yang tinggi secara bersamaan menunjukkan bahwa topik-topik yang dihasilkan tidak hanya koheren secara semantik, tetapi juga beragam satu sama lain.

Pada $K = 80$, performa *topic diversity* terbaik diperoleh oleh JDST-LSTM-2L dengan nilai $0,3645 \pm 0,2166$. Namun, nilai standar deviasi yang besar menunjukkan bahwa hasil tersebut kurang stabil antar *run*, sehingga keandalannya perlu dipertimbangkan. Sebagai perbandingan, JDST-xLSTM[1:1]-ms pada $K = 80$ memperoleh TD sebesar $0,2890 \pm 0,0314$ dengan standar deviasi yang jauh lebih kecil, mengindikasikan hasil yang lebih konsisten.

Di antara konfigurasi xLSTM lainnya, JDST-xLSTM[1:1]-sm menunjukkan performa TC-W2V yang cukup bersaing, khususnya pada $K = 50$ dengan nilai $0,3853 \pm 0,0025$ yang mendekati JDST-xLSTM[1:1]-ms. Namun, model ini cenderung menghasilkan *topic diversity* yang lebih rendah, khususnya pada $K = 50$ dan $K = 80$ dengan nilai masing-masing $0,1894 \pm 0,0149$ dan $0,1772 \pm 0,0309$, yang mengindikasikan adanya tumpang tindih kata antar topik. Sementara itu, JDST-xLSTM[0:1] menghasilkan TC-W2V yang relatif stabil di seluruh nilai K , berkisar antara $0,3541 \pm 0,0395$ hingga $0,3827 \pm 0,0207$.

Kedua konfigurasi JDST-LSTM sebagai *baseline* menghasilkan performa yang lebih rendah dan kurang stabil dibandingkan konfigurasi xLSTM. JDST-LSTM-1L

memperoleh nilai TC-W2V berkisar antara $0,2413 \pm 0,1307$ hingga $0,3128 \pm 0,1256$ dengan standar deviasi yang cukup besar, menunjukkan sensitivitas yang tinggi terhadap inisialisasi acak. JDST-LSTM-2L memperoleh TC-W2V yang lebih tinggi pada $K = 30$ sebesar $0,3885 \pm 0,0131$, namun mengalami penurunan pada $K = 80$ menjadi $0,2742 \pm 0,1490$. Secara keseluruhan, JDST-xLSTM[1:1]-ms menunjukkan keunggulan yang konsisten pada tugas pendeteksian topik di *dataset* IMDB dari segi koherensi maupun keragaman topik, serta menunjukkan stabilitas yang lebih baik dibandingkan *baseline* LSTM. Perbandingan nilai TC-W2V dan TD antar model untuk setiap nilai K ditunjukkan pada Gambar 4.6.



Gambar 4.6: Perbandingan TC-W2V dan TD pendeteksian topik model pada *dataset* IMDB

Tabel 4.13: Hasil evaluasi pendeteksian topik model pada *dataset* Yelp

<i>Model</i>	<i>K</i>	<i>TC-W2V</i>	<i>Topic Diversity</i>
JDST-LSTM-1L	30	$0,1656 \pm 0,0346$	$0,3321 \pm 0,0573$
	50	$0,2334 \pm 0,0757$	$0,3024 \pm 0,1585$
	80	$0,1489 \pm 0,0822$	$0,2865 \pm 0,1161$
JDST-LSTM-2L	30	$0,2104 \pm 0,1078$	$0,3625 \pm 0,2552$
	50	$0,2033 \pm 0,1182$	$0,3290 \pm 0,2248$
	80	$0,1535 \pm 0,0208$	$0,2709 \pm 0,0285$
JDST-xLSTM[0:1]	30	$0,2748 \pm 0,0260$	$0,3538 \pm 0,0699$
	50	$0,2389 \pm 0,1032$	$0,3449 \pm 0,1074$
	80	$0,2852 \pm 0,0153$	$0,2899 \pm 0,0464$
JDST-xLSTM[1:0]	30	$0,2971 \pm 0,0105$	$0,2501 \pm 0,0440$
	50	$0,2637 \pm 0,0206$	$0,2885 \pm 0,0291$
	80	$0,2664 \pm 0,0162$	$0,2471 \pm 0,0332$

<i>Model</i>	<i>K</i>	<i>TC-W2V</i>	<i>Topic Diversity</i>
JDST-xLSTM [1:1]-sm	30	$0,3518 \pm 0,0129$	$0,3933 \pm 0,0661$
	50	$0,2750 \pm 0,0357$	$0,3190 \pm 0,1556$
	80	$0,2207 \pm 0,0900$	$0,2600 \pm 0,1682$
JDST-xLSTM [1:1]-ms	30	$0,3153 \pm 0,0096$	$0,2533 \pm 0,0260$
	50	$0,2682 \pm 0,0315$	$0,2827 \pm 0,0600$
	80	$0,2579 \pm 0,0318$	$0,2917 \pm 0,0925$

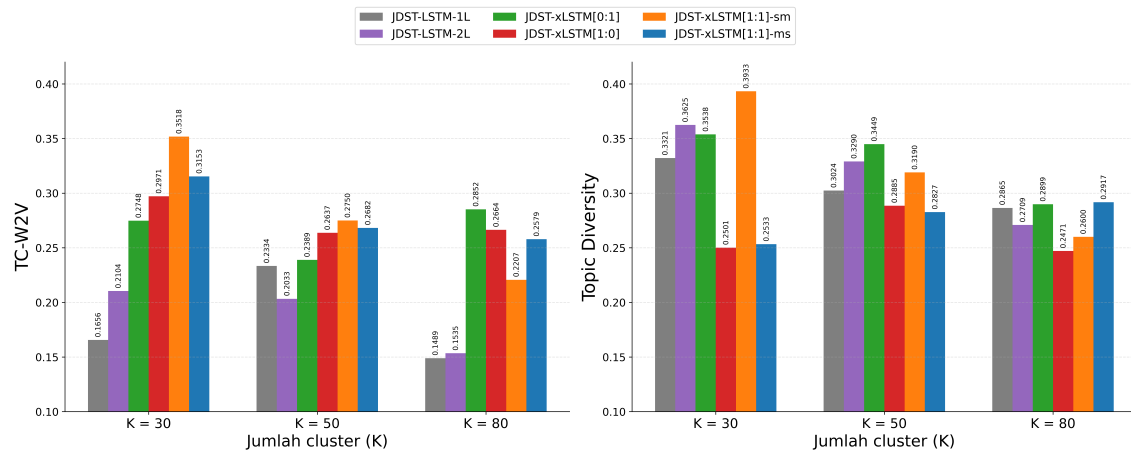
Keterangan: $K = 30$, $K = 50$, $K = 80$, (warna menandai model dengan kinerja terbaik) pada nilai K dan metrik tersebut.

Berdasarkan hasil evaluasi pendeteksian topik pada *dataset* Yelp, performa terbaik tersebar di antara beberapa konfigurasi xLSTM, dengan JDST-xLSTM[1:1]-sm memperoleh keunggulan pada mayoritas skenario. Model ini menghasilkan nilai TC-W2V tertinggi pada $K = 30$ dan $K = 50$, masing-masing sebesar $0,3518 \pm 0,0129$ dan $0,2750 \pm 0,0357$, sekaligus memperoleh *topic diversity* tertinggi pada $K = 30$ sebesar $0,3933 \pm 0,0661$. Kombinasi kedua nilai tersebut menunjukkan bahwa topik-topik yang dihasilkan pada $K = 30$ memiliki koherensi semantik yang baik sekaligus beragam satu sama lain.

Pada $K = 80$, performa terbaik beralih ke konfigurasi lain. JDST-xLSTM[0:1] memperoleh TC-W2V tertinggi sebesar $0,2852 \pm 0,0153$, sedangkan JDST-xLSTM[1:1]-ms memperoleh *topic diversity* tertinggi sebesar $0,2917 \pm 0,0925$. Pada $K = 50$, *topic diversity* tertinggi juga diperoleh oleh JDST-xLSTM[0:1] dengan nilai $0,3449 \pm 0,1074$. Hasil ini mengindikasikan bahwa konfigurasi blok tunggal mLSTM memiliki keunggulan dalam mempertahankan keragaman topik ketika jumlah topik semakin banyak.

Kedua konfigurasi JDST-LSTM sebagai *baseline* menghasilkan performa yang secara konsisten lebih rendah dibandingkan konfigurasi xLSTM. JDST-LSTM-2L memperoleh *topic diversity* yang relatif tinggi pada $K = 30$ sebesar $0,3625 \pm 0,2552$, namun nilai standar deviasi yang sangat besar menunjukkan ketidakstabilan hasil antar *run*. Pola serupa terlihat pada TC-W2V kedua konfigurasi LSTM, di mana nilai standar deviasi secara konsisten lebih besar dibandingkan konfigurasi xLSTM, khususnya JDST-LSTM-2L yang memperoleh TC-W2V sebesar $0,2033 \pm 0,1182$ pada $K = 50$ dan $0,1535 \pm 0,0208$ pada $K = 80$. Secara keseluruhan, konfigurasi xLSTM menunjukkan keunggulan yang jelas pada tugas pendeteksian topik di *dataset* Yelp, baik dari segi nilai metrik yang lebih tinggi maupun stabilitas hasil yang lebih baik dibandingkan *baseline* LSTM. Perbandingan nilai TC-W2V dan TD antar model untuk setiap nilai K

ditunjukkan pada Gambar 4.7.



Gambar 4.7: Perbandingan TC-W2V dan TD pendeteksian topik model pada *dataset* Yelp

4.6 Analisis Kontribusi Token pada Representasi Bersama untuk Analisis Sentimen dan Pendeteksian Topik

Selain evaluasi kuantitatif, dilakukan analisis kualitatif sebagai tambahan untuk melihat bagaimana representasi bersama z yang dihasilkan *encoder* dimanfaatkan oleh tugas analisis sentimen dan pendeteksian topik pada tingkat token. Analisis ini bertujuan untuk mengamati sejauh mana kedua tugas memanfaatkan informasi dari token-token yang sama dalam representasi bersama. Karena JDST-xLSTM[1:1] - ms memperoleh performa terbaik pada sebagian besar skenario pengujian, konfigurasi tersebut dipilih sebagai representasi model untuk analisis ini.

Kontribusi setiap token terhadap keputusan sentimen diukur menggunakan turunan *logit* kelas prediksi terhadap *embedding input* setiap token. Nilai turunan tersebut dihitung untuk seluruh token dalam dokumen, kemudian dinormalisasi ke rentang 0 hingga 1 menggunakan normalisasi *min-max*, sehingga token dengan turunan terbesar bernilai 1 dan token yang paling tidak berpengaruh bernilai mendekati 0. Nilai yang semakin tinggi menunjukkan bahwa perubahan kecil pada *embedding* token tersebut mengakibatkan perubahan yang lebih besar pada skor sentimen, sehingga token tersebut dianggap lebih berpengaruh terhadap keputusan sentimen model.

Kontribusi setiap token terhadap keputusan topik diukur menggunakan turunan jarak negatif antara representasi dokumen dan *centroid cluster*-nya terhadap *embedding input* setiap token. Jarak negatif digunakan agar token yang membantu dokumen berada lebih dekat ke *centroid cluster*-nya memperoleh nilai turunan yang lebih besar. Nilai turunan

tersebut kemudian dinormalisasi ke rentang 0 hingga 1 dengan cara yang sama, sehingga token bernilai mendekati 1 menunjukkan bahwa token tersebut paling berpengaruh dalam menentukan penugasan dokumen ke topik yang berkaitan, sedangkan token bernilai mendekati 0 memiliki pengaruh yang lebih kecil terhadap penugasan topik tersebut.

Skor kontribusi pada kedua kolom berasal dari kuantitas yang berbeda, yaitu *logit* sentimen dan jarak ke *centroid*, sehingga masing-masing dinormalisasi secara independen. Oleh karena itu, nilai pada kolom keputusan sentimen dan keputusan topik hanya dapat dibandingkan antar token dalam satu kolom yang sama, bukan lintas kolom.

Tabel 4.14: Contoh kontribusi token pada dokumen ulasan bersentimen negatif

Topik 21: "watching / actors / weird"	
Dokumen: "...everybody I talked to said that this movie would be good and really weird so I figured that I would rent it. Half way through the movie I was thinking to myself what the heck was going on... and even Robert Englund makes the movie worse... I was actually rooting for the bad guy to win that's how bad it was..."	
Keputusan Sentimen	Keputusan Topik
weird (1,00)	weird (1,00)
good (0,85)	bad (0,70)
everybody (0,71)	everybody (0,38)
bad (0,51)	worse (0,31)
movie (0,51)	movie (0,26)
worse (0,45)	movie (0,25)

Berdasarkan Tabel 4.14, token *weird* memperoleh bobot tertinggi pada keputusan sentimen maupun topik, masing-masing sebesar 1.00. Selain itu, beberapa token seperti *everybody*, *bad*, dan *movie* juga muncul pada kedua kolom dengan bobot yang relatif serupa. Temuan ini menunjukkan bahwa sebagian informasi yang digunakan model untuk menentukan sentimen dan topik berasal dari token yang sama. Di sisi lain, terdapat pula token yang lebih menonjol pada salah satu tugas, seperti *good* pada keputusan sentimen dan *worse* pada keputusan topik.

Tabel 4.15: Contoh kontribusi token pada dokumen ulasan bersentimen positif

Topik 2: "funny / watching / script"	
Dokumen: "...the duke is a very silly film—a dog becoming a duke! but it's a very fun movie. it has some of those corny pranks that many kids movies have, but (thankfully!) no bodily function jokes, as so many an..."	
Keputusan Sentimen	Keputusan Topik
silly (1,00)	silly (1,00)
fun (0,85)	fun (0,53)
duke (0,32)	duke (0,25)
dog (0,27)	dog (0,21)
duke (0,26)	prank (0,20)

Berdasarkan Tabel 4.15, token *silly* memperoleh bobot tertinggi pada keputusan sentimen maupun topik, masing-masing sebesar 1.00. Selain itu, token *fun*, *duke*, dan *dog* juga muncul pada kedua kolom dengan bobot yang relatif serupa. Meskipun demikian, terdapat beberapa perbedaan pada token yang dianggap penting oleh masing-masing tugas. Sebagai contoh, token *prank* muncul pada keputusan topik, tetapi tidak termasuk dalam token dengan kontribusi terbesar pada keputusan sentimen. Hal ini menunjukkan bahwa kedua tugas memanfaatkan informasi yang sama dalam banyak kasus, tetapi tetap dapat memberikan perhatian yang berbeda pada token tertentu.

Berdasarkan Tabel 4.14 dan Tabel 4.15, terlihat bahwa sebagian besar token dengan kontribusi tinggi muncul pada keputusan sentimen maupun topik. Hal ini menunjukkan bahwa kedua tugas memanfaatkan representasi laten yang sama yang dihasilkan oleh *shared encoder*. Kemunculan token-token yang sama pada kedua keputusan mengindikasikan bahwa informasi yang digunakan untuk menentukan sentimen dan topik saling berkaitan.

BAB 5

PENUTUP

5.1 Kesimpulan

Berdasarkan hasil penelitian yang telah dilakukan, diperoleh kesimpulan sebagai berikut:

1. Model *Joint Document Level Sentiment-Topic* (JDST) berbasis *Extended Long Short-Term Memory* (xLSTM) berhasil dikembangkan dengan mengintegrasikan analisis sentimen dan pendeteksian topik secara *end-to-end* dalam satu kerangka *multi-task learning*. Model ini menggunakan xLSTM sebagai model representasi teks yang bobotnya dibekukan setelah tahap pra-pelatihan, kemudian representasi token yang dihasilkan diproses oleh *encoder* berbasis MLP 2 layer untuk menghasilkan representasi bersama z . Representasi tersebut digunakan secara bersamaan oleh jalur klasifikasi sentimen melalui MLP 3 layer dan pendeteksian topik melalui DEKM, dengan fungsi *loss* gabungan yang menggunakan bobot adaptif η untuk menyeimbangkan kedua tugas secara otomatis. Enam konfigurasi model diuji, yaitu empat varian xLSTM (xLSTM[1:0], xLSTM[0:1], xLSTM[1:1]-sm, dan xLSTM[1:1]-ms) serta dua varian LSTM (LSTM-1L dan LSTM-2L) sebagai *baseline*.
2. Seluruh konfigurasi JDST-xLSTM menunjukkan kinerja analisis sentimen yang lebih baik dibandingkan kedua konfigurasi JDST-LSTM pada *dataset* IMDB dan Yelp. Di antara seluruh konfigurasi yang dievaluasi, JDST-xLSTM[1:1]-ms secara umum memberikan performa terbaik pada mayoritas skenario pengujian. Hasil ini menunjukkan bahwa mekanisme *exponential gating* dan *matrix memory* pada xLSTM mampu menghasilkan representasi token yang lebih efektif untuk tugas klasifikasi sentimen dibandingkan arsitektur LSTM konvensional.
3. Seluruh konfigurasi JDST-xLSTM secara umum menghasilkan kualitas pendeteksian topik yang lebih baik dan lebih stabil dibandingkan kedua konfigurasi JDST-LSTM pada kedua *dataset*. Meskipun konfigurasi terbaik bervariasi bergantung pada nilai K dan metrik evaluasi yang digunakan, model berbasis xLSTM secara konsisten memberikan nilai koherensi topik dan keberagaman topik yang lebih baik daripada *baseline* LSTM. Secara keseluruhan, JDST-xLSTM[1:1]-ms merupakan konfigurasi yang paling konsisten memberikan kinerja terbaik pada tugas pendeteksian topik.

5.2 Saran

Berdasarkan hasil penelitian yang telah diperoleh, beberapa saran untuk penelitian selanjutnya disampaikan sebagai berikut:

1. Memperluas tahap pra-pelatihan dengan korpus data yang lebih besar dan jumlah *epoch* yang lebih banyak untuk menghasilkan representasi xLSTM yang jauh lebih informatif.
2. Melakukan *hyperparameter tuning* untuk meningkatkan stabilitas dan performa model.

DAFTAR PUSTAKA

- Aggarwal, C. C. *et al.* (2018). *Neural networks and deep learning*, volume 10. Springer.
- Alharthi, M. & Mahmood, A. (2024). xlstmtime: Long-term time series forecasting with xlstm. *AI*, 5(3), 1482–1495. <https://doi.org/10.3390/ai5030071>
- Almeida, F. & Xexéo, G. (2023). *Word embeddings: A survey*. <https://arxiv.org/abs/1901.09069>
- Andreotta, M., Nugroho, R., Hurlstone, M., Boschetti, F., Farrell, S., Walker, I., & Paris, C. (2019). Analyzing social media data: A mixed-methods framework combining computational and qualitative text analysis. *Behavior Research Methods*, 51. <https://doi.org/10.3758/s13428-019-01202-8>
- Ba, J. L., Kiros, J. R., & Hinton, G. E. (2016). *Layer normalization*. <https://arxiv.org/abs/1607.06450>
- Beck, M., Pöppel, K., Spanring, M., Auer, A., Prudnikova, O., Kopp, M., Klambauer, G., Brandstetter, J., & Hochreiter, S. (2024). *xlstm: Extended long short-term memory*. <https://arxiv.org/abs/2405.04517>
- Bengio, Y., Courville, A. C., & Vincent, P. (2012). Unsupervised feature learning and deep learning: A review and new perspectives. *CoRR*, abs/1206.5538. <http://arxiv.org/abs/1206.5538>
- Bishop, C. M. & Nasrabadi, N. M. (2006). *Pattern recognition and machine learning*, volume 4. Springer.
- Blei, D. M. (2012). Probabilistic topic models. *Commun. ACM*, 55(4), 77–84. <https://doi.org/10.1145/2133806.2133826>
- Chung, J., Gulcehre, C., Cho, K., & Bengio, Y. (2014). *Empirical evaluation of gated recurrent neural networks on sequence modeling*. <https://arxiv.org/abs/1412.3555>
- Ciampiconi, L., Elwood, A., Leonardi, M., Mohamed, A., & Rozza, A. (2024). *A survey and taxonomy of loss functions in machine learning*. <https://arxiv.org/abs/2301.05579>
- Da Silva, I. N., Hernane Spatti, D., Andrade Flauzino, R., Liboni, L. H. B., & dos Reis Alves, S. F. (2016). Artificial neural network architectures and training processes. *Artificial Neural Networks: A Practical Course*, 21–28. Springer.

- Dieng, A. B., Ruiz, F. J. R., & Blei, D. M. (2019). Topic modeling in embedding spaces. *CoRR*, abs/1907.04907. <http://arxiv.org/abs/1907.04907>
- Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., & Houlsby, N. (2021). *An image is worth 16x16 words: Transformers for image recognition at scale*. <https://arxiv.org/abs/2010.11929>
- Dubey, S. R., Singh, S. K., & Chaudhuri, B. B. (2022). *Activation functions in deep learning: A comprehensive survey and benchmark*. <https://arxiv.org/abs/2109.14545>
- Elharrouss, O., Mahmood, Y., Bechqito, Y., Serhani, M. A., Badidi, E., Riffi, J., & Tairi, H. (2025). *Task-based loss functions in computer vision: A comprehensive review*. <https://arxiv.org/abs/2504.04242>
- Fu, X., Wu, H., & Cui, L. (2016). Topic sentiment joint model with word embeddings. *Proceedings of the DMNLP Workshop at ECML/PKDD*. <https://ceur-ws.org/Vol-1646/paper6.pdf>. Extended version available as TSWE: Topic Sentiment Joint Model with Word Embeddings
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. The MIT Press.
- Gui, L., Leng, J., Zhou, J., Xu, R., & He, Y. (2022). Multi task mutual learning for joint sentiment classification and topic detection. *IEEE Transactions on Knowledge and Data Engineering*, 34(4), 1915–1927. <https://doi.org/10.1109/TKDE.2020.2999489>
- Guo, W., Lin, K., & Ye, W. (2021). *Deep embedded k-means clustering*. <https://arxiv.org/abs/2109.15149>
- Hendrycks, D. & Gimpel, K. (2023). *Gaussian error linear units (gelus)*. <https://arxiv.org/abs/1606.08415>
- Ho, W. K., Ho, F., & Lee, T. (2012). Exponential function and its derivative revisited. *International Journal of Mathematical Education in Science and Technology*, 44, 1–6. <https://doi.org/10.1080/0020739X.2012.703341>
- Huang, L., Qin, J., Zhou, Y., Zhu, F., Liu, L., & Shao, L. (2020). Normalization techniques in training dnns: Methodology, analysis and application. *CoRR*, abs/2009.12836. <https://arxiv.org/abs/2009.12836>
- Jain, A. K., Murty, M. N., & Flynn, P. J. (1999). Data clustering: a review. *ACM Comput. Surv.*, 31(3), 264–323. <https://doi.org/10.1145/331499.331504>

- Khoshvaght, H., Permala, R. R., Razmjou, A., & Khiadani, M. (2025). A critical review on selecting performance evaluation metrics for supervised machine learning models in wastewater quality prediction. *Journal of Environmental Chemical Engineering*, 13(6), 119675. <https://doi.org/https://doi.org/10.1016/j.jece.2025.119675>
- Kingma, D. P. & Ba, J. (2017). *Adam: A method for stochastic optimization*. <https://arxiv.org/abs/1412.6980>
- Lin, C. (2009). Joint sentiment/topic model for sentiment analysis. *International Conference on Information and Knowledge Management, Proceedings*. <https://doi.org/10.1145/1645953.1646003>
- Liu, B. (2012). Sentiment analysis and opinion mining. *Sentiment Analysis and Opinion Mining*, volume 5. <https://doi.org/10.2200/S00416ED1V01Y201204HLT016>
- Luong, M.-T., Pham, H., & Manning, C. D. (2015). *Effective approaches to attention-based neural machine translation*. <https://arxiv.org/abs/1508.04025>
- Maas, A. L., Daly, R. E., Pham, P. T., Huang, D., Ng, A. Y., & Potts, C. (2011). Learning word vectors for sentiment analysis. *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, 142–150. <https://aclanthology.org/P11-1015/>
- Mienye, I. D., Swart, T. G., & Obaido, G. (2024). Recurrent neural networks: A comprehensive review of architectures, variants, and applications. *Information*, 15(9). <https://doi.org/10.3390/info15090517>
- Murfi, H., Siagian, F. L., & Satria, Y. (2018). Topic features for machine learning-based sentiment analysis in indonesian tweets. *International Journal of Intelligent Computing and Cybernetics*, 12(1), 70–81. <https://doi.org/https://doi.org/10.1108/IJICC-04-2018-0057>
- Nguyen, T. H. & Shirai, K. (2015). Topic modeling based sentiment analysis on social media for stock market prediction. *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, 1354–1364. <https://doi.org/10.3115/v1/P15-1131>
- Nie, Y., Nguyen, N. H., Sinthong, P., & Kalagnanam, J. (2023). *A time series is worth 64 words: Long-term forecasting with transformers*. <https://arxiv.org/abs/2211.14730>

- Nieradzik, L., Scheuermann, G., Saur, D., & Gillmann, C. (2021). *Effect of the output activation function on the probabilities and errors in medical image segmentation*. <https://arxiv.org/abs/2109.00903>
- Nikhil, Nikhil Tikoo, S. K. H. S. P. D. P. G. R. (2015). A survey on text mining and sentiment analysis for unstructured web data. *SSRN Electronic Journal*, 02, 1292–1297.
- O’Callaghan, D., Greene, D., Carthy, J., & Cunningham, P. (2015). An analysis of the coherence of descriptors in topic modeling. *Expert Systems with Applications*, 42(13), 5645–5657. <https://doi.org/https://doi.org/10.1016/j.eswa.2015.02.055>
- Pang, B. & Lee, L. (2008). Opinion mining and sentiment analysis. *Found. Trends Inf. Retr.*, 2(1–2), 1–135. <https://doi.org/10.1561/15000000011>
- Petrus, J., Ermatita, & Sukemi (2019). Soft and hard clustering for abstract scientific paper in indonesian. *2019 International Conference on Informatics, Multimedia, Cyber and Information System (ICIMCIS)*, 131–136. <https://doi.org/10.1109/ICIMCIS48181.2019.8985231>
- Pradipta Hardinatha, E., Malikulmulki Arasy, J., Walidain, C., Multazam, I. H., Athallah Adnan, R., & Aulia Adnan, R. (2025). *Large language models via rust: The state of the art open foundational models in rust*. <https://lmvr.rantai.dev/docs/lmvr/>. Accessed: 2026-01-11.
- Qamar, R. & Zardari, B. (2023). Artificial neural networks: An overview. *Mesopotamian Journal of Computer Science*, 2023, 130–139. <https://doi.org/10.58496/MJCSC/2023/015>
- Radford, A. & Narasimhan, K. (2018). Improving language understanding by generative pre-training. <https://api.semanticscholar.org/CorpusID:49313245>
- Ramachandran, P., Zoph, B., & Le, Q. (2017). Swish: a self-gated activation function. <https://doi.org/10.48550/arXiv.1710.05941>
- Röder, M., Both, A., & Hinneburg, A. (2015). Exploring the space of topic coherence measures. *Proceedings of the Eighth ACM International Conference on Web Search and Data Mining, WSDM 2015, Shanghai, China, February 2-6, 2015*, 399–408. <https://doi.org/10.1145/2684822.2685324>
- Ruder, S. (2017). *An overview of multi-task learning in deep neural networks*. <https://arxiv.org/abs/1706.05098>

- Schmidt, R. M. (2019). *Recurrent neural networks (rnns): A gentle introduction and overview*. <https://arxiv.org/abs/1912.05911>
- Sengupta, A., Paka, W. S., Roy, S., Ranjan, G., & Chakraborty, T. (2021). *An embedding-based joint sentiment-topic model for short texts*. <https://arxiv.org/abs/2103.14410>
- Sokolova, M. & Lapalme, G. (2009). A systematic analysis of performance measures for classification tasks. *Information Processing Management*, 45(4), 427–437. <https://doi.org/10.1016/j.ipm.2009.03.002>
- Titov, I. & McDonald, R. (2008). A joint model of text and aspect ratings for sentiment summarization. *Proceedings of ACL-08: HLT*, 308–316. <https://aclanthology.org/P08-1036/>
- van den Oord, A., Dieleman, S., Zen, H., Simonyan, K., Vinyals, O., Graves, A., Kalchbrenner, N., Senior, A., & Kavukcuoglu, K. (2016). *Wavenet: A generative model for raw audio*. <https://arxiv.org/abs/1609.03499>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2023). *Attention is all you need*. <https://arxiv.org/abs/1706.03762>
- Vincent, P., Larochelle, H., Lajoie, I., Bengio, Y., & Manzagol, P.-A. (2010). Stacked denoising autoencoders: Learning useful representations in a deep network with a local denoising criterion. *J. Mach. Learn. Res.*, 11, 3371–3408.
- Wu, Y. & He, K. (2018). *Group normalization*. <https://arxiv.org/abs/1803.08494>
- Xu, G., Wang, X., Wu, X., Leng, X., & Xu, Y. (2025). Development of residual learning in deep neural networks for computer vision: A survey. *Engineering Applications of Artificial Intelligence*, 142, 109890. <https://doi.org/10.1016/j.engappai.2024.109890>
- Zafarani-Moattar, E., Kangavari, M. R., & Rahmani, A. M. (2024). *A comprehensive study on frequent pattern mining and clustering categories for topic detection in persian text stream*. <https://arxiv.org/abs/2403.10237>
- Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (2023). *Dive into deep learning*. <https://arxiv.org/abs/2106.11342>
- Zhang, L., Wang, S., & Liu, B. (2018). *Deep learning for sentiment analysis : A survey*. <https://arxiv.org/abs/1801.07883>

Zhang, X., Zhao, J., & LeCun, Y. (2015). Character-level convolutional networks for text classification. *arXiv:1509.01626 [cs]*. https://huggingface.co/datasets/fancyzhx/yelp_polarity

Lampiran 1 Code Pra-pelatihan xLSTM

```
1 !pip -q install xlstm transformers datasets tokenizers accelerate gensim nltk
2
3 import os, re, json, time, random, gc
4 import numpy as np
5 import torch
6 import torch.nn as nn
7 import torch.nn.functional as F
8 from torch.utils.data import Dataset, DataLoader
9 from torch.optim import AdamW
10 from torch.amp import GradScaler, autocast
11 from transformers import AutoTokenizer
12 from datasets import load_dataset
13 from tqdm.auto import tqdm
14 import matplotlib
15 matplotlib.use("Agg")
16 import matplotlib.pyplot as plt
17 import gensim.downloader as api
18 import nltk
19
20 from xlstm import (
21     xLSTMBlockStack, xLSTMBlockStackConfig,
22     mLSTMBlockConfig, mLSTMLayerConfig,
23     sLSTMBlockConfig, sLSTMLayerConfig,
24     FeedForwardConfig,
25 )
26
27 nltk.download("stopwords", quiet=True)
28
29
30 def set_seed(seed: int = 42):
31     random.seed(seed); np.random.seed(seed)
32     torch.manual_seed(seed); torch.cuda.manual_seed_all(seed)
33
34
35 set_seed(42)
36 DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
37 SLSTM_BACKEND = "cuda"
38
39 print(f"Device : {DEVICE}")
40 if torch.cuda.is_available():
41     print(f"GPU : {torch.cuda.get_device_name(0)}")
42     print(f"VRAM total : {torch.cuda.get_device_properties(0).total_memory / 1e9:.1f} GB")
43 print(f"PyTorch : {torch.__version__}")
44 print(f"sLSTM backend : {SLSTM_BACKEND}")
45
46
47 XLSTM_CONFIGS = [
48     {"config_name": "1:0", "num_blocks": 1, "slstm_at": [], "save_folder": "10"},
49     {"config_name": "0:1", "num_blocks": 1, "slstm_at": [0], "save_folder": "01"},
50     {"config_name": "1:1", "num_blocks": 2, "slstm_at": [1], "save_folder": "11"},
51     {"config_name": "7:1", "num_blocks": 8, "slstm_at": [7], "save_folder": "71"},
52 ]
53
54
55 TOKENIZER_NAME = "bert-base-uncased"
56 MAX_LENGTH = 128
57 EMBEDDING_DIM = 128
58 GLOVE_DIM = 300
59
60 LSTM_HIDDEN = 64
61 LSTM_LAYERS = 2
62 LSTM_DROPOUT = 0.1
63
64 BATCH_SIZE = 256
65 GRAD_ACCUM = 1
66 USE_AMP = True
67 EMPTY_CACHE_EVERY = 200
68
69 LR = 3e-4
70 WEIGHT_DECAY = 0.01
```

```

71 EPOCHS = 15
72 GRAD_CLIP = 1.0
73 PATIENCE = 4
74 VAL_MAX_BATCHES = 100
75
76 SAVE_BASE_XLSTM = "/content/xlstm_pretrain_glove300"
77 SAVE_BASE_LSTM = "/content/lstm_pretrain_glove300"
78 os.makedirs(SAVE_BASE_XLSTM, exist_ok=True)
79 os.makedirs(SAVE_BASE_LSTM, exist_ok=True)
80
81 print(f"\nGlobal config")
82 print(f"glove_dim : {GLOVE_DIM} (kompresi: {GLOVE_DIM}->{EMBEDDING_DIM}->{GLOVE_DIM})")
83 print(f"embedding_dim : {EMBEDDING_DIM} (bottleneck z)")
84 print(f"batch_size : {BATCH_SIZE}")
85 print(f"epochs (max) : {EPOCHS}")
86 print(f"patience : {PATIENCE}")
87
88
89 def preprocess_english(text: str) -> str:
90     text = text.lower()
91     text = re.sub(r"<[^>+>", " ", text)
92     text = re.sub(r"http\S+|www\.\S+", " ", text)
93     text = text.encode("ascii", errors="ignore").decode()
94     return re.sub(r"\s+", " ", text).strip()
95
96
97 _DATA_CACHE = {}
98
99 def load_pretrain_data(yelp_n: int = 25_000, seed: int = 42):
100     global _DATA_CACHE
101     cache_key = (yelp_n, seed)
102     if cache_key in _DATA_CACHE:
103         print("\n[DATA] Menggunakan cache")
104         return _DATA_CACHE[cache_key]
105
106     print("\n[DATA] Loading IMDB...")
107     imdb = load_dataset("stanfordnlp/imdb")
108     imdb_texts = [preprocess_english(t) for t in imdb["train"]["text"]]
109
110     print("[DATA] Loading Yelp Polarity...")
111     yelp = load_dataset("fancyzhx/yelp-polarity")
112     rng = random.Random(seed)
113     yelp_texts = [preprocess_english(t)
114                   for t in rng.sample(list(yelp["train"]["text"]), yelp_n)]
115
116     all_texts = imdb_texts + yelp_texts
117     rng.shuffle(all_texts)
118
119     split = int(len(all_texts) * 0.9)
120     train_texts = all_texts[:split]
121     val_texts = all_texts[split:]
122
123     print(f"[DATA] Train={len(train_texts):,} | Val={len(val_texts):,}")
124     _DATA_CACHE[cache_key] = (train_texts, val_texts)
125     return train_texts, val_texts
126
127
128 # Tokenizer dan GloVe 300d
129 _SHARED_RESOURCES = {}
130
131 def load_shared_resources():
132     global _SHARED_RESOURCES
133     if _SHARED_RESOURCES:
134         print("\n[SHARED] Menggunakan cache")
135         return _SHARED_RESOURCES
136
137     print(f"\n[TOKENIZER] Loading {TOKENIZER_NAME}...")
138     tok = AutoTokenizer.from_pretrained(TOKENIZER_NAME)
139
140     print("\n[GLOVE] Loading glove-wiki-gigaword-300...")
141     glove = api.load("glove-wiki-gigaword-300")
142
143     print("[GLOVE] Building embedding matrix (300d)...")

```

```

144 vocab_size = tok.vocab_size
145 matrix = np.zeros((vocab_size, GLOVE_DIM), dtype=np.float32)
146 found = subword = rand = 0
147 for token, idx in tok.get_vocab().items():
148     if idx >= vocab_size:
149         continue
150     if token in glove:
151         matrix[idx] = glove[token]
152         found += 1
153     elif token.startswith("##") and token[2:] in glove:
154         matrix[idx] = glove[token[2:]]
155         subword += 1
156     else:
157         matrix[idx] = np.random.normal(0, 0.02, GLOVE_DIM)
158         rand += 1
159 total = found + subword + rand
160 print(f"[GLOVE] Direct={found/total*100:.1f}% "
161       f"Subword={subword/total*100:.1f}% Random={rand/total*100:.1f}%")
162
163 del glove; gc.collect()
164
165 _SHARED_RESOURCES = {
166     "tokenizer" : tok,
167     "vocab_size": vocab_size,
168     "glove_mat" : matrix,
169 }
170 return _SHARED_RESOURCES
171
172
173 class DEKMPretrainDataset(Dataset):
174     def __init__(self, texts, tokenizer, glove_matrix, max_length):
175         self.texts = texts
176         self.tokenizer = tokenizer
177         self.glove_matrix = torch.tensor(glove_matrix, dtype=torch.float32)
178         self.max_length = max_length
179
180     def __len__(self):
181         return len(self.texts)
182
183     def __getitem__(self, idx):
184         enc = self.tokenizer(
185             self.texts[idx], max_length=self.max_length,
186             truncation=True, padding="max_length", return_tensors="pt")
187         ids = enc["input_ids"].squeeze(0)
188         attn_mask = enc["attention_mask"].squeeze(0)
189         emb = self.glove_matrix[ids] # (seq_len, 300)
190         return emb, attn_mask
191
192
193 class xLSTMAutoencoder(nn.Module):
194     """
195     GloVe(300) -> emb_proj(300->128, LN) -> xLSTM bidirectional -> h_t(128)
196     -> mean pool -> z(128) [bottleneck: 300->128]
197     -> decoder(128->300) [rekonstruksi per token]
198     """
199     def __init__(self, glove_dim, embedding_dim,
200                 context_length, slstm_backend,
201                 num_blocks, slstm_at):
202         super().__init__()
203         self.glove_dim = glove_dim
204         self.embedding_dim = embedding_dim
205
206         self.emb_proj = nn.Sequential(
207             nn.Linear(glove_dim, embedding_dim, bias=False),
208             nn.LayerNorm(embedding_dim),
209         )
210         self.emb_dropout = nn.Dropout(0.1)
211
212     def _make_stack():
213         return xLSTMBlockStack(xLSTMBlockStackConfig(
214             mlstm_block=mlstmBlockConfig(mlstm=mlstmLayerConfig(
215                 conv1d_kernel_size=4, qkv_proj_blocksize=4, num_heads=4)),
216             slstm_block=slstmBlockConfig(

```

```

217         slstm=sLSTMLayerConfig(
218             backend=slstm_backend, num_heads=4,
219             conv1d_kernel_size=4,
220             bias_init="powerlaw_blockdependent"),
221         feedforward=FeedForwardConfig(proj_factor=1.3, act_fn="gelu")),
222         context_length=context_length,
223         num_blocks=num_blocks,
224         slstm_at=slstm_at,
225         embedding_dim=embedding_dim,
226         add_post_blocks_norm=True,
227         dropout=0.1)
228
229 self.forward_xlstm = _make_stack()
230 self.backward_xlstm = _make_stack()
231
232 self.bidir_proj = nn.Sequential(
233     nn.Linear(embedding_dim * 2, embedding_dim),
234     nn.LayerNorm(embedding_dim),
235     nn.GELU(),
236 )
237
238 self.decoder = nn.Sequential(
239     nn.Linear(embedding_dim, embedding_dim * 2),
240     nn.GELU(),
241     nn.LayerNorm(embedding_dim * 2),
242     nn.Linear(embedding_dim * 2, glove_dim),
243 )
244
245 def encode(self, x, attention_mask=None):
246     h = self.emb_dropout(self.emb_proj(x))
247     fwd = self.forward_xlstm(h)
248     bwd = torch.flip(self.backward_xlstm(torch.flip(h, dims=[1])), dims=[1])
249     h_t = self.bidir_proj(torch.cat([fwd, bwd], dim=-1))
250     if attention_mask is not None:
251         mask = attention_mask.unsqueeze(-1).float()
252         z = (h_t * mask).sum(1) / mask.sum(1).clamp(min=1e-6)
253     else:
254         z = h_t.mean(1)
255     return z # (B, 128)
256
257 def forward(self, x, attention_mask=None):
258     h = self.emb_dropout(self.emb_proj(x))
259     fwd = self.forward_xlstm(h)
260     bwd = torch.flip(self.backward_xlstm(torch.flip(h, dims=[1])), dims=[1])
261     h_t = self.bidir_proj(torch.cat([fwd, bwd], dim=-1))
262     if attention_mask is not None:
263         mask = attention_mask.unsqueeze(-1).float()
264         z = (h_t * mask).sum(1) / mask.sum(1).clamp(min=1e-6)
265     else:
266         z = h_t.mean(1)
267     recon = self.decoder(h_t) # (B, L, 300)
268     return recon, z
269
270
271 class LSTMAutoencoder(nn.Module):
272     """
273     GloVe(300) -> emb_proj(300->128, LN) -> LSTM(64x2) -> h_t(128)
274     -> mean pool -> z(128) [bottleneck: 300->128]
275     -> decoder(128->300) [rekonstruksi per token]
276     """
277     def __init__(self, glove_dim, embedding_dim,
278                 lstm_hidden, lstm_layers, lstm_dropout):
279         super().__init__()
280         self.glove_dim = glove_dim
281         self.embedding_dim = embedding_dim
282
283         self.emb_proj = nn.Sequential(
284             nn.Linear(glove_dim, embedding_dim, bias=False),
285             nn.LayerNorm(embedding_dim),
286         )
287         self.emb_dropout = nn.Dropout(0.1)
288
289         self.LSTM = nn.LSTM(

```

```

290         input_size      = embedding_dim,
291         hidden_size      = lstm_hidden,
292         num_layers       = lstm_layers,
293         batch_first      = True,
294         bidirectional     = True,
295         dropout           = lstm_dropout if lstm_layers > 1 else 0.0,
296     )
297
298     self.bidir_proj = nn.Sequential(
299         nn.Linear(lstm_hidden * 2, embedding_dim),
300         nn.LayerNorm(embedding_dim),
301         nn.GELU(),
302     )
303
304     self.decoder = nn.Sequential(
305         nn.Linear(embedding_dim, embedding_dim * 2),
306         nn.GELU(),
307         nn.LayerNorm(embedding_dim * 2),
308         nn.Linear(embedding_dim * 2, glove_dim),
309     )
310
311     def encode(self, x, attention_mask=None):
312         h = self.emb_dropout(self.emb_proj(x))
313         lstm_out, _ = self.LSTM(h)
314         h_t = self.bidir_proj(lstm_out)
315         if attention_mask is not None:
316             mask = attention_mask.unsqueeze(-1).float()
317             z = (h_t * mask).sum(1) / mask.sum(1).clamp(min=1e-6)
318         else:
319             z = h_t.mean(1)
320         return z # (B, 128)
321
322     def forward(self, x, attention_mask=None):
323         h = self.emb_dropout(self.emb_proj(x))
324         lstm_out, _ = self.LSTM(h)
325         h_t = self.bidir_proj(lstm_out)
326         if attention_mask is not None:
327             mask = attention_mask.unsqueeze(-1).float()
328             z = (h_t * mask).sum(1) / mask.sum(1).clamp(min=1e-6)
329         else:
330             z = h_t.mean(1)
331         recon = self.decoder(h_t) # (B, L, 300)
332         return recon, z
333
334
335     def recon_loss(recon, target, attention_mask):
336         """MSE rekonstruksi, dirata-ratakan per dimensi dan dimask pada token non-padding."""
337         diff = (recon - target) ** 2
338         diff = diff.mean(-1)
339         mask = attention_mask.float()
340         return (diff * mask).sum() / mask.sum().clamp(min=1e-6)
341
342
343     @torch.no_grad()
344     def evaluate_pretrain(model, loader, max_batches=VAL_MAX_BATCHES):
345         model.eval()
346         total = 0.0; n = 0
347         for step, (emb, amsk) in enumerate(loader):
348             if step >= max_batches:
349                 break
350             emb, amsk = emb.to(DEVICE), amsk.to(DEVICE)
351             with autocast("cuda", enabled=False):
352                 recon, _ = model(emb, amsk)
353                 loss = recon_loss(recon, emb, amsk)
354             total += loss.item(); n += 1
355         return total / max(n, 1)
356
357
358     def print_epoch_header():
359         hdr = (f"{'Ep':>4} {'Train MSE':>12} {'Val MSE':>12} "
360              f"{'VRAM(GB)':>9} {'Time(s)':>8}")
361         print("\n" + "-" * len(hdr))
362         print(hdr)

```

```

363     print("--" * len(hdr))
364
365
366 def print_epoch_row(epoch, tr, val, vram, elapsed, best=False):
367     flag = " (best)" if best else ""
368     print(f"{epoch:>4} {tr:>12.6f} {val:>12.6f} "
369           f"{vram:>9.2f} {elapsed:>8.1f}{flag}")
370
371
372 def get_vram_gb():
373     return torch.cuda.memory_reserved(0) / 1e9 if torch.cuda.is_available() else 0.0
374
375
376 def try_load_resume(model, optimizer, scheduler, scaler, save_dir):
377     path = os.path.join(save_dir, "resume.pt")
378     if not os.path.exists(path):
379         print("[RESUME] Tidak ada checkpoint, mulai dari epoch 1.")
380         return 1, float("inf"), [], 0
381     print(f"[RESUME] Ditemukan: {path}")
382     ck = torch.load(path, map_location=DEVICE, weights_only=False)
383     model.load_state_dict(ck["model_state"])
384     optimizer.load_state_dict(ck["optimizer_state"])
385     scheduler.load_state_dict(ck["scheduler_state"])
386     scaler.load_state_dict(ck["scaler_state"])
387     print(f"[RESUME] Lanjut dari epoch {ck['epoch']+1} | best={ck['best_val']:.6f}")
388     return ck["epoch"] + 1, ck["best_val"], ck.get("history", []), ck.get("no_improve", 0)
389
390
391 def plot_loss_curve(history, title, save_path):
392     if not history:
393         return
394     epochs = [h["epoch"] for h in history]
395     tr_loss = [h["train_mse"] for h in history]
396     val_loss = [h["val_mse"] for h in history]
397     ep_times = [h["epoch_time_s"] for h in history]
398     vram = [h["vram_gb"] for h in history]
399
400     fig, axes = plt.subplots(1, 3, figsize=(15, 4))
401     fig.suptitle(title, fontsize=13)
402
403     axes[0].plot(epochs, tr_loss, label="Train MSE")
404     axes[0].plot(epochs, val_loss, label="Val MSE")
405     axes[0].set_title("Reconstruction MSE Loss")
406     axes[0].set_xlabel("Epoch"); axes[0].legend(); axes[0].grid(True, alpha=0.3)
407
408     axes[1].bar(epochs, ep_times, color="steelblue", alpha=0.7)
409     axes[1].set_title("Time per Epoch (s)")
410     axes[1].set_xlabel("Epoch"); axes[1].grid(True, alpha=0.3, axis="y")
411
412     axes[2].plot(epochs, vram, color="tomato", marker="o", markersize=4)
413     axes[2].set_title("VRAM (GB)"); axes[2].set_xlabel("Epoch")
414     axes[2].axhline(15.0, color="gray", ls="--", lw=0.8, label="~15GB A100")
415     axes[2].legend(); axes[2].grid(True, alpha=0.3)
416
417     plt.tight_layout()
418     plt.savefig(save_path, dpi=120, bbox_inches="tight")
419     plt.close()
420     print(f" Plot disimpan -> {save_path}")
421
422
423 def train_one_model(model, config_name, save_dir,
424                    train_loader, val_loader):
425     """Loop training generik, dipakai baik untuk xLSTM maupun LSTM."""
426     set_seed(42)
427     os.makedirs(save_dir, exist_ok=True)
428
429     n_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
430     print(f"\nPretrain {config_name}")
431     print(f" Kompresi : GloVe {GLOVE_DIM}d -> z({EMBEDDING_DIM}) -> recon {GLOVE_DIM}d")
432     print(f" Params : {n_params:,}")
433     print(f" VRAM : {get_vram_gb():.2f} GB")
434     print(f" save_dir : {save_dir}")
435

```

```

436 optimizer = AdamW(model.parameters(), lr=LR, weight_decay=WEIGHT_DECAY)
437 total_steps = max(1, (len(train_loader) // GRAD_ACCUM) * EPOCHS)
438 scheduler = torch.optim.lr_scheduler.OneCycleLR(
439     optimizer, max_lr=LR, total_steps=total_steps,
440     pct_start=0.1, anneal_strategy="cos",
441     div_factor=10, final_div_factor=100,
442 )
443 scaler = GradScaler("cuda", enabled=USE_AMP)
444
445 start_epoch, best_val, history, no_improve = try_load_resume(
446     model, optimizer, scheduler, scaler, save_dir)
447
448 global_t0 = time.time()
449 print_epoch_header()
450
451 for epoch in range(start_epoch, EPOCHS + 1):
452     model.train()
453     optimizer.zero_grad()
454     ep_loss = 0.0; n_batches = 0
455     ep_t0 = time.time()
456
457     for step, (emb, amsk) in enumerate(
458         tqdm(train_loader, desc=f" [{config_name}] Ep {epoch:>2}/{EPOCHS}",
459             leave=False)):
460         emb, amsk = emb.to(DEVICE), amsk.to(DEVICE)
461
462         with autocast("cuda", enabled=USE_AMP):
463             recon, _ = model(emb, amsk)
464             loss = recon_loss(recon, emb, amsk) / GRAD_ACCUM
465
466         scaler.scale(loss).backward()
467
468         if (step + 1) % GRAD_ACCUM == 0:
469             scaler.unscale_(optimizer)
470             nn.utils.clip_grad_norm_(model.parameters(), GRAD_CLIP)
471             scaler.step(optimizer)
472             scaler.update()
473             scheduler.step()
474             optimizer.zero_grad()
475
476         ep_loss += loss.item() * GRAD_ACCUM
477         n_batches += 1
478
479         if (step + 1) % EMPTY_CACHE_EVERY == 0:
480             torch.cuda.empty_cache()
481
482     leftover = n_batches % GRAD_ACCUM
483     if GRAD_ACCUM > 1 and leftover != 0:
484         scaler.unscale_(optimizer)
485         nn.utils.clip_grad_norm_(model.parameters(), GRAD_CLIP)
486         scaler.step(optimizer); scaler.update(); optimizer.zero_grad()
487     torch.cuda.empty_cache()
488
489     tr_loss = ep_loss / max(n_batches, 1)
490     val_loss = evaluate_pretrain(model, val_loader)
491     torch.cuda.empty_cache()
492
493     ep_elapsed = time.time() - ep_t0
494     vram_used = get_vram_gb()
495     is_best = val_loss < best_val
496
497     print_epoch_row(epoch, tr_loss, val_loss, vram_used, ep_elapsed, is_best)
498
499     if is_best:
500         best_val = val_loss; no_improve = 0
501         torch.save({
502             "encoder_state": {
503                 k: v for k, v in model.state_dict().items()
504                 if not k.startswith("decoder")
505             },
506             "full_model_state": model.state_dict(),
507             "config": {
508                 "config_name" : config_name,

```

```

509         "glove_dim"      : GLOVE_DIM,
510         "embedding_dim"  : EMBEDDING_DIM,
511         "max_length"     : MAX_LENGTH,
512         "slstm_backend"  : SLSTM_BACKEND,
513         "pretrain_type": "autoencoder_glove300_mse",
514     },
515     os.path.join(save_dir, "pretrain_best.pt"))
516     print(f"Checkpoint terbaik disimpan | val_mse={best_val:.6f}")
517 else:
518     no_improve += 1
519     print(f"    Tidak ada peningkatan: {no_improve}/{PATIENCE}")
520     if no_improve >= PATIENCE:
521         print(f"\n    Early stopping [{config_name}]]")
522         break
523
524     torch.save({
525         "epoch"          : epoch,
526         "model_state"     : model.state_dict(),
527         "optimizer_state": optimizer.state_dict(),
528         "scheduler_state": scheduler.state_dict(),
529         "scaler_state"    : scaler.state_dict(),
530         "best_val"        : best_val,
531         "no_improve"      : no_improve,
532         "history"         : history,
533     }, os.path.join(save_dir, "resume.pt"))
534
535     history.append({
536         "epoch"          : epoch,
537         "train_mse"      : round(tr_loss, 6),
538         "val_mse"        : round(val_loss, 6),
539         "epoch_time_s"   : round(ep_elapsed, 2),
540         "vram_gb"        : round(vram_used, 3),
541     })
542
543     with open(os.path.join(save_dir, "history.json"), "w") as f:
544         json.dump(history, f, indent=2)
545     plot_loss_curve(
546         history,
547         title = f"Pretrain {config_name} - GloVe300 -> z128 -> recon300",
548         save_path = os.path.join(save_dir, "loss_curve.png"),
549     )
550
551     elapsed = time.time() - global_t0
552     print(f"\n Selesai {config_name}")
553     print(f"    Best val_mse : {best_val:.6f}")
554     print(f"    Total waktu   : {elapsed/60:.1f} menit")
555
556     del model; torch.cuda.empty_cache(); gc.collect()
557     return best_val
558
559 def run_all():
560     total_start = time.time()
561
562     train_texts, val_texts = load_pretrain_data(yelp_n=25_000, seed=42)
563     res = load_shared_resources()
564     tokenizer = res["tokenizer"]
565     glove_mat = res["glove_mat"]
566
567     train_ds = DEKMPretrainDataset(train_texts, tokenizer, glove_mat, MAX_LENGTH)
568     val_ds = DEKMPretrainDataset(val_texts, tokenizer, glove_mat, MAX_LENGTH)
569     print(f"\n[DATASET] train={len(train_ds):,} | val={len(val_ds):,}")
570
571     train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE,
572                               shuffle=True, num_workers=0, pin_memory=True)
573     val_loader = DataLoader(val_ds, batch_size=BATCH_SIZE,
574                             shuffle=False, num_workers=0, pin_memory=True)
575
576     summary = {}
577
578     # Pretraining untuk setiap konfigurasi xLSTM
579     for i, cfg in enumerate(XLSTM_CONFIGS):
580         cname = f"xLSTM[{cfg['config_name']}]"
581

```

```

582     save_dir = os.path.join(SAVE_BASE_XLSTM, cfg["save_folder"])
583
584     print(f"\n\nxLSTM config {i+1}/{len(XLSTM_CONFIGS)} : {cname}")
585
586     model = xLSTMAutoencoder(
587         glove_dim = GLOVE_DIM,
588         embedding_dim = EMBEDDING_DIM,
589         context_length = MAX_LENGTH,
590         slstm_backend = SLSTM_BACKEND,
591         num_blocks = cfg["num_blocks"],
592         slstm_at = cfg["slstm_at"],
593     ).to(DEVICE)
594
595     best_val = train_one_model(model, cname, save_dir,
596                               train_loader, val_loader)
597     summary[cname] = {"best_val_mse": round(best_val, 6)}
598
599     # Baseline LSTM
600     print(f"\n\nBaseline : LSTM")
601
602     lstm_model = LSTMAutoencoder(
603         glove_dim = GLOVE_DIM,
604         embedding_dim = EMBEDDING_DIM,
605         lstm_hidden = LSTM_HIDDEN,
606         lstm_layers = LSTM_LAYERS,
607         lstm_dropout = LSTM_DROPOUT,
608     ).to(DEVICE)
609
610     best_val_lstm = train_one_model(lstm_model, "LSTM", SAVE_BASE_LSTM,
611                                    train_loader, val_loader)
612     summary["LSTM"] = {"best_val_mse": round(best_val_lstm, 6)}
613
614     elapsed = time.time() - total_start
615     print(f"\n\nSemua selesai - {elapsed/60:.1f} menit")
616     print(f"Glove: {GLOVE_DIM}d | Bottleneck z: {EMBEDDING_DIM}d")
617     for name, v in summary.items():
618         print(f" {name:<20} best_val_mse={v['best_val_mse']:.6f}")
619
620     all_summary = {
621         "glove_dim" : GLOVE_DIM,
622         "embedding_dim": EMBEDDING_DIM,
623         "results" : summary,
624     }
625     with open(os.path.join(SAVE_BASE_XLSTM, "pretrain_summary.json"), "w") as f:
626         json.dump(all_summary, f, indent=2)
627
628
629 run_all()

```

Listing 1: *Code pretraining xLSTM dan LSTM*

Lampiran 2 Code Joint Training JDST

```
1
2
3
4 !pip -q install xlstm transformers tokenizers accelerate nltk gensim scikit-learn huggingface_hub pandas
5
6 import os, re, json, time, random, gc
7 from collections import Counter
8
9 import numpy as np
10 import pandas as pd
11 import torch
12 import torch.nn as nn
13 import torch.nn.functional as F
14 from torch.utils.data import Dataset, DataLoader
15 from torch.optim import AdamW
16
17 from transformers import AutoTokenizer
18 from huggingface_hub import hf_hub_download
19
20 from sklearn.cluster import KMeans
21 from sklearn.manifold import TSNE
22 from sklearn.metrics import (
23     accuracy_score, f1_score, precision_score, recall_score,
24     normalized_mutual_info_score,
25 )
26 from scipy.optimize import linear_sum_assignment
27 from scipy.linalg import eigh
28 from tqdm.auto import tqdm
29
30 import matplotlib
31 matplotlib.use("Agg")
32 import matplotlib.pyplot as plt
33
34 import gensim.downloader as api
35 import nltk
36 from nltk.corpus import stopwords
37
38 from xlstm import (
39     xLSTMBlockStack, xLSTMBlockStackConfig,
40     mLSTMBlockConfig, mLSTMLayerConfig,
41     sLSTMBlockConfig, sLSTMLayerConfig,
42     FeedForwardConfig,
43 )
44
45 nltk.download("stopwords", quiet=True)
46
47
48 # DEVICE
49 def _setup():
50     dev = torch.device("cuda" if torch.cuda.is_available() else "cpu")
51     be = "vanilla"
52     try:
53         if torch.cuda.is_available() and torch.cuda.get_device_capability()[0] >= 8:
54             be = "cuda"
55     except Exception: pass
56     return dev, be
57
58 DEVICE, SLSTM_BACKEND = _setup()
59
60
61 # HYPERPARAMETERS
62 MODEL_REPO = "Fyodorr/xLSTMCompressed"
63 DATASET_REPO = "Fyodorr/Benchmark"
64
65 SAVE_BASE_ROOT = "/content/jst_results"
66
67 TOKENIZER_NAME = "bert-base-uncased"
68 MAX_VOCAB = 30_000
69 MAX_LENGTH = 256
70 GLOVE_DIM = 300
```

```

71 ENCODER_DIM      = 256
72 BOTTLENECK_DIM  = 128
73 DROPOUT          = 0.3
74
75 LSTM_HIDDEN     = 128
76 LSTM_DROPOUT    = 0.1
77
78 BATCH_SIZE      = 64
79 LR              = 3e-4
80 ENCODER_LR_SCALE = 0.05    # xLSTM/LSTM encoder gets LR * 0.05
81 WEIGHT_DECAY    = 1e-3
82 GRAD_CLIP       = 1.0
83
84 TMAX            = 60
85 ETA_MIN         = 1e-6
86
87 LAMBDA_KL        = 0.7
88 LAMBDA_MSE       = 0.3
89 CLUSTER_UPDATE_EVERY = 5
90
91 WARMUP_EPOCHS   = 15
92 MAX_EPOCHS      = 60
93 PATIENCE        = 10
94 KMEANS_N_INIT   = 20
95
96 SEEDS           = [42, 123, 456]
97 N_RUNS          = 3
98
99 TSNE_INTERVAL   = 10
100 TOP_WORDS_N     = 10
101 BERT_SPECIAL_IDS = {0, 100, 101, 102, 103}
102
103 JOINT_W_F1      = 1.0
104 JOINT_W_TC      = 0.1
105 JOINT_W_TD      = 0.1
106 JOINT_W_STAB    = 0.1
107
108 METRICS = [
109     "accuracy", "f1", "precision", "recall",
110     "nmi_vs_sentiment", "tc_w2v", "td",
111 ]
112 METRIC_LABELS = {
113     "accuracy"      : "Accuracy",
114     "f1"            : "F1 macro",
115     "precision"     : "Precision",
116     "recall"        : "Recall",
117     "nmi_vs_sentiment": "NMI (vs sentiment)",
118     "tc_w2v"        : "TC-W2V",
119     "td"            : "Topic Diversity",
120 }
121
122
123
124 # CONFIG REGISTRY
125 CONFIG_REGISTRY = {
126     "lstm-1layer": dict(
127         config_name="LSTM-1layer", model_type="lstm", lstm_layers=1,
128         ckpt_filename="lstm_1layer.pt", save_folder="LSTM_1layer"),
129     "lstm-2layer": dict(
130         config_name="LSTM-2layer", model_type="lstm", lstm_layers=2,
131         ckpt_filename="lstm_2layer.pt", save_folder="LSTM_2layer"),
132     "slstm-1block": dict(
133         config_name="sLSTM-1block", model_type="xlstm", num_blocks=1,
134         slstm_at=[0], ckpt_filename="slstm_1block.pt",
135         save_folder="sLSTM_1block"),
136     "mlstm-1block": dict(
137         config_name="mLSTM-1block", model_type="xlstm", num_blocks=1,
138         slstm_at=[], ckpt_filename="mlstm_1block.pt",
139         save_folder="mLSTM_1block"),
140     "xlstm-s-m": dict(
141         config_name="xLSTM[sLSTM - mLSTM]", model_type="xlstm", num_blocks=2,
142         slstm_at=[0], ckpt_filename="xlstm_s_m.pt",
143         save_folder="xLSTM_s-m"),

```

```

144     "xlstm-m-s": dict(
145         config_name="XLSTM[mLSTM - sLSTM]", model_type="xlstm", num_blocks=2,
146         slstm_at=1, ckpt_filename="xlstm_m_s.pt",
147         save_folder="XLSTM-m-s"),
148     }
149
150
151     # STOPWORDS
152     _BASE_STOP = set(stopwords.words("english")).union({
153         "like", "just", "know", "get", "would", "could", "also", "really",
154         "good", "great", "bad", "one", "even", "still", "well", "see",
155         "go", "come", "make", "time", "way", "think", "people", "say",
156         "thing", "feel", "made", "first", "much", "many", "two", "little",
157         "nothing", "lot", "back", "got", "us", "look", "watch",
158     })
159     _STOP_DS = {
160         "imdb": _BASE_STOP | {
161             "movie", "film", "story", "character", "characters", "scene", "scenes",
162             "plot", "show", "act", "acting", "actor", "actress", "director",
163             "cinema", "screen", "role", "cast", "watched", "films", "movies",
164             "series", "episode", "sequel", "ending", "begin", "version",
165             "seen", "part", "love", "hate"},
166         "yelp": _BASE_STOP | {
167             "place", "food", "restaurant", "service", "order", "ordered",
168             "came", "went", "going", "table", "menu", "staff", "night",
169             "dining", "lunch", "dinner", "breakfast", "eat", "eating",
170             "drink", "drinking", "bar", "cafe", "hotel", "store", "shop",
171             "location", "experience", "visit", "visited", "wait", "waited",
172             "minute", "minutes", "hour", "hours", "day", "told", "asked",
173             "said", "give", "given", "taking", "took"},
174     }
175
176
177     # UTILS
178     def set_seed(s):
179         random.seed(s); np.random.seed(s)
180         torch.manual_seed(s); torch.cuda.manual_seed_all(s)
181
182
183     # SHARED RESOURCES
184     _SHARED = {}
185
186     def get_shared():
187         if _SHARED: return _SHARED
188
189         tok = AutoTokenizer.from_pretrained(TOKENIZER_NAME)
190
191         glove = api.load("glove-wiki-gigaword-300")
192
193         vsize = tok.vocab_size
194         M = np.zeros((vsize, GLOVE_DIM), dtype=np.float32)
195         for token, idx in tok.get_vocab().items():
196             if idx >= vsize: continue
197             if token in glove:
198                 M[idx] = glove[token]
199             elif token.startswith("##") and token[2:] in glove:
200                 M[idx] = glove[token[2:]]
201             else:
202                 M[idx] = np.random.normal(0, .02, GLOVE_DIM)
203
204         _SHARED.update(
205             tokenizer = tok,
206             glove_model = glove,
207             glove_matrix = M,
208             id2word = {v: k for k, v in tok.get_vocab().items()},
209         )
210         return _SHARED
211
212
213     _DATASET_CACHE = {}
214
215     class IndexedDataset(Dataset):
216         def __init__(self, input_ids, attention_mask, labels):

```



```

290         conv1d_kernel_size=4,
291         bias_init="powerlaw_blockdependent"),
292         feedforward=FeedForwardConfig(
293             proj_factor=1.3, act_fn="gelu")),
294         context_length=context_length,
295         num_blocks=num_blocks, slstm_at=slstm_at,
296         embedding_dim=encoder_dim,
297         add_post_blocks_norm=True, dropout=0.1))
298
299     self.forward_xlstm = _stack()
300     self.backward_xlstm = _stack()
301     self.bidir_proj = nn.Sequential(
302         nn.Linear(encoder_dim * 2, encoder_dim),
303         nn.LayerNorm(encoder_dim), nn.GELU())
304
305     self.fc_encoder = nn.Sequential(
306         nn.Linear(encoder_dim, encoder_dim * 2), nn.ReLU(),
307         nn.Linear(encoder_dim * 2, bottleneck_dim))
308
309     def get_sentence_embedding(self, input_ids, attention_mask=None):
310         x = self.emb_dropout(self.emb_proj(
311             self.token_embedding(input_ids)))
312         fwd = self.forward_xlstm(x)
313         bwd = torch.flip(
314             self.backward_xlstm(torch.flip(x, dims=[1])), dims=[1])
315         h_t = self.bidir_proj(torch.cat([fwd, bwd], dim=-1))
316         if attention_mask is not None:
317             m = attention_mask.unsqueeze(-1).float()
318             z = (h_t * m).sum(1) / m.sum(1).clamp(min=1e-6)
319         else:
320             z = h_t.mean(1)
321         return self.fc_encoder(z)
322
323
324     class LSTMEncoderCompressed(nn.Module):
325     def __init__(self, vocab_size, glove_dim, encoder_dim, bottleneck_dim,
326                 pad_token_id, glove_matrix,
327                 lstm_hidden, lstm_layers, lstm_dropout):
328         super().__init__()
329         self.token_embedding = nn.Embedding(
330             vocab_size, glove_dim, padding_idx=pad_token_id)
331         self.token_embedding.weight.data.copy_(torch.from_numpy(glove_matrix))
332
333         self.emb_proj = nn.Sequential(
334             nn.Linear(glove_dim, encoder_dim, bias=False),
335             nn.LayerNorm(encoder_dim))
336         self.emb_dropout = nn.Dropout(0.1)
337
338         self.bilstm = nn.LSTM(
339             input_size=encoder_dim, hidden_size=lstm_hidden,
340             num_layers=lstm_layers, batch_first=True,
341             bidirectional=True,
342             dropout=lstm_dropout if lstm_layers > 1 else 0.0)
343         self.bidir_proj = nn.Sequential(
344             nn.Linear(encoder_dim, encoder_dim),
345             nn.LayerNorm(encoder_dim), nn.GELU())
346
347         self.fc_encoder = nn.Sequential(
348             nn.Linear(encoder_dim, encoder_dim * 2), nn.ReLU(),
349             nn.Linear(encoder_dim * 2, bottleneck_dim))
350
351     def get_sentence_embedding(self, input_ids, attention_mask=None):
352         x = self.emb_dropout(self.emb_proj(
353             self.token_embedding(input_ids)))
354         h_t, _ = self.bilstm(x)
355         h_t = self.bidir_proj(h_t)
356         if attention_mask is not None:
357             m = attention_mask.unsqueeze(-1).float()
358             z = (h_t * m).sum(1) / m.sum(1).clamp(min=1e-6)
359         else:
360             z = h_t.mean(1)
361         return self.fc_encoder(z)
362

```

```

363
364 # JST MODEL
365 class JSTModel(nn.Module):
366     def __init__(self, encoder, n_clusters, dim=BOTTLENECK_DIM):
367         super().__init__()
368         self.encoder = encoder
369         self.n_clusters = n_clusters
370
371         self.sentiment_head = nn.Sequential(
372             nn.Linear(dim, dim), nn.LayerNorm(dim),
373             nn.GELU(), nn.Dropout(DROPOUT),
374             nn.Linear(dim, 32), nn.LayerNorm(32),
375             nn.GELU(), nn.Dropout(DROPOUT),
376             nn.Linear(32, 2))
377
378         self.topic_proj = nn.Sequential(
379             nn.Linear(dim, dim), nn.LayerNorm(dim), nn.Tanh())
380
381         self.cluster_centers = nn.Parameter(torch.randn(n_clusters, dim))
382         self.log_sigma_cluster = nn.Parameter(torch.zeros(1))
383
384     def forward(self, input_ids, attention_mask):
385         z = self.encoder.get_sentence_embedding(input_ids, attention_mask)
386         z_topic = self.topic_proj(z)
387         s_logit = self.sentiment_head(z)
388         return z, z_topic, s_logit
389
390
391 # LOSS & DEKM
392 def student_t_q(z, centers, alpha=1.0):
393     d2 = torch.cdist(z, centers, p=2.0).pow(2)
394     q = (1.0 + d2 / alpha).pow(-(alpha + 1.0) / 2.0)
395     return q / q.sum(1, keepdim=True).clamp(min=1e-8)
396
397 def target_p(q):
398     w = q.pow(2) / q.sum(0, keepdim=True).clamp(min=1e-8)
399     return w / w.sum(1, keepdim=True).clamp(min=1e-8)
400
401 def dekm_mse_loss(z, centers, a_idx):
402     return F.mse_loss(z, centers[a_idx].detach())
403
404 def compute_orthogonal_transform(H, assignment, K):
405     D = H.shape[1]
406     Sw = np.zeros((D, D), dtype=np.float64)
407     for k in range(K):
408         m = assignment == k
409         if m.sum() < 2: continue
410         Hk = H[m].astype(np.float64)
411         d = Hk - Hk.mean(0, keepdims=True)
412         Sw += d.T @ d
413     _, V = eigh(Sw)
414     return V.astype(np.float32)
415
416 def kmeans_refresh(H, K, a_old, V, seed):
417     Y = H @ V
418     km = KMeans(n_clusters=K, n_init=KMEANS_N_INIT,
419                 init="k-means++", random_state=seed)
420     km.fit(Y)
421     a_new = km.labels_.copy()
422     w = np.zeros((K, K), dtype=np.int64)
423     for i in range(len(a_new)):
424         w[a_new[i], a_old[i]] += 1
425     row, col = linear_sum_assignment(-w)
426     mapping = {c: r for r, c in zip(row, col)}
427     a_match = np.array([mapping.get(a, a) for a in a_new])
428     U = km.cluster_centers_ @ V.T
429     return a_match, U, int((a_match != a_old).sum())
430
431
432 # EXTRACTION & EVAL
433 @torch.no_grad()
434 def extract_embeddings(model, loader):
435     model.eval(); zs, ys, ids = [], [], []

```

```

436     for i, m, l, _ in loader:
437         _, zt, _ = model(i.to(DEVICE), m.to(DEVICE))
438         zs.append(zt.cpu().numpy()); ys.append(l.numpy()); ids.append(i.numpy())
439     return np.vstack(zs), np.concatenate(ys), np.vstack(ids)
440
441 @torch.no_grad()
442 def eval_sentiment(model, loader):
443     model.eval(); preds, trues = [], []
444     for i, m, l, _ in loader:
445         _, _, s = model(i.to(DEVICE), m.to(DEVICE))
446         preds.append(s.argmax(1).cpu().numpy()); trues.append(l.numpy())
447     yp, yt = np.concatenate(preds), np.concatenate(trues)
448     return dict(accuracy=float(accuracy_score(yt, yp)),
449               f1=float(f1_score(yt, yp, average="macro")),
450               precision=float(precision_score(yt, yp, average="macro", zero_division=0)),
451               recall=float(recall_score(yt, yp, average="macro", zero_division=0)))
452
453
454 # TOPIC METRICS
455 def get_top_words(all_ids, assignment, K, id2word,
456                  top_n=10, stop_words=None):
457     if stop_words is None: stop_words = _BASE_STOP
458     counts = {k: Counter() for k in range(K)}
459     for doc_ids, cid in zip(all_ids, assignment):
460         for tid in doc_ids:
461             if int(tid) in BERT_SPECIAL_IDS: continue
462             w = id2word.get(int(tid), "")
463             if (w in stop_words or w.startswith("##")
464                 or len(w) <= 2 or not w.isalpha()):
465                 continue
466             counts[cid][w] += 1
467     return {k: [w for w, _ in counts[k].most_common(top_n)] for k in range(K)}
468
469 def topic_coherence_w2v(tw, gm, top_n=10):
470     scores = []
471     for words in tw.values():
472         valid = [w for w in words[:top_n] if w in gm]
473         if len(valid) < 2: scores.append(0.); continue
474         vecs = np.array([gm[w] for w in valid])
475         vecs /= np.linalg.norm(vecs, axis=1, keepdims=True).clip(1e-8)
476         sim = vecs @ vecs.T; n = len(valid)
477         scores.append(float((sim.sum() - n) / max(n*(n-1), 1)))
478     return float(np.mean(scores)) if scores else 0.
479
480 def topic_diversity(tw, top_n=10):
481     all_w = [w for words in tw.values() for w in words[:top_n]]
482     return float(len(set(all_w)) / max(len(all_w), 1))
483
484
485 # T-SNE
486 def plot_tsne(Z, y, a, K, path, title, n=3000):
487     idx = np.random.choice(len(Z), min(n, len(Z)), replace=False)
488     Z2 = TSNE(n_components=2, perplexity=30, max_iter=700,
489              random_state=42, init="pca",
490              learning_rate="auto").fit_transform(Z[idx])
491     cmap = matplotlib.colormaps.get_cmap("tab20")
492     fig, ax = plt.subplots(figsize=(10, 8))
493     for k in range(K):
494         for s in [0, 1]:
495             m = (a[idx] == k) & (y[idx] == s)
496             if m.sum() == 0: continue
497             ax.scatter(Z2[m, 0], Z2[m, 1], c=[cmap(k % 20)],
498                      marker="^" if s == 0 else "o",
499                      s=16 if s == 0 else 12, alpha=.5, linewidths=0)
500     ax.set_title(title, fontsize=9); ax.axis("off")
501     plt.tight_layout(); plt.savefig(path, dpi=130, bbox_inches="tight")
502     plt.close()
503
504
505 def build_encoder(cfg, glove_matrix):
506     ckpt = torch.load(
507         hf_hub_download(repo_id=MODEL_REPO, filename=cfg["ckpt_filename"]),
508         map_location=DEVICE, weights_only=False)

```

```

509
510     res = get_shared()
511     vs = res["tokenizer"].vocab_size
512     pid = res["tokenizer"].pad_token_id
513
514     if cfg["model_type"] == "xlstm":
515         enc = xLSTMEncoderCompressed(
516             vocab_size=vs, glove_dim=GLOVE_DIM,
517             encoder_dim=ENCODER_DIM, bottleneck_dim=BOTTLENECK_DIM,
518             context_length=MAX_LENGTH, slstm_backend=SLSTM_BACKEND,
519             pad_token_id=pid, glove_matrix=glove_matrix,
520             num_blocks=cfg["num_blocks"], slstm_at=cfg["slstm_at"]
521         ).to(DEVICE)
522     else:
523         enc = LSTMEncoderCompressed(
524             vocab_size=vs, glove_dim=GLOVE_DIM,
525             encoder_dim=ENCODER_DIM, bottleneck_dim=BOTTLENECK_DIM,
526             pad_token_id=pid, glove_matrix=glove_matrix,
527             lstm_hidden=LSTM_HIDDEN, lstm_layers=cfg["lstm_layers"],
528             lstm_dropout=LSTM_DROPOUT
529         ).to(DEVICE)
530
531     missing, _ = enc.load_state_dict(ckpt["encoder_state"], strict=False)
532     if missing:
533
534         n_total = sum(p.numel() for p in enc.parameters())
535         return enc
536
537
538 def build_optimizer(model):
539     encoder_slow = [] # xLSTM/LSTM backbone    slow LR
540     fc_params = [] # FC compressor    full LR
541     head_params = [] # everything else    full LR
542
543     for name, p in model.named_parameters():
544         if not p.requires_grad:
545             continue
546         if name.startswith("encoder.fc_encoder"):
547             fc_params.append(p)
548         elif name.startswith("encoder."):
549             encoder_slow.append(p)
550         else:
551             head_params.append(p)
552
553     enc_lr = LR * ENCODER_LR_SCALE
554
555
556     optimizer = AdamW([
557         {"params": encoder_slow, "lr": enc_lr},
558         {"params": fc_params, "lr": LR},
559         {"params": head_params, "lr": LR},
560     ], weight_decay=WEIGHT_DECAY)
561
562     return optimizer
563
564
565 # SINGLE RUN
566 def _single_run(cfg, K, seed, save_dir,
567                 train_loader, embed_loader, test_loader,
568                 glove_matrix, glove_model, id2word, stop_words):
569     set_seed(seed)
570     os.makedirs(save_dir, exist_ok=True)
571     tsne_dir = os.path.join(save_dir, "tsne")
572     os.makedirs(tsne_dir, exist_ok=True)
573     cname = cfg["config_name"]
574
575     enc = build_encoder(cfg, glove_matrix)
576     model = JSTModel(enc, K).to(DEVICE)
577
578     H, y_tr, ids_tr = extract_embeddings(model, embed_loader)
579     km = KMeans(n_clusters=K, n_init=KMEANS_N_INIT,
580                 init="k-means++", random_state=seed)
581     km.fit(H)

```

```

582 assignment = km.labels_.copy()
583 V = compute_orthogonal_transform(H, assignment, K)
584 model.cluster_centers.data.copy_(
585     torch.tensor(km.cluster_centers_, dtype=torch.float32, device=DEVICE))
586 N = len(H)
587
588 optimizer = build_optimizer(model)
589 scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
590     optimizer, T_max=TMAX, eta_min=ETA_MIN)
591
592 best_score = -1e9; patience_cnt = 0; history = []; t0 = time.time()
593
594 for epoch in range(1, MAX_EPOCHS + 1):
595     is_warmup = epoch <= WARMUP_EPOCHS
596     model.train()
597     els = elc = elkl = elmse = esig = nb = 0.
598
599     for ids, mask, labels, indices in tqdm(
600         train_loader, desc=f" Ep {epoch:>2}", leave=False):
601         ids = ids.to(DEVICE); mask = mask.to(DEVICE)
602         labels = labels.to(DEVICE); idx_np = indices.numpy()
603
604         z, zt, sl = model(ids, mask)
605         ls = F.cross_entropy(sl, labels, label_smoothing=0.1)
606
607         if is_warmup:
608             lkl = lmse = lc = torch.tensor(0., device=DEVICE)
609             loss = ls
610         else:
611             q = student_t_q(zt, model.cluster_centers)
612             p = target_p(q).detach()
613             lkl = F.kl_div(torch.log(q.clamp(1e-8)), p,
614                 reduction="batchmean")
615             at = torch.tensor(assignment[idx_np],
616                 dtype=torch.long, device=DEVICE)
617             lmse = dekm_mse_loss(zt, model.cluster_centers, at)
618             lc = LAMBDA_KL * lkl + LAMBDA_MSE * lmse
619
620             precision = torch.exp(-model.log_sigma_cluster)
621             loss = ls + precision * lc + model.log_sigma_cluster
622
623             optimizer.zero_grad()
624             loss.backward()
625             nn.utils.clip_grad_norm_(model.parameters(), GRAD_CLIP)
626             optimizer.step()
627
628             els += ls.item(); elkl += lkl.item(); elmse += lmse.item()
629             elc += lc.item(); esig += model.log_sigma_cluster.item()
630             nb += 1
631
632     scheduler.step()
633
634 H, y_tr, ids_tr = extract_embeddings(model, embed_loader)
635 if epoch % CLUSTER_UPDATE_EVERY == 0:
636     V = compute_orthogonal_transform(H, assignment, K)
637     assignment, U, n_chg = kmeans_refresh(H, K, assignment, V, seed)
638     model.cluster_centers.data.copy_(
639         torch.tensor(U, dtype=torch.float32, device=DEVICE))
640 else:
641     U_np = model.cluster_centers.detach().cpu().numpy()
642     assignment = np.argmin(np.linalg.norm(
643         H[:, None, :] - U_np[None, :, :], axis=-1), axis=1)
644     n_chg = -1
645
646 sm = eval_sentiment(model, test_loader)
647 tw = get_top_words(ids_tr, assignment, K, id2word,
648     TOP_WORDS_N, stop_words)
649 tc = topic_coherence_w2v(tw, glove_model, TOP_WORDS_N)
650 td = topic_diversity(tw, TOP_WORDS_N)
651 nmi = normalized_mutual_info_score(y_tr, assignment)
652 stab = 0. if n_chg < 0 else 1. - n_chg / N
653
654 js = (JOINT_W_F1 * sm["f1"] + JOINT_W_TC * tc

```

```

655         + JOINT_W_TD * td + JOINT_W_STAB * stab)
656
657     sig_v = esig / nb
658     prec_v = float(np.exp(-sig_v))
659     wt = " [WARMUP]" if is_warmup else ""
660
661     cur_enc_lr = optimizer.param_groups[0]["lr"]
662     cur_hd_lr = optimizer.param_groups[2]["lr"]
663
664     f"f1={sm['f1']:.4f} acc={sm['accuracy']:.4f} | "
665     f"tc={tc:.4f} td={td:.4f} nmi={nmi:.4f} | "
666     f"ls={els/nb:.4f} lkl={elkl/nb:.4f} lmse={elmse/nb:.4f} | "
667     f" =({sig_v:.3f} w={prec_v:.3f} | "
668     f" =({n_chg if n_chg>0 else '-' } | "
669     f"lr_enc={cur_enc_lr:.2e} lr_hd={cur_hd_lr:.2e} | "
670     f"js={js:.4f} | {time.time()-t0:.0fs}")
671
672     history.append(dict(
673         epoch=epoch, **sm, nmi=nmi, tc=tc, td=td,
674         ls=els/nb, lkl=elkl/nb, lmse=elmse/nb,
675         log_sigma=sig_v, joint_score=js,
676         lr_encoder=cur_enc_lr, lr_head=cur_hd_lr))
677
678     if epoch % TSNE_INTERVAL == 0 or epoch == MAX_EPOCHS:
679         plot_tsne(H, y_tr, assignment, K,
680                 os.path.join(tsne_dir, f"ep(epoch:03d).png"),
681                 f"{cname} K={K} ep={epoch}")
682
683     if js > best_score:
684         best_score = js; patience_cnt = 0
685         torch.save(dict(
686             model_state=model.state_dict(),
687             assignment=assignment,
688             U=model.cluster_centers.detach().cpu().numpy(),
689             V=V, epoch=epoch, joint_score=js,
690             log_sigma=model.log_sigma_cluster.item(),
691             ), os.path.join(save_dir, "best.pt"))
692     else:
693         patience_cnt += 1
694         if patience_cnt >= PATIENCE:
695
696     with open(os.path.join(save_dir, "history.json"), "w") as f:
697         json.dump(history, f, indent=2)
698
699     ckpt = torch.load(os.path.join(save_dir, "best.pt"),
700                      map_location=DEVICE, weights_only=False)
701     model.load_state_dict(ckpt["model_state"])
702     U_best = ckpt["U"]; best_ep = ckpt["epoch"]
703     f" log_ = {ckpt['log_sigma']:.4f}"
704
705     H_te, y_te, ids_te = extract_embeddings(model, test_loader)
706     a_te = np.argmax(np.linalg.norm(
707         H_te[:, None, :] - U_best[None, :, :], axis=-1), axis=1)
708
709     sm_te = eval_sentiment(model, test_loader)
710     tw_te = get_top_words(ids_te, a_te, K, id2word, TOP_WORDS_N, stop_words)
711     tc_te = topic_coherence_w2v(tw_te, glove_model, TOP_WORDS_N)
712     td_te = topic_diversity(tw_te, TOP_WORDS_N)
713     nmi_te = normalized_mutual_info_score(y_te, a_te)
714
715     plot_tsne(H_te, y_te, a_te, K,
716             os.path.join(save_dir, "tsne_final_test.png"),
717             f"FINAL TEST {cname} K={K} ep={best_ep}")
718
719     result = dict(
720         accuracy=sm_te["accuracy"], f1=sm_te["f1"],
721         precision=sm_te["precision"], recall=sm_te["recall"],
722         nmi_vs_sentiment=float(nmi_te),
723         tc_w2v=float(tc_te), td=float(td_te))
724
725     for k, v in result.items():
726
727     with open(os.path.join(save_dir, "result.json"), "w") as f:

```

```

728         json.dump(dict(
729             best_epoch=best_ep, log_sigma=ckpt["log_sigma"],
730             result=result,
731             top_words={str(k): v for k, v in tw_te.items()}),
732             f, indent=2)
733
734 del model, enc; torch.cuda.empty_cache(); gc.collect()
735 return result
736
737
738 def run_experiment(config="mlstm-1block", dataset="yelp",
739                   K_values=[30, 50, 80]):
740     cfg = CONFIG_REGISTRY[config]
741     cname = cfg["config_name"]
742     sdir = os.path.join(SAVE_BASE_ROOT, dataset, cfg["save_folder"])
743     os.makedirs(sdir, exist_ok=True)
744     stop = _STOP_DS.get(dataset, _BASE_STOP)
745
746
747     res = get_shared()
748     data = load_hf_dataset(dataset)
749
750     train_ds = tokenize_texts(
751         data["train_texts"], data["train_labels"],
752         res["tokenizer"], MAX_LENGTH)
753     test_ds = tokenize_texts(
754         data["test_texts"], data["test_labels"],
755         res["tokenizer"], MAX_LENGTH)
756
757     train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE,
758                             shuffle=True, num_workers=0, pin_memory=True)
759     embed_loader = DataLoader(train_ds, batch_size=BATCH_SIZE,
760                             shuffle=False, num_workers=0, pin_memory=True)
761     test_loader = DataLoader(test_ds, batch_size=BATCH_SIZE * 2,
762                             shuffle=False, num_workers=0, pin_memory=True)
763
764     all_results = {K: {} for K in K_values}
765
766     for K in K_values:
767         for run_i, seed in enumerate(SEEDS[:N_RUNS]):
768             rd = os.path.join(sdir, f"K{K}", f"run{run_i+1}_seed{seed}")
769             r = _single_run(
770                 cfg, K, seed, rd,
771                 train_loader, embed_loader, test_loader,
772                 res["glove_matrix"], res["glove_model"],
773                 res["id2word"], stop)
774             all_results[K][run_i] = r
775             torch.cuda.empty_cache()
776
777     summary = {}
778     for K in K_values:
779         summary[K] = {}
780         for m in METRICS:
781             vals = np.array([all_results[K][r][m] for r in range(N_RUNS)])
782             summary[K][m] = dict(mean=float(vals.mean()),
783                                 std=float(vals.std()),
784                                 vals=vals.tolist())
785
786     with open(os.path.join(sdir, "summary.json"), "w") as f:
787         json.dump(summary, f, indent=2)
788
789     for K in K_values:
790         for m, label in METRIC_LABELS.items():
791             mu = summary[K][m]["mean"]; sd = summary[K][m]["std"]
792
793     return summary
794
795
796 # RUN ALL
797 def run_all(configs=None, datasets=None, K_values=[30, 50, 80]):
798     if configs is None: configs = list(CONFIG_REGISTRY.keys())
799     if datasets is None: datasets = ["imdb", "yelp"]
800

```

```

801     t0 = time.time(); all_s = {}
802     for ds in datasets:
803         for cfg_key in configs:
804             s = run_experiment(config=cfg_key, dataset=ds, K_values=K_values)
805             all_s[f"{cfg_key}__{ds}"] = s
806
807     with open(os.path.join(SAVE_BASE_ROOT, "grand_summary.json"), "w") as f:
808         json.dump(all_s, f, indent=2)
809     return all_s
810
811
812 # USAGE
813 if __name__ == "__main__":
814     for k, v in CONFIG_REGISTRY.items():

```

Listing 2: Implementasi inti model JDST-xLSTM dan JDST-LSTM